KLASIFIKASI TINGKAT *ANXIETY* MENGGUNAKAN METODE *PHASE ONLY CORRELATION WITH ALL BLOCK SEARCH* DAN MODEL ARSITEKTUR *TABULAR RETRIEVAL NETWORK*

SKRIPSI

Digunakan Sebagai Syarat Maju Ujian Sarjana Terapan

Politeknik Negeri Malang

Oleh:

SYAHRUL BHUDI FERDIANSYAH NIM. 2241720167



PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

KLASIFIKASI TINGKAT *ANXIETY* MENGGUNAKAN METODE *PHASE ONLY CORRELATION WITH ALL BLOCK SEARCH* DAN MODEL ARSITEKTUR *TABULAR RETRIEVAL NETWORK*

SKRIPSI

Digunakan Sebagai Syarat Maju Ujian Sarjana Terapan

Politeknik Negeri Malang

Oleh:

SYAHRUL BHUDI FERDIANSYAH NIM. 2241720167



PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI MALANG

2026

HALAMAN PENGESAHAN

KLASIFIKASI TINGKAT ANXIETY MENGGUNAKAN METODE PHASE ONLY CORRELATION WITH ALL BLOCK SEARCH DAN MODEL ARSITEKTUR TABULAR RETRIEVAL NETWORK

Disusun oleh:

Syahrul Bhudi Ferdiansyah NIM. 2241720167

Skripsi ini telah disidangkan pada hari **Kamis, 25 Juni 2026.**

Disetujui oleh:

Pembimbing Utama	: Dr. Ulla Delfana Rosiani, ST., MT.	
	NIP. 197803272003122002	
Pembimbing Pendamping	: Wilda Imama Sabilla, S.Kom., M.Kom.	
	NIP. 199208292019032023	
Penguji Utama	: Prof. Dr. Eng. Rosa Andrie Asmara, ST., MT.	
	NIP. 198010102005011001	
Penguji Pendamping	: Mamluatul Hani'ah, S.Kom., M.Kom.	
	NIP. 199002062019032013	

Mengetahui

Ketua Jurusan Teknologi Informasi	Koordinator Program Studi S.Tr. Teknik Informatika
Mungki Astiningrum, ST., M.Kom.	Dr. Ely Setyo Astuti, S.T., M.T.
NIP. 197710302005012001	NIP. 197605152009122001

PERNYATAAN

Dengan ini saya menyatakan dengan sebenar-benarnya bahwa skripsi ini tidak terdapat karya, baik seluruh maupun sebagian, yang pernah diajukan untuk memperoleh gelar akademik di perguruan tinggi manapun. Sepanjang pengetahuan saya, dalam skripsi ini juga tidak terdapat karya atau pendapat orang lain yang pernah ditulis atau diterbitkan, kecuali yang secara tertulis telah dikutip dalam naskah ini dan disebutkan dalam daftar pustaka.

Malang, 24 Juni 2026



Syahrul Bhudi Ferdiansyah

ABSTRAK

Ferdiansyah., Syahrul Bhudi. "Klasifikasi Tingkat *Anxiety* Menggunakan Metode *Phase Only Correlation With All Block Search* Dan Model Arsitektur *Tabular Retrieval Network*". **Pembimbing: (1) Dr. Ulla Delfana Rosiani ST., MT. (2) Wilda Imama Sabilla, S.Kom., M.Kom.**

Skripsi, Program Studi Teknik Informatika, Jurusan Teknologi Informasi, Politeknik Negeri Malang, 2026.

Anxiety merupakan kondisi psikologis yang dapat memengaruhi ekspresi wajah, namun perubahan visual yang muncul sering kali bersifat halus, singkat, dan tidak mudah diamati secara langsung. Kondisi tersebut berkaitan dengan *micro-expression*, yaitu pergerakan wajah yang muncul secara tidak disadari ketika individu menekan atau mengendalikan emosi. Penelitian ini bertujuan untuk mengenali tingkat *anxiety* berdasarkan pola *micro-expression* wajah menggunakan metode *Phase Only Correlation with All Block Search* (POC-ABS) dan model *Tabular Retrieval Network* (TabR). Data diperoleh melalui perekaman video wajah pada skema *pre-post* situasi ujian, kemudian divalidasi menggunakan kuesioner dan pengelompokan tingkat kecemasan oleh pakar. Video diubah menjadi rangkaian *frame*, lalu dilakukan deteksi wajah menggunakan Dlib 68 *facial landmark* untuk mengambil *Region of Interest* (ROI) pada area mata-alis kanan, mata-alis kiri, dan mulut. Fitur pergerakan mikro diekstraksi menggunakan POC-ABS dengan *block size* 7×7 dalam bentuk X, Y, *theta*, dan *magnitude*. Fitur dari setiap ROI kemudian digabungkan menjadi data tabular sebagai masukan model TabR. Pengujian menggunakan data uji menghasilkan *accuracy* sebesar 58%, dengan *precision*, *recall*, dan F1-score masing-masing sebesar 54%, 55%, dan 55% pada kelas *anxiety* rendah, serta 60%, 59%, dan 59% pada kelas *anxiety* tinggi.. Hasil tersebut menunjukkan bahwa POC-ABS dan TabR dapat digunakan untuk memodelkan pola pergerakan mikro wajah, namun karakteristik visual *anxiety* pada *micro-expression* masih perlu dikaji lebih lanjut agar klasifikasi dapat dilakukan lebih akurat.

Kata Kunci: *Anxiety*, *Micro-expression*, POC-ABS, *Tabular Retrieval Network*, *Facial Landmark*

ABSTRACT

Ferdiansyah., Syahrul Bhudi. *"Anxiety Face Clasification Using the Phase Only Correlation with All Block Search Method and Tabular Retrieval Network Architecture Model."* ***Supervised by: (1) Dr. Ulla Delfana Rosiani, S.T., M.T., (2) Wilda Imama Sabilla, S.Kom., M.Kom.***

Thesis, Informatics Engineering Study Program, Department of Information Technology, State Polytechnic of Malang, 2026.

Anxiety is a psychological condition that can affect facial expressions. However, the visual changes that appear are often subtle, brief, and difficult to observe directly. This condition is related to micro-expression, which refers to involuntary facial movements that occur when individuals suppress or control their emotions. This study aims to recognize anxiety levels based on facial micro-expression patterns using the Phase Only Correlation with All Block Search (POC-ABS) method and the Tabular Retrieval Network (TabR) model. The data was obtained through facial video recordings using a pre-post examination scenario, then validated using a questionnaire and expert-based anxiety level grouping. The videos were converted into frame sequences, followed by face detection using Dlib 68 facial landmarks to extract Regions of Interest (ROI) from the right eye-eyebrow, left eye-eyebrow, and mouth areas. Micro-movement features were extracted using POC-ABS with a 7×7 block size in the form of X, Y, theta, and magnitude. Features from each ROI were then combined into tabular data as input for the TabR model. Testing produced an accuracy of 58%, with precision, recall, and F1-score of 54%, 55%, and 55% for the low anxiety class, and 60%, 59%, and 59% for the high anxiety class. These results indicate that POC-ABS and TabR can be used to model facial micro-movement patterns, although the visual characteristics of anxiety in micro-expressions still require further investigation to improve classification accuracy.

Keywords: Anxiety, Micro-expression, POC-ABS, Tabular Retrieval Network, Facial Landmark

KATA PENGANTAR

Puji Syukur kami panjatkan ke hadirat Allah SWT atas segala rahmat dan hidayah-Nya sehingga penulis dapat menyelesaikan skripsi dengan judul "Klasifikasi Tingkat *Anxiety* Menggunakan Metode *Phase Only Correlation With All Block Search* Dan Model Arsitektur *Tabular Retrieval Network*". Skripsi ini disusun sebagai salah satu persyaratan untuk menyelesaikan studi Sarjana Terapan, Program Studi Teknik Informatika, Jurusan Teknologi Informasi, Politeknik Negeri Malang. Penulis menyadari bahwa tanpa adanya dukungan dan kerja sama dari berbagai pihak, penyusunan laporan akhir ini tidak akan terlaksana dengan baik. Oleh karena itu, penulis ingin menyampaikan rasa terima kasih kepada:

1. Ibu Mungki Astiningrum, S.T., M.Kom., selaku Ketua Jurusan Teknologi Informasi, atas dukungan dan arahannya.
2. Ibu Dr. Ely Setyo Astuti, S.T., M.T., Ketua Program Studi DIV Teknik Informatika, atas dukungan dan arahannya.
3. Ibu Dr. Ulla Delfana Rosiani, S.T., M.T., Selaku Dosen Pembimbing Utama yang bersedia meluangkan waktu untuk memberikan bimbingan, petunjuk dan saran sehingga penyusunan laporan skripsi dan studi ini dapat terselesaikan.
4. Ibu Wilda Imama Sabilla, S.Kom., M.Kom., selaku Dosen Pembimbing Pendamping, atas waktu, bimbingan, serta saran yang sangat membantu penyelesaian skripsi ini.
5. Kedua orang tua penulis, atas cinta, doa, dan dukungan tanpa henti, baik moral maupun materiil.
6. Rekan-rekan Kos Yayuk, Discord Anunya-anu, Basecamp Batu, Koi atas dukungannya
7. Rekan-rekan mahasiswa Jurusan Teknologi Informasi angkatan 2022, atas semangat dan dukungan yang selalu diberikan.
8. Semua pihak yang telah berkontribusi dalam proses penyusunan laporan ini, baik secara langsung maupun tidak langsung, yang tidak dapat disebutkan satu per satu.

Penulis menyadari bahwa dalam penyusunan laporan akhir ini, masih banyak terdapat kekurangan dan kelemahan yang dimiliki penulis baik itu sistematika penulisan maupun penggunaan bahasa. Untuk itu penulis mengharapkan saran dan kritik dari berbagai pihak yang bersifat membangun demi penyempurnaan laporan ini. Semoga laporan ini berguna bagi pembaca secara umum dan penulis secara khusus. Akhir kata, penulis ucapkan banyak terima kasih.

Malang, 24 Juni 2026

Syahrul Bhudi Ferdiansyah

DAFTAR ISI

HALAMAN PENGESAHAN II
PERNYATAAN III
ABSTRAK IV
ABSTRACT V
KATA PENGANTAR VI
DAFTAR ISI VIII
DAFTAR GAMBAR XII
DAFTAR TABEL XIV
BAB I. PENDAHULUAN 15
1.1 Latar Belakang 15
1.2 Rumusan Masalah 17
1.3 Batasan Masalah 18
1.4 Tujuan 19
1.5 Manfaat 19
BAB II. LANDASAN TEORI 21
2.1 Studi Literatur 21
2.2 Dasar Teori 23
2.2.1 Ekspresi Wajah 23
2.2.2 Ekspresi Mikro (Micro Expression) 24
2.2.3 Dinamika Fase Micro-Expression (Spotting) 25
2.2.4 Savitzky-Golay filter 27
2.2.5 Gangguan Kecemasan (Anxiety Disorder) 27
2.2.6 Pengolahan Citra (Image Processing) 29
2.2.7 Deteksi Wajah 30
2.2.8 Dlib 30

2.2.9 Phase Only Correlation with All Block Search (POC-ABS) 31
2.2.10 Data Tabular 41
2.2.11 Facebook AI Similarity Search (FAISS) 42
2.2.12 Tabular Retrieval Network (TabR) 43
2.2.13 Question-Wise Split 47
2.2.14 Confussion Matrix 47
BAB III. METODOLOGI PENELITIAN 50
3.1 Waktu dan Tempat Penelitian 50
3.2 Teknik Pengumpulan Data 50
3.2.1 Tahapan Awal 51
3.2.2 Tahapan Perekaman 54
3.2.3 Tahapan Validasi 55
3.3 Teknik Pengolahan Data 56
3.3.1 Tahap Pengolahan Data Kuesioner dan Validasi Video 56
3.3.2 Tahap Preprocessing Citra Wajah dan Ekstraksi Fitur 57
3.4 Desain Sistem 60
3.4.1 Arsitektur Sistem 60
3.4.2 Alur Proses Sistem 61
3.5 Pembuatan Model 63
3.5.1 Persiapan dan Pra-pemrosesan Data 64
3.5.2 Pembentukan Model TabR dan Mekanisme Context 65
3.5.3 Retrieval Berbasis FAISS 65
3.5.4 Konfigurasi Model 66
3.6 Uji Coba Sistem 68
3.6.1 Skema Pengujian 68
3.6.2 Confusion Matrix 68

3.6.3 Interpretasi Hasil Evaluasi 69
BAB IV. ANALISA DAN PERANCANGAN SISTEM 70
4.1 Analisis Sistem 70
4.2 Kebutuhan Perangkat Keras 70
4.3 Kebutuhan Perangkat Lunak 71
4.4 Analisis Pengguna 71
4.5 Analisis Kebutuhan Sistem 72
4.5.1 Kebutuhan Fungsional 72
4.5.2 Kebutuhan Non Fungsional 73
4.6 Perancangan Sistem 74
4.6.1 Perancangan Use Case 74
4.6.2 Activity Diagram 76
4.6.3 Perancangan Arsitektur Sistem 78
4.6.4 Perancangan Database 80
4.6.5 Alur Proses Sistem 84
4.6.6 Perancangan Antar Muka Sistem 88
BAB V. IMPLEMENTASI DAN PENGUJIAN 90
5.1 Pengolahan *Dataset* dan Implementasi Model 90
5.1.1 Implementasi Pengolahan Data 91
5.1.2 Implementasi Model TabR 120
5.2 Pengujian 131
5.2.1 Persiapan Data Test 132
5.2.2 Skenario Pengujian 132
5.2.3 Metrik Evaluasi 133
BAB VI. HASIL DAN PEMBAHASAN 134
6.1 Hasil 134

6.2 Pembahasan 146
6.2.1 Representasi Pergerakan Micro-Expression dengan POC-ABS . 146
6.2.2 Performa Model TabR dalam Klasifikasi Anxiety 147
6.2.3 Analisis Kesalahan Prediksi Model 149
6.2.4 Evaluasi Metrik Precision, Recall, dan F1-Score..... 151
BAB VII. KESIMPULAN DAN SARAN 153
7.1 Kesimpulan..... 153
7.2 Saran 153
DAFTAR PUSTAKA 155

DAFTAR GAMBAR

Gambar 2.1 Ekspresi Wajah 24
Gambar 2.2 *Micro-expressions* 25
Gambar 2.3 Grafik *onset apex offset* 25
Gambar 2.4 *Facial Landmark* Dlib 31
Gambar 2.5 Proses POC deteksi pergerakan dari gambar contoh 2.2 36
Gambar 2.6 Arsitektur TabR 44
Gambar 2.7 *Encoder dan Decoder* 45
Gambar 2.8 *Retreival Module* 46
Gambar 2.9 *Confusion Matrix* 48
Gambar 3.1 Alur Pengumpulan Data Primer Penelitian 51
Gambar 3.2 Proses Pengambilan Data 55
Gambar 3.3 Contoh *Dataset* Dalam Bentuk *Frame* 55
Gambar 3.4 *Frame* Video 57
Gambar 3.5 Area Wajah Yang Diambil 58
Gambar 3.6 Pergerakan Yang Ditangkap POC-ABS 58
Gambar 3.7 Contoh Tiap Fase *Onset*, *Apex*, Dan *Offset* 59
Gambar 3.8 Contoh Data Yang Dihasilkan 60
Gambar 3.9 Diagram Alur 61
Gambar 3.10 Alur Model TabR 64
Gambar 4.1 *Use Case* Diagram 74
Gambar 4.2 *Activity* Diagram Prediksi *Anxiety* 77
Gambar 4.3 Arsitektur Sistem 78
Gambar 4.4 *Entity Relationship Diagram* 80
Gambar 4.5 Alur Perancangan Sistem 85
Gambar 4.6 Deteksi 68 *Facial Landmark* menggunakan Dlib 85
Gambar 4.7 *Cropping* Komponen Wajah 86
Gambar 4.8 Halaman *Input* Video 88
Gambar 4.9 Tampillan Sebelum Hasil Muncul 88
Gambar 4.10 Tampilan Setelah Hasil Muncul 89
Gambar 4.11 Tampilan *Chart Spotting* 89

Gambar 5.1 Contoh *Cropping* ROI 93
Gambar 5.2 Gambar Sinyal magnitude *raw* 101
Gambar 5. 3 Gambar Sinyal magnitude hasil *smoothing* 101
Gambar 5. 4 Gambar Perbandingan Sinyal *Raw* dan *Smoothed* 102
Gambar 5.5 Grafik Hasil *Spotting* 106
Gambar 5.6 *Case* 1 validasi algoritma POC-ABS 115
Gambar 5.7 *Case* 2 validasi algoritma POC-ABS 116
Gambar 5.8 Visualisasi Data *Real* POC-ABS 116
Gambar 5.9 Parameter yang digunakan 124
Gambar 6.1 Gambar Hasil video menjadi *frame* 136
Gambar 6.2 Gambar Contoh Hasil *Cropping Region of Interest* (ROI) 136
Gambar 6.3 Contoh Bentuk Data Fitur Hasil Ekstraksi POC-ABS 137
Gambar 6.4 Perbandingan Hasil *Spotting* dengan *Ground Truth* CASME 138
Gambar 6.5 Hasil *Spotting* pada Video *Anxiety* Tinggi 139
Gambar 6.6 Hasil *Spotting* pada Video *Anxiety* Rendah 139
Gambar 6.7 Grafik Pelatihan Model TabR 143
Gambar 6.8 *Confusion Matrix Training* dan *Validation* 144
Gambar 6.9 *Confussion Matrix Datatest* 145

DAFTAR TABEL

Tabel 2.1 Penelitian Terakhir 21
Tabel 2.2 Contoh hasil ekstraksi fitur 40
Tabel 3.1 Pertanyaan Kuesioner *Faces Anxiety Scale* (FAS) 52
Tabel 3.2 Kategorisasi dan skor 56
Tabel 3.3 Ruang Pencarian *Hyperparameter* 66
Tabel 3.4 Konfigurasi Akhir Model TabR 67
Tabel 4.1 Perangkat yang digunakan 70
Tabel 4.2 Rincian Kebutuhan Perangkat Lunak 71
Tabel 4.3 Rincian kebutuhan fungsional 72
Tabel 4.4 Rincian kebutuhan non-fungsional 73
Tabel 4.5 Rincian *Use case* Prediksi *Anxiety* 75
Tabel 4.6 Deskripsi Tabel Video *Predictions* 81
Tabel 4.7 Deskripsi Video *Prediction Event*s 83
Tabel 4.8 Contoh bentuk hasil data POC-ABS 87
Tabel 6.1 Komposisi *Dataset* 134
Tabel 6.2 Tabel Komposisi *Datatest* 135
Tabel 6.3 Contoh Fitur Hasil Ekstraksi POC-ABS per Blok 137
Tabel 6.4 Contoh Bentuk Data Tabular Hasil Ekstraksi Fitur 140
Tabel 6.5 Ruang Pencarian Hyperparameter Model TabR 141
Tabel 6.6 Parameter Tetap Pelatihan Model TabR 142
Tabel 6.7 Konfigurasi Terbaik Hasil *Hyperparameter Tuning* 142
Tabel 6.8 Hasil Performa Model Terbaik pada Tahap Tuning 143
Tabel 6.9 *Metrik* Evaluasi *Datatest* 145
Tabel 6.10 Ringkasan Hasil *Spotting* 147
Tabel 6.11 Ringkasan Performa Model TabR 148
Tabel 6.12 Ringkasan Kesalahan Prediksi *Datatest* 149
Tabel 6.13 Statistik *Event* Benar dan Salah Prediksi 150
Tabel 6.14 Ringkasan Perbandingan Performa Model 151
Tabel 6.15 Metrik Evaluasi *Datatest* 151

BAB I. PENDAHULUAN

1.1 Latar Belakang

Ekspresi wajah merupakan salah satu bentuk komunikasi yang paling kuat dan universal, serta sangat terkait dengan kondisi mental, sikap, dan niat seseorang. Di samping ekspresi wajah biasa (*macro-expressions*) yang umum terlihat sehari-hari, emosi juga dapat muncul dalam bentuk khusus ketika seseorang berada pada kondisi tertentu, yaitu *micro-expressions* (Y. Li dkk., 2022). *Micro-expressions* merupakan gerakan wajah yang sangat singkat dan tidak disengaja ketika seseorang berusaha menekan atau menyembunyikan emosi sebenarnya, sehingga memberikan gambaran yang lebih autentik dibandingkan *macro-expressions* yang dapat disamarkan (J. Chen dkk., 2025). Dalam konteks kategorisasi ekspresi wajah, terdapat berbagai model yang pada umumnya merujuk pada emosi dasar yang diperkenalkan oleh Ekman (1992), di mana tipe emosi primer dalam model-model yang banyak digunakan terdiri dari lima, enam, atau tujuh kategori, mencakup *happiness, sadness, anger, disgust, surprise,* dan *fear* (Coppini dkk., 2023). Dalam konteks ini, *micro-expressions* menjadi relevan untuk dikaji lebih lanjut ketika dikaitkan dengan emosi kompleks seperti *anxiety*.

Secara psikologis, *anxiety* berbeda dari fear karena dipicu oleh ancaman yang tidak pasti dan memunculkan kewaspadaan berkelanjutan, sedangkan *fear* berkaitan dengan ancaman yang jelas dan langsung (American Psychiatric Association, 2022; Watson dkk., 2022). Dalam konteks sosial, individu dengan *anxiety* cenderung mengendalikan ekspresi emosinya, sehingga sinyal visual kecemasan jarang muncul secara eksplisit dan lebih sering termanifestasi sebagai gerakan wajah yang tidak disengaja, sangat halus, dan berlangsung singkat, selaras dengan karakteristik *micro-expressions* (Lei dkk., 2024; Zhao dkk., 2023). Kondisi ini menjadikan pengenalan *anxiety* sebagai tantangan visual yang bersifat *spasio-temporal* karena informasi penting terletak pada dinamika pergerakan antar *frame*. Namun, pengembangan model pengenalan *micro-expression* masih terkendala keterbatasan data dan anotasi, di mana mayoritas *dataset* standar

mengadopsi taksonomi emosi dasar tanpa label *anxiety* eksplisit (Zhang & Zhang, 2022). Keterbatasan ini memaksa emosi kompleks seperti *anxiety* untuk dilebur ke dalam kelas general *'Others'*, yang pada *dataset* standar seperti CASME II terbukti menampung proporsi sampel sangat signifikan hingga mencapai 41,3% (Ben dkk., 2021), sehingga karakteristik visual *anxiety* menjadi tidak terdefinisi dengan jelas dan menyulitkan pembelajaran representasi yang spesifik dan robust (Zhao dkk., 2023). Oleh karena itu, pengumpulan dan anotasi data *micro-expression* secara mandiri menjadi penting untuk memperoleh representasi *anxiety* yang lebih terkontrol, dengan prosedur pengambilan dan pelabelan dijelaskan pada bagian metodologi.

Sejalan dengan keterbatasan anotasi tersebut, pengembangan metode dalam penelitian *micro-expression* selama ini masih berfokus pada emosi dasar yang telah tersedia dalam *dataset* standar. Penelitian SOTA umumnya berfokus pada peningkatan akurasi emosi dasar melalui pengembangan arsitektur *spatiotemporal* dan mekanisme atensi, dengan evaluasi pada *dataset* seperti CASME II, SAMM, dan SMIC yang masih mengadopsi taksonomi emosi dasar (Bin Talib dkk., 2025; J. Li dkk., 2025). Meskipun pendekatan SOTA mampu mencapai performa tinggi pada klasifikasi emosi dasar, pendekatan tersebut secara konseptual masih dibatasi oleh asumsi emosi dasar yang mendasari anotasi *dataset*. Hal ini menjadikan pendekatan berbasis emosi dasar kurang sesuai untuk mengkaji *anxiety* yang bersifat implisit dan tidak terdefinisi dalam taksonomi *dataset* standar, sehingga menegaskan adanya celah metodologis dalam pengkajian *micro-expression anxiety*.

Perkembangan penelitian *anxiety* dalam beberapa tahun terakhir menunjukkan pergeseran dari pendekatan subjektif berbasis kuesioner dan wawancara klinis menuju pendekatan objektif berbasis analisis sinyal visual, sekaligus mencerminkan perubahan cara pandang terhadap *anxiety* yang tidak lagi diperlakukan sebagai kategori emosi diskrit, melainkan sebagai kondisi psikologis yang terefleksi melalui pola perilaku dan dinamika ekspresi wajah yang bersifat halus dan berkelanjutan, khususnya

melalui analisis perubahan mikro antar-*frame* pada data ekspresi wajah berbasis video (S. Chen & Zhong, 2025; Ghafarfaraji, 2025). Pergeseran ini didorong oleh keterbatasan metode subjektif yang rentan terhadap bias pelaporan, keterlambatan evaluasi, serta kecenderungan individu dengan *anxiety* untuk menyamarkan kondisi emosionalnya (S. Chen & Zhong, 2025). Oleh karena itu, pendekatan pengenalan wajah yang memodelkan *anxiety* sebagai pola pergerakan mikro wajah secara *spasio-temporal* menjadi semakin relevan untuk menghasilkan representasi yang lebih valid dan kontekstual.

Untuk menjawab celah penelitian tersebut, penelitian ini mengusulkan pendekatan representasi pergerakan mikro-wajah yang memodelkan *anxiety* berdasarkan karakteristik pergerakan antar-*frame* (spasio-temporal), bukan berdasarkan kategori emosi *dataset* standar. *Phase Only Correlation with All Block Search* (POC-ABS) digunakan untuk mengekstraksi pergeseran mikro antar-*frame* secara eksplisit melalui analisis fase, sehingga mampu menangkap kebocoran gerak halus yang muncul ketika emosi ditekan (Rosiani dkk., 2020). Hasil ekstraksi menggunakan metode POC-ABS berupa nilai-nilai numerik yang merepresentasikan arah dan besar pergerakan mikro wajah pada setiap area pengamatan. Dengan demikian, setiap sampel ekspresi wajah direpresentasikan sebagai sekumpulan fitur numerik yang terstruktur, sehingga proses klasifikasi difokuskan pada pembedaan pola pergerakan mikro antar-sampel. Representasi pergerakan tersebut selanjutnya dimodelkan menggunakan arsitektur *Tabular Retrieval Network* (TabR), yang dirancang untuk menangani data *tabular* dan memanfaatkan hubungan kedekatan antar-sampel dalam proses klasifikasi (Pasaribu dkk., 2024).

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, dapat diidentifikasi beberapa permasalahan penelitian sebagai berikut:

1. Bagaimana fitur pergerakan halus pada area wajah yang diekstraksi menggunakan metode POC-ABS mampu membedakan kondisi *anxiety* tinggi dan *anxiety* rendah ?

2. Bagaimana performa model TabR dalam mengklasifikasikan kondisi *anxiety* tinggi dan *anxiety* rendah berdasarkan fitur POC-ABS yang diekstraksi dari data penelitian berdasarkan metrik *accuracy*, *precision*, *recall*, dan F1-score?

1.3 Batasan Masalah

Agar penelitian ini lebih terarah dan fokus, maka penulis menetapkan batasan-batasan masalah sebagai berikut:

1. ***Dataset*:** Penelitian ini menggunakan *dataset micro-expression anxiety* yang dikumpulkan secara mandiri melalui skema pre–post situasi ujian dengan dukungan kuesioner, pengelompokan label tingkat kecemasan (*anxiety*) berdasarkan oleh pakar Prof. Dr. Adi Atmoko, M.Pd., seorang dosen dari Fakultas Ilmu Pendidikan yang memiliki keahlian di bidang bimbingan dan konseling..
2. **Fokus Representasi Emosi:** Penelitian ini difokuskan pada pengenalan kondisi *anxiety* tinggi dan *anxiety* rendah, tanpa mengacu pada taksonomi emosi dasar seperti *happiness, anger, fear,* dan emosi dasar lainnya.
3. **Ekstraksi Pergerakan Mikro Wajah:** Ekstraksi pergerakan mikro wajah dibatasi pada penggunaan metode *Phase Only Correlation with All Block Search (POC-ABS)* untuk merepresentasikan pergeseran mikro antar-*frame*, tanpa membandingkan dengan metode ekstraksi gerakan lainnya.
4. **Area Pengamatan:** Penelitian hanya difokuskan untuk mengamati dan mengambil nilai pada area mata-alis kanan, mata-alis kiri, dan mulut
5. **Model Klasifikasi:** Pemodelan dan klasifikasi data dilakukan menggunakan *Tabular Retrieval Network (TabR)* berbasis data tabular, tanpa membahas atau membandingkan dengan model *deep learning* lain seperti CNN atau transformer berbasis citra.
6. **Ruang Lingkup Sistem dan Evaluasi:** Penelitian ini dibatasi pada pemrosesan data citra wajah dan evaluasi performa model menggunakan metrik *Accuracy*, *Precision*, *Recall*, dan *F1-Score*.

7. **Skenario *Real-Time***: Penelitian ini belum membahas penerapan sistem secara *real-time*, sehingga proses pengenalan anxiety dilakukan secara offline menggunakan video wajah yang telah direkam sebelumnya.

1.4 Tujuan

Berdasarkan rumusan masalah di atas, tujuan dari penelitian ini adalah:

1. Dapat mengidentifikasi dan menganalisis kemampuan fitur pergerakan halus pada area wajah yang diekstraksi menggunakan metode POC-ABS dalam membedakan *anxiety* tinggi dan *anxiety* rendah.
2. Dapat mengevaluasi performa model TabR dalam mengklasifikasikan kondisi *anxiety* tinggi dan *anxiety* rendah berdasarkan fitur POC-ABS yang diekstraksi dari data penelitian.
3. Dapat mengevaluasi kinerja model klasifikasi yang diusulkan berdasarkan metrik akurasi, precision, recall, dan F1-score.

1.5 Manfaat

Penelitian ini diharapkan dapat memberikan manfaat sebagai berikut:

1. **Kontribusi terhadap Analisis *Anxiety* Berbasis Ekspresi Wajah**

   Penelitian ini memberikan kontribusi dalam pengembangan analisis *micro-expression anxiety* pada wajah yang belum banyak dikaji secara eksplisit dalam penelitian *micro-expression* sebelumnya.

2. **Referensi Pengembangan *Dataset Micro-Expression Anxiety***

   Dataset pre–post exam yang dikumpulkan secara mandiri melalui pengisian kuesioner dan perekaman ekspresi wajah dapat menjadi referensi awal bagi penelitian selanjutnya dalam membangun dan mengolah data *micro-expression anxiety* yang bersifat natural dan kontekstual.

3. **Pendekatan Teknis untuk Deteksi *Micro-Expression Anxiety***

   Pendekatan teknis yang mengombinasikan ekstraksi pergerakan mikro wajah menggunakan *Phase Only Correlation with*

All Block Search (POC-ABS) dan pemodelan klasifikasi berbasis data tabular menggunakan *Tabular Retrieval Network (TabR)* dapat menjadi acuan metodologis bagi penelitian selanjutnya dalam mendeteksi emosi wajah yang bersifat halus dan sulit diamati secara visual.

BAB II. LANDASAN TEORI

2.1 Studi Literatur

Beberapa Penelitian yang dijadikan rujukan dalam penelitian ini adalah sebagai berikut:

Tabel 2.1 Penelitian Terakhir

Judul Jurnal	Tahun	Metode	Hasil Penelitian
A Novel Approach on Motion Estimation for *Micro-Expression* Recognition Using Phase Only Correlation with All Block Search (POC-ABS) (Rosiani dkk., 2020)	2020	POC-ABS dan SVM	Metode POC-ABS efektif dalam menangkap pergerakan mikro-ekspresi yang sangat halus secara cepat dan presisi, khususnya pada area alis yang menjadi indikator utama kebocoran ekspresi. Dengan rata-rata waktu komputasi sekitar 0,03 detik serta tingkat akurasi yang tinggi ketika dikombinasikan dengan klasifier SVM dan KNN, metode ini dinilai sesuai untuk penerapan pengenalan mikro-ekspresi secara *real*-time.
TabNet: Attentive Interpretable Tabular Learning (Arik & Pfister, 2021)	2021	TabNet	TabNet menunjukkan performa kompetitif pada data tabular, dengan akurasi hingga 96,99% pada beberapa *dataset* tabular dan peningkatan

			konsisten dibanding metode *tree-based*, sambil mempertahankan interpretabilitas berbasis seleksi fitur bertahap.
Micro-expression recognition model based on TV-L1 optical flow method and improved ShuffleNet (Liu dkk., 2022)	2022	TV-L1 *Optical Flow*, ShuffleNetV2	Model TV-L1 *optical flow* dengan ShuffleNetV2 dan improved ViT mencapai UAR ≈0,70 dan UF1 ≈0,73 secara konsisten pada berbagai *dataset micro-expression*, dengan waktu inferensi sekitar 37,9 ms per sampel.
TabR: Tabular Deep Learning Meets Nearest Neighbors (Gorishniy dkk., 2024)	2024	TabR	Pada *benchmark* OpenML-CC18, TabR menunjukkan peringkat rata-rata terbaik di antara model *deep learning* tabular dan pada sejumlah *dataset* melampaui GBDT, terutama pada skenario data kecil hingga menengah.
Tabular Data Classification and Regression: XGBoost or Deep Learning with Retrieval-Augmented Generation (Pasaribu dkk., 2024)	2024	XGBoost, TabNet, dan TabR	Model TabR efektif meningkatkan kinerja klasifikasi data tabular melalui mekanisme *retrieval-augmented* yang memanfaatkan konteks antar-sampel.

			Evaluasi pada *dataset benchmark* OpenML menunjukkan bahwa TabR unggul dibandingkan XGBoost dan TabNet pada beberapa skenario, dengan nilai AUC hingga 0,98 dan akurasi sekitar 0,93 pada *dataset electricity*, sehingga sesuai untuk klasifikasi data tabular dengan kompleksitas fitur yang beragam.

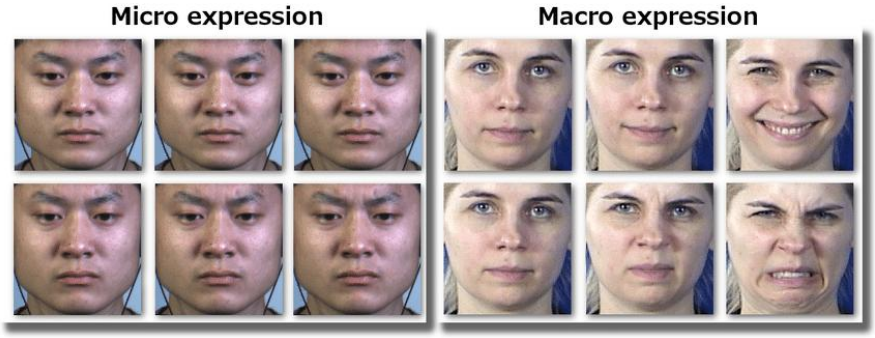
2.2 Dasar Teori

2.2.1 Ekspresi Wajah

Ekspresi wajah merupakan salah satu saluran komunikasi nonverbal utama yang digunakan manusia untuk menyampaikan kondisi emosional. Analisis ekspresi wajah menjadi topik penting dalam berbagai penelitian karena mampu merepresentasikan keadaan psikologis seseorang secara langsung (Zhao dkk., 2023). Umumnya, ekspresi wajah muncul secara tidak disengaja sebagai respons terhadap perasaan atau emosi yang dirasakan oleh individu (Dawel dkk., 2025).

Ekspresi wajah dapat dikenali melalui perubahan halus pada satu atau beberapa fitur wajah tertentu. Perubahan ini umumnya terjadi pada area seperti alis, kelopak mata, dan mulut, yang berperan penting dalam menyampaikan informasi emosional (Kim dkk., 2022). Keberadaan variasi perubahan pada fitur-fitur wajah tersebut menunjukkan bahwa ekspresi wajah tidak muncul dalam satu bentuk yang seragam, melainkan memiliki

beragam pola yang dapat dibedakan berdasarkan karakteristik kemunculannya.



Gambar 2.1 Ekspresi Wajah

Sumber : (Allaert dkk., 2022)

Berdasarkan karakteristik kemunculannya, ekspresi wajah dapat dibedakan menjadi ekspresi makro dan ekspresi mikro. Ekspresi makro merupakan bentuk ekspresi wajah yang mudah diamati dan dibedakan karena melibatkan perubahan pada hampir seluruh area wajah**.** Berbeda dari ekspresi makro, ekspresi mikro merupakan gerakan otot wajah yang terjadi secara tidak disengaja dengan durasi yang sangat singkat dan intensitas gerakan yang halus, berlangsung sekitar 0.04 hingga 0.2 detik, serta muncul pada beberapa bagian wajah sebagai indikasi adanya emosi yang disembunyikan oleh individu (Choirina dkk., 2022).

2.2.2 Ekspresi Mikro (Micro Expression)

Ekspresi mikro (*micro-expression*) merupakan ekspresi wajah yang muncul secara tidak disengaja, berdurasi sangat singkat, dan berintensitas rendah. Ekspresi ini terjadi ketika individu berusaha menyembunyikan atau menekan emosi yang sebenarnya, sehingga gerakan wajah yang muncul bersifat spontan dan sulit dikendalikan. Berbeda dengan ekspresi makro yang dapat dimanipulasi secara sadar, ekspresi mikro mampu mengungkap emosi autentik yang tersembunyi melalui perubahan otot wajah yang sangat halus dan berlangsung dalam waktu singkat (Ben dkk., 2021).

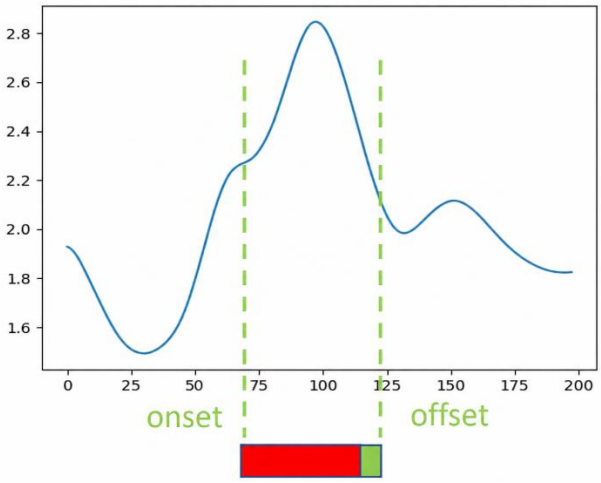


Gambar 2.2 *Micro-expressions*

Sumber: (Y. Wang dkk., 2015)

Secara empiris, ekspresi mikro tidak hanya dibedakan oleh durasinya yang singkat, tetapi juga oleh intensitas aktivasi otot wajah yang sangat rendah dan bersifat lokal. Intensitas pergerakan otot pada ekspresi mikro terbukti secara signifikan lebih kecil dibandingkan ekspresi makro, sehingga ekspresi yang muncul sering kali tidak membentuk pola wajah yang utuh dan sulit dikenali secara visual (Lu dkk., 2022). Selain itu, ekspresi mikro bersifat tidak disengaja dan muncul dalam konteks interaksi antara sistem motorik emosional dan sistem kontrol volunter wajah, yang menyebabkan pergerakan otot tidak selalu terorganisasi secara penuh seperti pada ekspresi sadar (Dong dkk., 2022). Dengan demikian, ekspresi mikro lebih tepat dipahami sebagai indikator kebocoran emosi yang kuat.

2.2.3 Dinamika Fase Micro-Expression (Spotting)



Gambar 2.3 Grafik *onset apex offset*

Sumber: (Yu dkk., 2024)

Dalam analisis *micro-expression* berbasis video, pengenalan tidak dilakukan pada keseluruhan rekaman secara acak, melainkan difokuskan pada durasi spesifik di mana ekspresi tersebut muncul. Siklus pergerakan

otot wajah pada ekspresi mikro secara umum bergeser melalui urutan fase *neutral–onset–apex–offset–neutral* (Bruni & Vitulano, 2021). Proses identifikasi dan pelokalan fase kritis ini dalam rangkaian *frame* video dikenal dengan istilah *spotting*. *Micro-expression spotting* merupakan proses untuk menentukan segmen kemunculan ekspresi mikro pada video panjang, yaitu rentang *frame* yang dibatasi oleh *onset* dan *offset* (J. Wang dkk., 2024).

Karakteristik temporal dari ekspresi mikro tersebut didefinisikan ke dalam tiga fase utama sebagai berikut:

- Fase *Onset*: Merupakan titik awal dimulainya kontraksi otot wajah. Fase ini ditandai dengan *frame* di mana intensitas perubahan wajah mulai meningkat dari posisi netral (Bruni & Vitulano, 2021).
- Fase *Apex*: Merupakan instans atau titik di mana pergerakan wajah mencapai tingkat intensitas tertinggi (*maximum expression*) relatif terhadap fase netral (Bruni & Vitulano, 2021; Lai Yee dkk., 2022). *Frame apex* memuat informasi visual paling representatif untuk dianalisis oleh model klasifikasi.
- Fase *Offset*: Merupakan fase akhir atau disipasi, di mana kontraksi otot wajah mulai mereda dan pergerakan berkurang hingga wajah kembali ke posisi netral (Bruni & Vitulano, 2021; Lai Yee dkk., 2022).

Dalam proses *spotting*, deteksi kandidat *event* dapat dilakukan dengan menganalisis puncak pergerakan pada sinyal. Ambang batas adaptif dapat dihitung menggunakan rata-rata dan standar deviasi, seperti rumus $T = Cmean + p \times Cstd$ yang digunakan untuk menentukan puncak pergerakan signifikan dari semua puncak dalam satu video (J. Wang dkk., 2024). Dalam penelitian ini, konsep *threshold* adaptif tersebut digunakan sebagai dasar untuk menentukan kandidat *event*.

Berdasarkan kandidat *event* tersebut, identifikasi ketiga fase ini sangat krusial dalam sistem pengenalan wajah otomatis. Dengan mengisolasi rentang *frame* dari *onset* hingga *offset*, sistem dapat mereduksi data yang tidak relevan.

2.2.4 Savitzky-Golay filter

Savitzky-Golay *filter* merupakan metode penghalusan sinyal yang digunakan untuk mengurangi fluktuasi kecil pada data tanpa mengubah pola utama sinyal secara berlebihan. Dalam pengolahan citra wajah, metode ini juga digunakan pada data wajah yang dipengaruhi oleh perubahan pencahayaan, perubahan ekspresi, variasi pose, dan bagian wajah yang tertutup (Kagawade & Angadi, 2021).

Sebelum proses deteksi puncak, sinyal pergerakan dapat dihaluskan terlebih dahulu untuk mengurangi fluktuasi kecil yang tidak merepresentasikan perubahan ekspresi. Savitzky-Golay *filter* dapat digunakan karena mampu mengurangi *noise* pada sinyal tanpa menghilangkan bentuk utama serta puncak gelombang, sehingga pola pergerakan tetap dapat dianalisis dalam proses *spotting* (Han dkk., 2023).

2.2.5 Gangguan Kecemasan (Anxiety Disorder)

Menurut Darajat dalam (Nurshabrina dkk., 2024), kecemasan dipahami sebagai perasaan antisipatif terhadap kemungkinan terjadinya situasi yang menakutkan, yang mendorong kesiapan individu untuk bertindak, namun pada kenyataannya ancaman tersebut tidak benar-benar terjadi dan tidak berkaitan dengan objek atau kondisi tertentu yang perlu dihindari. Secara psikologis, kondisi ini berbeda dari *fear*, karena *fear* muncul sebagai respons terhadap ancaman yang nyata dan spesifik serta memicu reaksi cepat seperti *fight or flight*, sedangkan *anxiety* berkaitan dengan ancaman yang tidak terdefinisi secara jelas dan menghasilkan kewaspadaan yang bersifat berkelanjutan (Watson dkk., 2022).

Kecemasan menjadi kondisi yang bersifat abnormal ketika intensitas dan durasinya tidak sebanding dengan ancaman yang dihadapi atau muncul tanpa sebab yang jelas, sehingga tidak lagi dapat dipahami sebagai respons adaptif terhadap lingkungan. Pada kondisi tersebut, kecemasan dapat berkembang menjadi gangguan psikologis yang mengganggu fungsi sosial, pekerjaan, serta aktivitas sehari-hari individu (Oktamarin dkk., 2022). Kondisi kecemasan yang bersifat abnormal tersebut tercermin pada tingginya prevalensi gangguan kecemasan di Indonesia menunjukkan

bahwa kecemasan abnormal merupakan masalah kesehatan mental yang serius, dengan angka 9,8% pada remaja dan dewasa pada Data Riset Kesehatan Dasar tahun 2018 yang meningkat menjadi 26,7% berdasarkan survei I-NAMHS 2022, terutama pada kelompok remaja dan pelajar (Dewi dkk., 2025).

Dalam interaksi sosial, kecemasan yang berlangsung terus-menerus tidak selalu diekspresikan secara jelas melalui ekspresi wajah yang mudah dikenali. Individu dengan *anxiety* cenderung melakukan regulasi emosi dengan menahan ekspresi negatif guna menjaga citra diri dan menyesuaikan diri dengan lingkungan sosial, sehingga tanda-tanda visual kecemasan sering kali tidak tampak secara eksplisit (Lei dkk., 2024). Sebagai konsekuensinya, respons emosional yang berkaitan dengan kecemasan lebih sering muncul dalam bentuk aktivasi otot wajah yang bersifat tidak disengaja, berintensitas sangat rendah, dan berdurasi singkat. Pola kemunculan tersebut sejalan dengan karakteristik *micro-expressions*, yaitu ekspresi wajah yang timbul ketika individu berupaya menyembunyikan emosi yang sesungguhnya, namun tetap menunjukkan kebocoran emosi secara spontan (Zhao dkk., 2023). Dalam kerangka *micro-expressions*, perubahan tersebut tampak pada area berikut:

1. Area mata, khususnya perubahan halus di sekitar bagian bawah mata yang mencerminkan peningkatan kewaspadaan.
2. Area sudut bibir (*lip corner*), yang menunjukkan ketegangan ringan sebagai respons terhadap ketidaknyamanan emosional.

Perubahan pada area-area tersebut bersifat sementara dan sulit dikenali secara kasat mata, sehingga lebih relevan dianalisis dalam kerangka *micro-expressions* dan pendekatan visual yang sensitif terhadap perubahan kecil pada fitur wajah (Sharma dkk., 2024).

Kecemasan tidak hanya muncul sebagai respons terhadap ancaman yang akan dihadapi, tetapi juga dapat berlanjut setelah situasi pemicu berakhir. Dalam konteks akademik, dinamika kecemasan ujian mencakup beberapa fase, yaitu sebelum ujian, periode setelah ujian hingga pengumuman hasil, serta fase pasca pengumuman nilai. Penelitian

menunjukkan bahwa individu tidak berhenti memproses pengalaman ujian ketika ujian selesai, melainkan masih terlibat dalam pemikiran dan evaluasi berulang terhadap performa yang telah dilakukan. Bahkan, *post-event processing* ditemukan muncul secara bermakna setelah situasi ujian dan berhubungan dengan tingkat kecemasan yang dialami individu (Möcklinghoff dkk., 2024). Temuan ini menunjukkan bahwa gangguan kecemasan melibatkan proses kognitif yang dapat berlangsung baik sebelum maupun setelah stimulus pemicu kecemasan berakhir.

2.2.6 Pengolahan Citra (Image Processing)

Pengolahan citra merupakan bidang komputasi yang berfokus pada pemrosesan dan analisis citra digital untuk memperoleh informasi visual yang bermakna. Citra digital dipahami sebagai representasi dua dimensi dari objek visual yang tersusun atas kumpulan piksel dengan nilai intensitas tertentu, sehingga dapat diolah secara komputasional. Dalam praktiknya, pengolahan citra mencakup berbagai teknik seperti peningkatan kualitas citra, restorasi, segmentasi, ekstraksi fitur, dan klasifikasi guna meningkatkan interpretabilitas citra oleh manusia maupun sistem komputer (Archana & Jeevaraj, 2024). Dalam perkembangannya, pengolahan citra juga menjadi bagian dari kecerdasan buatan karena memanfaatkan citra digital sebagai masukan utama untuk proses analisis dan pengambilan keputusan berbasis komputasi.

Berdasarkan jenis datanya, pengolahan citra dibedakan menjadi pengolahan citra statis dan pengolahan citra bergerak. Citra statis merepresentasikan objek visual pada satu waktu tertentu, sedangkan citra bergerak tersusun atas rangkaian citra digital (*frames*) yang diperoleh secara berurutan dari aliran video. Setiap *frame* pada citra bergerak diperlakukan sebagai citra digital yang dapat dianalisis secara individual maupun dengan membandingkannya terhadap *frame* lain yang berurutan dalam domain waktu, sehingga memungkinkan identifikasi perubahan visual yang terjadi antar *frame* (Sinulingga & Kong, 2023).

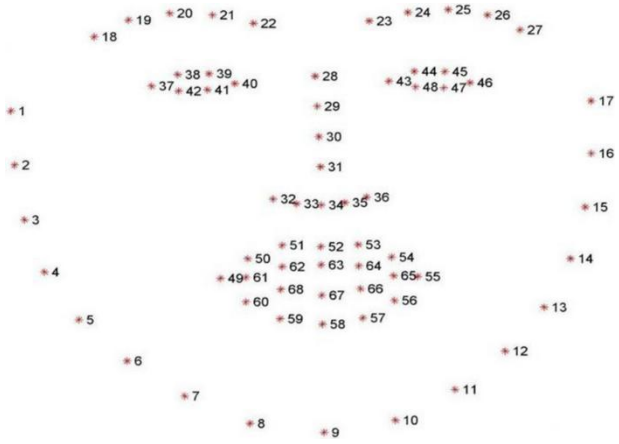
2.2.7 Deteksi Wajah

Deteksi wajah merupakan tahap awal dalam pemrosesan citra wajah yang bertujuan untuk mengidentifikasi keberadaan wajah manusia dan memisahkan area wajah dari latar belakang pada citra atau video. Tahap ini penting karena memastikan bahwa analisis selanjutnya hanya difokuskan pada wilayah wajah yang relevan sebelum dilakukan pemrosesan lanjutan (Y. Li dkk., 2022). Dalam konteks analisis *micro-expression*, hasil deteksi wajah digunakan sebagai *facial region* yang menjadi masukan untuk proses lanjutan seperti pelacakan titik fitur, ekstraksi pergerakan otot wajah, serta analisis perubahan visual yang bersifat sangat halus dan singkat (Choirina dkk., 2022).

Seiring perkembangan teknik komputasi, deteksi wajah tidak hanya diterapkan pada citra statis, tetapi juga pada citra bergerak atau video yang tersusun atas rangkaian *frame*. Dalam konteks ini, wajah diekstraksi dari urutan citra wajah sebagai dasar pemrosesan lanjutan, sehingga memungkinkan analisis wajah dilakukan secara konsisten pada setiap bagian dari rangkaian citra tersebut untuk mendukung analisis ekspresi wajah yang bersifat dinamis dan berlangsung singkat (Liu dkk., 2022).

2.2.8 Dlib

Dlib merupakan *library open-source* yang menyediakan berbagai fungsi dan algoritma untuk pengolahan citra dan visi komputer, khususnya pada pemrosesan wajah. Dlib digunakan untuk mengekstraksi dan menganalisis karakteristik wajah dari citra atau video, seperti deteksi wajah dan pelokalan titik-titik *landmark* wajah, sehingga mendukung proses pemrosesan lanjutan dalam analisis wajah (Sejati & Mardhiyyah, 2021). Melalui pendekatan *facial landmark*, Dlib memungkinkan identifikasi komponen wajah seperti mata, alis, dan mulut sebagai bagian dari tahap pra-pemrosesan sebelum dilakukan analisis lanjutan (Nur Cahyo dkk., 2023). Representasi *facial landmark* pada Dlib ditunjukkan pada gambar berikut:



Gambar 2.4 *Facial Landmark* Dlib

Sumber: (Nur Cahyo dkk., 2023)

Gambar tersebut menunjukkan representasi *facial landmark* yang digunakan oleh Dlib, di mana setiap titik merepresentasikan posisi geometris tertentu pada wajah. Titik-titik *landmark* ini digunakan untuk memetakan struktur wajah, seperti area alis, mata, hidung, dan mulut, sehingga pergerakan atau perubahan pada bagian wajah dapat dianalisis secara lebih terperinci.

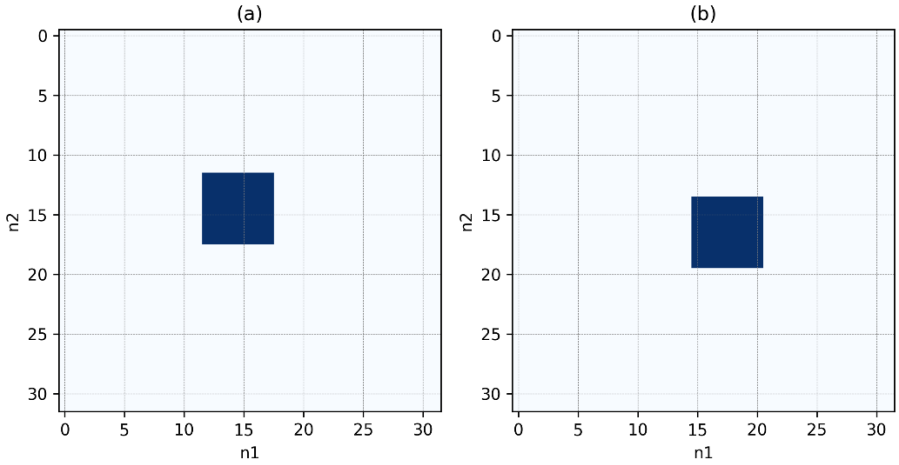
2.2.9 Phase Only Correlation with All Block Search (POC-ABS)

Phase Only Correlation with All Block Search (POC-ABS) merupakan metode estimasi pergerakan berbasis *block matching* yang dirancang untuk mendeteksi pergerakan wajah yang sangat halus dan berdurasi singkat pada *micro-expression*. Metode ini bekerja dengan membandingkan setiap blok pada dua *frame* yang berurutan untuk menentukan ada atau tidaknya pergerakan translasi pada blok yang sama. Jika dua blok identik, maka tidak dihasilkan vektor gerak, sedangkan perbedaan antar blok akan menghasilkan vektor gerak yang merepresentasikan adanya pergerakan halus pada area wajah tertentu. Vektor gerak tersebut selanjutnya digunakan sebagai fitur pergerakan untuk merepresentasikan kebocoran ekspresi yang muncul secara spontan dan berintensitas rendah (Rosiani dkk., 2020). Pada penelitian ini, POC diimplementasikan sebagai metode ekstraksi fitur berbasis gerak yang

bertujuan untuk merepresentasikan pergerakan pada berbagai area wajah melalui informasi perubahan pada tingkat piksel.

Dalam implementasinya, POC menggunakan *Fast Fourier Transform* (FFT) untuk mengubah gambar menjadi representasi dalam domain frekuensi. Data yang diproses oleh FFT berupa intensitas piksel dari citra atau blok citra yang dianalisis. Transformasi ini mengubah representasi spasial ke domain frekuensi sehingga informasi fase dapat dimanfaatkan untuk menentukan pergeseran antar blok gambar secara efisien berdasarkan karakteristik frekuensi (Rosiani dkk., 2020). Dengan cara ini, perhitungan fase dapat dilakukan secara efisien untuk menentukan pergeseran antara blok gambar berdasarkan informasi frekuensi.

Berdasarkan mekanisme tersebut, proses estimasi pergerakan pada POC-ABS dilakukan dengan membagi citra ke dalam blok-blok kecil yang dianalisis secara lokal. Pembagian citra ke dalam blok-blok kecil memungkinkan analisis pergerakan dilakukan secara lokal pada area wajah tertentu. Dengan pendekatan ini, dua *frame* yang berurutan, yaitu *n1* dan *n2*, dibandingkan pada blok yang bersesuaian untuk mengamati adanya pergeseran atau perubahan pergerakan, sebagaimana diilustrasikan pada Gambar 2.2.



Gambar 2.5 Contoh dua *frame* untuk n1 dan n2

Sumber: (Rosiani dkk., 2020)

POC-ABS digunakan untuk mengestimasi pergerakan antara dua *frame*, yaitu *frame* referensi dan *frame* pada waktu berikutnya. Misalkan

terdapat dua *frame* berukuran $N_1 \times N_2$ yang dinyatakan sebagai $f(n_1, n_2)$dan $g(n_1, n_2)$, sebagaimana ditunjukkan pada Gambar 2.2.

Untuk mempermudah formulasi, indeks koordinat diasumsikan berada pada rentang $n_1 = -M_1, \dots, M_1$dan $n_2 = -M_2, \dots, M_2$, dengan $N_1 = 2M_1 + 1$dan $N_2 = 2M_2 + 1$. Transformasi Fourier Diskrit dari kedua *frame* tersebut selanjutnya dinyatakan dalam bentuk persamaan sebagai berikut (Rosiani dkk., 2020).

$$F = (k1, k2) = \sum_{n_1, n_2} f(n_1, n_2) W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2}$$
$$= A_f(k_1 k_2) e^{j\theta_F\left(k_{1_{k_2}}\right)}$$

(2.1)

Keterangan**:**

- $F(k_1, k_2)$ merupakan hasil Transformasi Fourier dua dimensi dari *frame* $f(n_1, n_2)$ yang merepresentasikan citra pada domain frekuensi.
- $W_{N_1} = e^{-j\frac{2\pi}{N_1}}$ dan $W_{N_2} = e^{-j\frac{2\pi}{N_2}}$ adalah konstanta kompleks yang digunakan dalam perhitungan Transformasi Fourier Diskrit (DFT) untuk masing-masing dimensi citra.
- $k_1$dan $k_2$menyatakan indeks frekuensi pada domain Fourier yang merepresentasikan komponen frekuensi horizontal dan vertikal.
- Spektrum Fourier $F(k_1, k_2)$ dapat dinyatakan dalam bentuk polar sebagai $F(k_1, k_2) = A_F(k_1, k_2)\, e^{j\theta_F(k_1, k_2)}$, di mana $A_F(k_1, k_2)$ merepresentasikan amplitudo spektrum dan $\theta_F(k_1, k_2)$ menyatakan informasi fase.

$$G = (k1, k2) = \sum_{n_1, n_2} f(n_1, n_2) W_{N_1}^{k_1 n_1} W_{N_2}^{k_2 n_2}$$
$$= A_g(k_1 k_2) e^{j\theta_F\left(k_{1_{k_2}}\right)}$$

(2.2)

Keterangan:

- $G(k_1, k_2)$ merupakan hasil Transformasi Fourier dua dimensi dari *frame* $g(n_1, n_2)$ yang merepresentasikan citra pada domain frekuensi.
- $W_{N_1} = e^{-j\frac{2\pi}{N_1}}$dan $W_{N_2} = e^{-j\frac{2\pi}{N_2}}$ merupakan konstanta kompleks yang digunakan dalam perhitungan Transformasi Fourier Diskrit (DFT) pada masing-masing dimensi citra.
- Spektrum Fourier $G(k_1, k_2)$ dapat dinyatakan dalam bentuk polar sebagai $G(k_1, k_2) = A_G(k_1, k_2)\, e^{j\theta_G(k_1, k_2)}$, di mana $A_G(k_1, k_2)$ menyatakan amplitudo spektrum dan $\theta_G(k_1, k_2)$ merepresentasikan informasi fase.

Cross spectrum $\hat{R}(k_1, k_2)$ antara spektrum Fourier $F(k_1, k_2)$ dan $G(k_1, k_2)$ didefinisikan sebagai hasil normalisasi perkalian antara $F(k_1, k_2)$ dan konjugat kompleks dari $G(k_1, k_2)$. Normalisasi ini bertujuan untuk menghilangkan pengaruh amplitudo sehingga hanya informasi fase yang dipertahankan. Dengan demikian, cross spectrum dapat dinyatakan sebagai fungsi yang merepresentasikan selisih fase antara kedua spektrum *Fourier* menggunakan persamaan berikut (Rosiani dkk., 2020).

$$R(k_1 k_2) = \frac{F(k_1, k_2)G(k_1, k_2)}{|F(k_1, k_2)||G(k_1, k_2)|}$$

$$= e^{j(\theta_F(k_1, k_2) - \theta_G(k_1, k_2))} \tag{2.3}$$

Keterangan:

- $G^*(k_1, k_2)$ menyatakan konjugat kompleks dari spektrum Fourier $G(k_1, k_2)$.
- $\hat{R}(k_1, k_2)$ merupakan *cross-power spectrum* yang hanya memuat informasi perbedaan fase antara dua citra pada domain frekuensi.
- $\theta(k_1, k_2)$ menyatakan selisih fase antara dua spektrum Fourier, yang didefinisikan sebagai $\theta_F(k_1, k_2) - \theta_G(k_1, k_2)$.

Dengan menggunakan konjugat kompleks dari $G(k_1, k_2)$, *cross-power spectrum* menekankan perbedaan fase antara dua *frame* tanpa melibatkan informasi amplitudo. Selisih fase tersebut merepresentasikan pergeseran translasi yang terjadi antar*frame*. Selanjutnya, fungsi Phase Only Correlation (POC) $\hat{r}(n_1, n_2)$diperoleh melalui Transformasi Fourier Diskrit dua dimensi terbalik (*inverse DFT*) dari $\hat{R}(k_1, k_2)$, yang secara matematis dirumuskan sebagai berikut (Rosiani dkk., 2020).

$$\hat{r}(n_1, n_2) = \frac{1}{N_1 N_2} \sum_{k_1, k_2} R(k_1, k_2) W_{N_1}^{-k_1 n_1} W_{N_2}^{-k_2 n_2} \tag{2.4}$$

Keterangan:

- $r(n_1, n_2)$ merupakan fungsi korelasi *Phase Only Correlation* (POC) pada domain spasial yang diperoleh dari transformasi balik spektrum fase.
- Nilai maksimum dari $r(n_1, n_2)$ merepresentasikan keberadaan puncak korelasi, di mana tinggi puncak menunjukkan tingkat kemiripan fase antara dua *frame*, sedangkan posisi puncak menunjukkan estimasi pergeseran translasi antar*frame*.

Dalam kondisi dua *frame* identik, dengan $f(n_1, n_2) = g(n_1, n_2)$, fungsi POC dapat dirumuskan sebagai berikut (Rosiani dkk., 2020).

$$\hat{r}(n_1, n_2) = \frac{1}{N_1 N_2} \sum_{k_1, k_2} W_{N_1}^{-k_1 n_1} W_{N_2}^{-k_2 n_2} = \delta(n_1, n_2)$$

$$\delta(n_1, n_2) = \begin{cases} 1 & Jika\ n_1 = n_1 = 0 \\ 0 & Jika\ selain\ itu \end{cases}$$

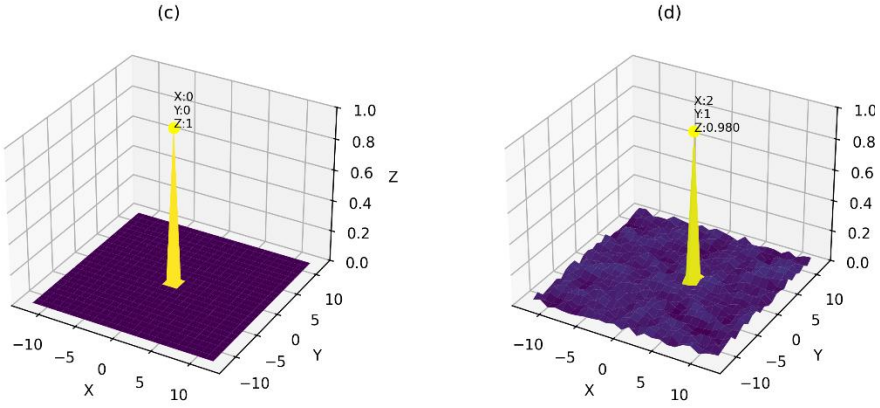
(2.5)

Keterangan:

- $\delta(n_1, n_2)$ merupakan fungsi delta Kronecker dua dimensi yang berperan sebagai indikator kesesuaian sempurna, di mana nilainya bernilai satu pada kondisi tertentu dan nol pada kondisi lainnya.

- Nilai maksimum dari $r(n_1, n_2)$ bernilai satu hanya ketika pergeseran translasi bernilai nol, yang menunjukkan bahwa kedua *frame* tidak mengalami pergeseran dan bersifat identik secara sempurna.

  Jika dua blok gambar identik, nilai POC akan sama dengan 1 dan menghasilkan puncak tajam yang sempurna. Sebaliknya, jika tidak identik, nilai POC akan turun secara signifikan.



Gambar 2.5 Proses POC deteksi pergerakan dari gambar contoh 2.2

Sumber: (Rosiani dkk., 2020)

Hasil perbandingan pada Gambar 2.3 menghasilkan dua fitur utama, yaitu koordinat Xdan Y, yang merepresentasikan pergeseran piksel dari titik pusat $(0, 0)$ menuju posisi baru (X, Y) setelah terjadinya pergeseran antar*frame*. Nilai Xdan Ymenunjukkan arah serta jarak perpindahan blok dari *frame* referensi ke *frame* berikutnya.

Berdasarkan kedua nilai tersebut, selanjutnya dapat diturunkan dua fitur tambahan, yaitu sudut pergeseran (*theta*) dan besar pergeseran (*magnitude*). Sudut *theta* merepresentasikan arah vektor pergeseran yang dihitung menggunakan hubungan trigonometri antara komponen Xdan Y, sedangkan *magnitude* menyatakan panjang vektor pergeseran yang menggambarkan besar perubahan posisi blok. Persamaan untuk menghitung nilai *theta* dinyatakan sebagai berikut (Rosiani dkk., 2020).

$$\theta_{rad} = arctan\frac{y}{x} \tag{2.6}$$

Keterangan:

- x menyatakan komponen pergeseran horizontal yang menunjukkan arah perpindahan ke kanan atau ke kiri.
- y menyatakan komponen pergeseran vertikal yang menunjukkan arah perpindahan ke atas atau ke bawah.
- Sudut θ merepresentasikan arah vektor pergeseran (x, y) terhadap sumbu X.
- Nilai sudut tersebut dihitung dalam satuan radian menggunakan fungsi *arc-tangent*, yang menunjukkan arah terjadinya pergerakan blok, seperti ke kanan, ke atas, ke kiri, atau kombinasi arah lainnya.

$$\theta_{deg} = \theta_{rad} \cdot \frac{180}{\pi} \tag{2.7}$$

Keterangan:

- Nilai θ_{rad} dinyatakan dalam satuan radian, sehingga perlu dikonversi ke satuan derajat $(°)$ agar arah pergerakan lebih mudah dipahami dan diinterpretasikan secara intuitif.

$$\theta = \theta_{deg} + \left(360 \cdot (y < 0)\right) \tag{2.8}$$

Keterangan:

- Kondisi $y < 0$ menunjukkan bahwa arah vektor pergeseran berada pada kuadran tertentu, sehingga diperlukan penyesuaian sudut dengan menambahkan $360°$ agar representasi sudut berada pada rentang yang konsisten.
- Operasi logika $(y < 0)$ menghasilkan nilai 1 ketika kondisi terpenuhi dan bernilai 0 ketika $y \geq 0$, sehingga penambahan sudut dilakukan secara kondisional.

Sedangkan untuk nilai Magnitude dapat dihitung dengan persamaan berikut (Rosiani dkk., 2020).

$$m = \sqrt{x^2 + y^2} \tag{2.9}$$

Keterangan:

- Nilai *magnitude* m menyatakan panjang vektor pergerakan yang dihitung dari komponen horizontal x dan komponen vertikal y menggunakan prinsip Teorema Pythagoras.
- Nilai *magnitude* yang besar menunjukkan adanya pergerakan dengan intensitas tinggi, sedangkan nilai *magnitude* yang kecil merepresentasikan pergerakan mikro atau perubahan yang sangat halus.

Dengan memanfaatkan fitur X, Y, sudut pergeseran (*theta*), dan *magnitude*, data hasil ekstraksi dapat disusun menjadi sebuah *dataset* yang digunakan pada tahap selanjutnya, yaitu proses klasifikasi. Melalui POC, informasi penting terkait pergerakan mikro pada wajah dapat diekstraksi secara terstruktur untuk dianalisis lebih lanjut dalam pengenalan *micro-expression.* Proses ekstraksi fitur ini dilakukan secara berulang pada setiap pasangan *frame*, sehingga dihasilkan representasi fitur yang menggambarkan arah dan besar vektor pergerakan sepanjang seluruh rangkaian citra yang dianalisis.

Berikut dijelaskan tahapan metode *Phase Only Correlation with All Block Search* (POC-ABS) secara lebih terperinci.

Tahapan Perhitungan POC-ABS per Blok

1.1 Pemotongan citra menjadi blok

Citra dibagi menjadi beberapa blok berukuran tetap. Sebagai contoh, untuk citra berukuran 70 × 35 piksel dengan ukuran blok (*block size*) sebesar 7, jumlah blok yang dihasilkan adalah (70/7) × (35/7) = 10 × 5, sehingga diperoleh total 50 blok.

Sebagai ilustrasi, blok ke-22 memiliki area citra pada koordinat x = 14 hingga 21 dan y = 14 hingga 21. Blok yang bersesuaian dari *frame* referensi dan *frame* berikutnya dipotong untuk kemudian dibandingkan.

1.2 Penerapan window Hanning

Untuk mengurangi efek tepi (*edge effect*) pada setiap blok, diterapkan fungsi window Hanning dua dimensi yang dirumuskan sebagai berikut:

$$w(x,y) = hanning(x) \cdot hanning(y)^T \tag{2.10}$$

Jika ukuran blok adalah 7, maka window Hanning yang dihasilkan berbentuk matriks berukuran 7 × 7 dan diaplikasikan pada setiap blok sebelum transformasi Fourier.

1.3 Transformasi Fourier

Blok citra hasil pemotongan kemudian ditransformasikan ke domain frekuensi menggunakan Transformasi Fourier Diskrit dua dimensi (FFT2):

$$\begin{aligned} F_1(u,v) &= FFT_2(block_{ref} \cdot w) \\ F_2(u,v) &= FFT_2(block_{curr} \cdot w) \end{aligned} \tag{2.11}$$

di mana $block_{ref}$ dan $block_{curr}$ masing-masing merupakan blok dari *frame* referensi dan *frame* saat ini.

1.4 Hitung Korelasi Fase

Hasil korelasi fase pada domain spasial diperoleh melalui transformasi Fourier balik:

$$r(x,y) = | \, IFFT_2(R) \, | \tag{2.12}$$

Matriks $r(x,y)$berukuran 7 × 7 merepresentasikan nilai korelasi spasial antarblok.

1.5 Deteksi Titik Puncak Korelasi (*Peak Detection*)

Titik puncak korelasi ditentukan dengan mencari nilai maksimum pada matriks $r(x,y)$. Koordinat puncak dihitung sebagai berikut:

$$peak = np.unravel_index(np.argmax(r), r.shape)$$
$$dy = peak[0] - center$$
$$dx = peak[1] - center$$

(2.13)

Sebagai contoh, jika puncak korelasi berada pada koordinat (5, 3) dan pusat blok berada pada (3, 3), maka diperoleh nilai dx = 0 dan dy = +2, yang menunjukkan bahwa blok mengalami pergeseran dua piksel ke arah bawah.

Vector Gerak (*Motion Vektor*)

Berdasarkan nilai *dx* dan *dy* pada setiap blok, vektor gerak dihitung menggunakan dua parameter utama, yaitu *magnitude* dan sudut pergeseran (*theta*).

- Magnitude dihitung sebagai:

$$m = \sqrt{dx^2 + dy^2}$$

(2.14)

- Sudut pergeseran (theta) dihitung sebagai:

$$\theta = \tan^{-1}\left(\frac{dy}{dx}\right)$$

(2.15)

Sebagai contoh, untuk sebuah blok dengan dx = 1 dan dy = −2, diperoleh nilai magnitude sebesar $\sqrt{5} \approx 2{,}24$dan sudut $\theta \approx -63{,}43°$.

Contoh fitur hasil ekstraksi

Setiap blok menghasilkan fitur berupa posisi dan vektor gerak. Contoh hasil ekstraksi fitur ditunjukkan pada Tabel berikut:

Tabel 2.2 Contoh hasil ekstraksi fitur

Blok	**X**	**Y**	**Magnitude**	**Theta**
10	10	10	2.00	0° (kanan)
22	17	17	2.24	-63° (kanan bawah)
41	10	31	2.00	90° (bawah)
46	45	31	1.41	-45° (kanan bawah)
50	63	31	2.00	180° (kiri)

Blok dengan vektor panjang dan arah yang jelas menunjukkan adanya pergerakan nyata, sedangkan blok dengan dx = dy = 0 menandakan tidak adanya pergerakan.

Visualisasi vektor gerak

Visualisasi pergerakan dilakukan menggunakan panah (*quiver*), dengan ketentuan sebagai berikut:

- Xdan Ymerepresentasikan posisi tengah blok.
- Udan Vmerepresentasikan komponen arah vektor gerak (dx, dy).
- Panah hanya tampil pada blok yang memiliki $dx \neq 0$ atau $dy \neq 0$.

2.2.10 Data Tabular

Data tabular merupakan representasi data yang disusun dalam struktur baris dan kolom, di mana setiap baris merepresentasikan satu objek atau sampel pengamatan, sedangkan setiap kolom merepresentasikan fitur yang menggambarkan karakteristik objek tersebut. Dalam format ini, setiap sampel dinyatakan sebagai vektor fitur numerik yang terorganisasi secara sistematis sehingga memudahkan proses analisis dan pemodelan, sejalan dengan definisi bahwa data tabular terdiri atas baris yang merepresentasikan objek dan kolom yang merepresentasikan fitur (Arik & Pfister, 2021). Representasi tabular banyak digunakan dalam berbagai permasalahan klasifikasi dan regresi karena mampu menyajikan informasi numerik secara terstruktur serta kompatibel dengan beragam algoritma pembelajaran mesin, dan bahkan merupakan tipe data yang paling umum dijumpai dalam aplikasi pembelajaran mesin di dunia nyata (Gorishniy dkk., 2024).

Dalam konteks analisis *micro-expression*, data tabular digunakan sebagai bentuk representasi fitur hasil ekstraksi pergerakan wajah yang bersifat numerik. Setiap sampel ekspresi wajah dapat direpresentasikan sebagai satu baris data yang memuat kumpulan fitur pergerakan, sedangkan setiap fitur merepresentasikan karakteristik pergerakan tertentu pada area wajah. Representasi ini memungkinkan informasi pergerakan mikro yang sangat halus dan berdurasi singkat untuk dimodelkan secara eksplisit tanpa bergantung pada struktur spasial citra atau urutan temporal video. Dengan demikian, data tabular menyediakan kerangka representasi yang dapat

digunakan untuk memodelkan hubungan antar-fitur dan antar-sampel secara sistematis, khususnya pada pendekatan klasifikasi berbasis *deep learning* tabular.

2.2.11 Facebook AI Similarity Search (FAISS)

Facebook AI Similarity Search (FAISS) merupakan pustaka *open-source* yang dikembangkan oleh Facebook AI *Research* untuk melakukan pencarian kemiripan (*similarity search*) pada vektor berdimensi tinggi secara efisien. FAISS bekerja dengan membangun indeks yang dapat menyimpan hingga miliaran vektor *embedding* dan digunakan untuk mengambil sejumlah *k* vektor yang paling mirip terhadap suatu vektor *query*. Proses pencarian kemiripan pada FAISS umumnya dilakukan menggunakan metrik jarak seperti *Euclidean distance* (L2) atau *cosine similarity*, sehingga memungkinkan penerapan *retrieval* berbasis *embedding* pada data berskala besar dengan waktu pencarian yang sangat cepat (Schwarz dkk., 2022).

Dalam konteks *embedding-based retrieval*, pencarian informasi dilakukan dengan merepresentasikan data dan *query* dalam bentuk *embedding* numerik, kemudian mengubah permasalahan *retrieval* menjadi pencarian tetangga terdekat pada ruang *embedding*. Pendekatan ini umumnya digunakan pada lapisan *retrieval* awal yang berfungsi mengambil sejumlah kandidat relevan secara efisien sebelum tahap pemeringkatan lanjutan. Dalam sistem *retrieval* modern, FAISS dimanfaatkan sebagai pustaka pendukung untuk menjalankan proses pencarian kemiripan berbasis *nearest neighbor* secara efisien, tanpa berperan sebagai model pembelajaran itu sendiri, melainkan sebagai infrastruktur yang mendukung mekanisme *retrieval* pada sistem berbasis *embedding* (Huang dkk., 2020).

Secara teknis, FAISS mengimplementasikan *Approximate Nearest Neighbor (ANN) search* dengan membangun indeks vektor berbasis *vector quantization* untuk menekan kompleksitas pencarian. Salah satu struktur indeks yang umum digunakan adalah *Inverted File Index (IVF)*, di mana ruang embedding dipartisi menggunakan *k-means clustering* ke dalam sejumlah *coarse clusters*. Pada saat *query*, pencarian hanya dilakukan pada

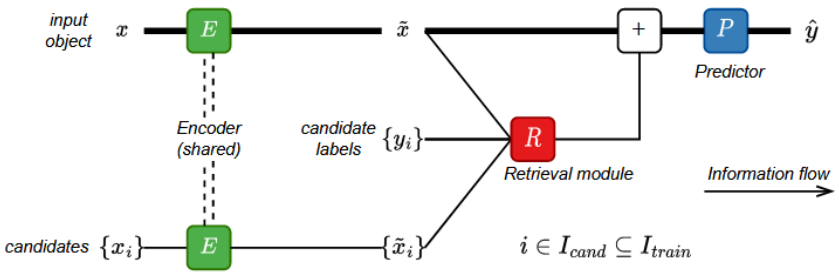
cluster terdekat (*nprobe*), sehingga menghindari komputasi jarak terhadap seluruh vektor. Untuk efisiensi memori dan kecepatan, FAISS mengombinasikan IVF dengan *Product Quantization (PQ)* yang mengompresi vektor menjadi representasi diskret dan memungkinkan perhitungan jarak secara aproksimasi menggunakan *lookup table*. Variasi lanjutan seperti *Optimized Product Quantization (OPQ)* digunakan untuk meningkatkan akurasi dengan melakukan rotasi ruang *embedding* sebelum kuantisasi. Melalui pengaturan parameter seperti jumlah *cluster* (*nlist*), jumlah *cluster* yang dipindai (*nprobe*), dan ukuran kode kuantisasi, FAISS menyediakan kontrol langsung terhadap *trade-off* antara *recall*, *latency*, dan *memory usage* pada sistem retrieval berskala besar (Huang dkk., 2020).

2.2.12 Tabular Retrieval Network (TabR)

Tabular Retrieval Network (TabR) merupakan model *deep learning* yang dirancang khusus untuk permasalahan klasifikasi dan regresi pada data tabular dengan mengintegrasikan mekanisme *retrieval* ke dalam jaringan saraf *feed-forward*. TabR dikembangkan untuk mengatasi keterbatasan model *deep learning* tabular konvensional yang sering kali masih tertinggal performanya dibandingkan metode *gradient-boosted decision trees* (GBDT). Dengan memanfaatkan prinsip *retrieval-augmented learning*, TabR memperkaya representasi suatu sampel dengan informasi dari sampel lain yang paling relevan dalam data latih, sehingga prediksi tidak hanya bergantung pada bobot model, tetapi juga pada konteks lokal antar-sampel (Gorishniy dkk., 2024).

Pendekatan *retrieval* pada TabR memungkinkan model menangkap hubungan lokal yang sulit direpresentasikan melalui transformasi global semata, terutama pada data tabular dengan ukuran kecil hingga menengah. Studi empiris menunjukkan bahwa TabR mencapai performa rata-rata terbaik di antara model *deep learning* tabular pada berbagai benchmark publik, serta pada sejumlah *dataset* mampu menyamai bahkan melampaui performa GBDT. Selain meningkatkan akurasi, mekanisme *retrieval* juga membantu TabR menghasilkan prediksi yang lebih adaptif terhadap

distribusi data lokal, khususnya pada tugas klasifikasi (Gorishniy dkk., 2024; Pasaribu dkk., 2024).



Gambar 2.6 Arsitektur TabR

Sumber: (Gorishniy dkk., 2024)

Berdasarkan Gambar 2.4, arsitektur *Tabular Retrieval Network* (TabR) dikembangkan dari jaringan *feed-forward* yang dinyatakan sebagai berikut (Gorishniy dkk., 2024).

$$f(x) = P(E(x)) \tag{2.16}$$

dengan menambahkan modul *retrieval* R setelah encoder dalam bentuk cabang *residual*. *Encoder* berfungsi memetakan sampel target x dan sampel kandidat $\{x_i\}$ ke dalam ruang representasi laten yang sama, sehingga diperoleh representasi sebagai berikut (Gorishniy dkk., 2024).

$$\tilde{x} = E(x) \text{ dan } \tilde{x}_i = E(x_i). \tag{2.17}$$

Selanjutnya, modul *retrieval* memanfaatkan kemiripan antar-sampel pada ruang laten, yang umumnya diukur menggunakan jarak, untuk mengambil informasi dari sampel kandidat yang paling relevan dan menggabungkannya dengan representasi laten sampel target. Representasi akhir dapat dinyatakan sebagai berikut (Gorishniy dkk., 2024).

$$z = \tilde{x} + R(\tilde{x}, \{\tilde{x}_i, y_i\}),$$

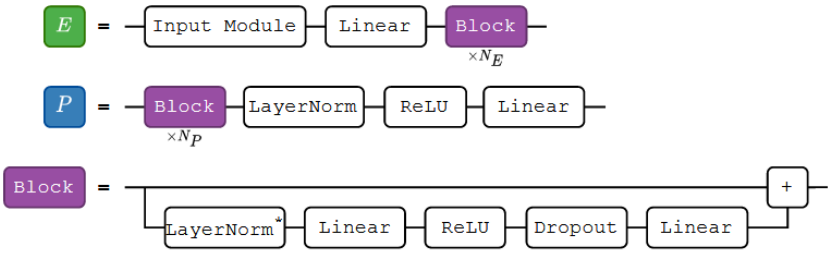
(2.18)

yang kemudian digunakan oleh *predictor* untuk menghasilkan prediksi (Gorishniy dkk., 2024).

$$\hat{y} = P(z).$$

(2.19)

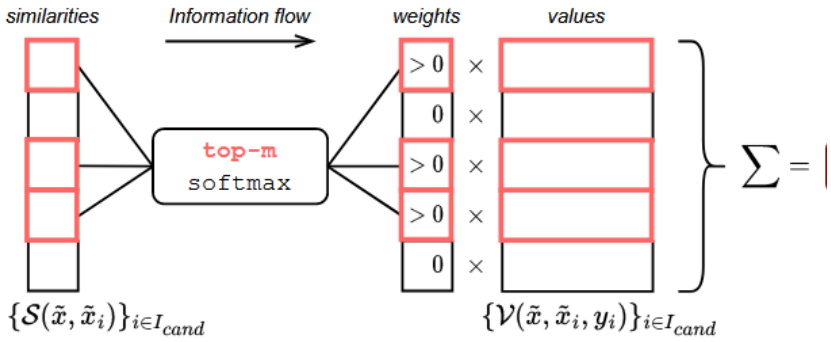
Dengan mekanisme ini, keputusan model tidak hanya bergantung pada bobot jaringan, tetapi juga mempertimbangkan konteks lokal antar-sampel.



Gambar 2.7 *Encoder dan Decoder*

Sumber: (Gorishniy dkk., 2024)

Berdasarkan Gambar 2.5, *encoder* E dan *predictor* P pada TabR dirancang sebagai jaringan *feed-forward* sederhana. *Encoder* memproses data tabular melalui *Input Module* untuk melakukan pemrosesan awal fitur, kemudian dilanjutkan dengan beberapa *block* pemrosesan guna menghasilkan representasi laten $\tilde{x}$ yang digunakan pada tahap retrieval dan prediksi. *Predictor* P selanjutnya memetakan representasi laten tersebut ke ruang keluaran melalui rangkaian *block* dan lapisan linear untuk menghasilkan prediksi akhir.



Gambar 2.8 *Retreival Module*

Sumber: (Gorishniy dkk., 2024)

Berdasarkan Gambar 2.6, modul *retrieval R* bekerja dengan memilih sejumlah m sampel kandidat yang paling mirip dengan sampel target pada ruang representasi laten. Nilai kemiripan antar-sampel dihitung menggunakan fungsi kemiripan (Gorishniy dkk., 2024).

$$s_i = S(\tilde{x}, \tilde{x}_i),$$

(2.20)

kemudian dinormalisasi menggunakan fungsi *softmax* untuk memperoleh bobot kontribusi (Gorishniy dkk., 2024).

$$w_i = \frac{\exp(s_i)}{\sum_{j \in I_{cand}} \exp(s_j)}.$$

(2.21)

Selanjutnya, informasi dari sampel kandidat diagregasi menggunakan bobot tersebut untuk menghasilkan keluaran modul *retrieval* (Gorishniy dkk., 2024).

$$R = \sum_{i=1}^{m} w_i \cdot V(\tilde{x}, \tilde{x}_i, y_i),$$

(2.22)

yang digunakan untuk memperkaya representasi laten sampel target sebelum dilakukan prediksi akhir.

2.2.13 Question-Wise Split

Question-wise split merupakan skema pembagian *dataset* dengan memisahkan data berdasarkan kelompok pertanyaan atau item tertentu. Pada skema ini, data dari pertanyaan tertentu digunakan sebagai data pelatihan, sedangkan pertanyaan lain digunakan sebagai data validasi atau pengujian. Berbeda dengan *random split* yang membagi data secara acak, *question-wise split* menjaga agar data dari kelompok pertanyaan tertentu tidak tercampur antar *subset*, sehingga evaluasi dapat dilakukan pada kelompok data yang berbeda dari data pelatihan.

Skema ini digunakan untuk menguji kemampuan generalisasi model terhadap data dari pertanyaan yang belum digunakan pada proses pelatihan. Prinsip ini memisahkan *subset* item secara penuh ke dalam partisi pengujian untuk mensimulasikan kondisi ketika model menghadapi item atau konten baru, serta mengevaluasi kemampuan model tanpa dukungan historis dari data *training* (Abu & Mohd, 2026). Dalam penelitian ini, prinsip *question-wise split* digunakan untuk membagi *dataset* berdasarkan nomor pertanyaan, di mana beberapa pertanyaan digunakan sebagai *training* set, sedangkan pertanyaan lain digunakan sebagai *validation* set dan testing set.

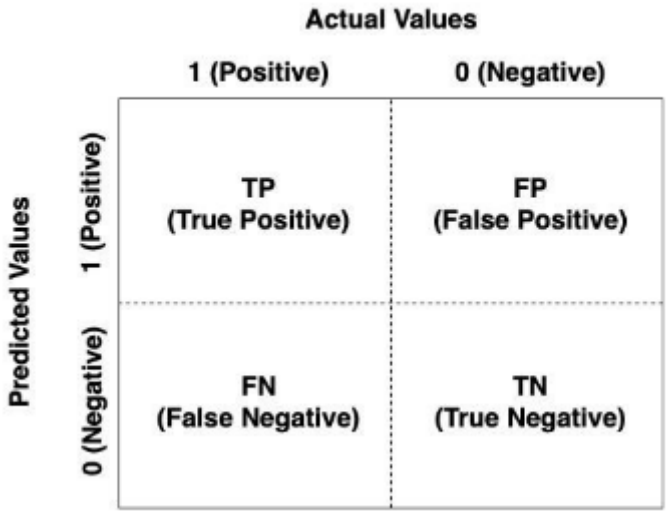
2.2.14 Confussion Matrix

Confusion matrix merupakan tabel kontingensi dua dimensi yang digunakan untuk mengevaluasi kinerja model klasifikasi dengan membandingkan label aktual dan label prediksi. Struktur ini memungkinkan analisis kesalahan klasifikasi secara eksplisit melalui empat komponen utama, yaitu *true positive*, *false positive*, *true negative*, dan *false negative*. Dengan representasi baris dan kolom yang merepresentasikan kelas aktual dan kelas prediksi, confusion *matrix* memberikan gambaran langsung mengenai pola kesalahan model serta tingkat keberhasilan klasifikasi pada setiap kelas, sehingga menjadi dasar penting dalam evaluasi sistem

klasifikasi, khususnya pada masalah biner maupun multikelas (Yang & Berdine, 2024).

Confusion matrix tidak hanya berfungsi sebagai alat visualisasi kesalahan klasifikasi, tetapi juga menjadi dasar perhitungan berbagai metrik evaluasi kinerja seperti *accuracy*, *precision*, *recall*, *sensitivity*, dan *specificity*. Metrik-metrik tersebut memungkinkan penilaian performa model secara lebih komprehensif, terutama pada kondisi data tidak seimbang, di mana akurasi saja dapat memberikan gambaran yang menyesatkan. Oleh karena itu, confusion *matrix* dianggap sebagai fondasi utama dalam evaluasi model pembelajaran mesin karena mampu menunjukkan secara rinci kekuatan dan kelemahan model pada masing-masing kelas (Sathyanarayanan, 2024).

Berdasarkan konsep tersebut, confusion *matrix* dapat direpresentasikan secara visual seperti pada Gambar 2.7, yang menunjukkan distribusi hasil klasifikasi antara kelas aktual dan kelas prediksi.



Gambar 2.9 *Confusion Matrix*

Dalam penelitian ini, *confusion matrix* digunakan untuk mengevaluasi kinerja model dalam mengklasifikasikan kondisi *anxiety* tinggi dan *anxiety* rendah. Kelas *anxiety* tinggi ditetapkan sebagai kelas positif, sedangkan kelas *anxiety* rendah sebagai kelas negatif. Dengan demikian, *true positive* (TP) adalah jumlah sampel *anxiety* tinggi yang

berhasil diprediksi sebagai *anxiety* tinggi oleh model, *sedangkan true negative* (TN) adalah jumlah sampel *anxiety* rendah yang berhasil diprediksi sebagai *anxiety* rendah. *False positive* (FP) menunjukkan sampel *anxiety* rendah yang keliru diprediksi sebagai *anxiety* tinggi, sementara *false negative* (FN) menunjukkan sampel *anxiety* tinggi yang keliru diprediksi sebagai *anxiety* rendah. Pendefinisian ini mengikuti konvensi umum dalam evaluasi klasifikasi biner.

Nilai-nilai pada *confusion matrix* selanjutnya digunakan sebagai dasar dalam perhitungan metrik evaluasi kinerja model. Metrik yang digunakan dalam penelitian ini meliputi *accuracy, precision*, dan *recall* yang dihitung menggunakan persamaan berikut (Sathyanarayanan, 2024).

4.1 *Accuracy*

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{2.23}$$

4.2 Precision

$$precision = \frac{TP}{TP + FP} \tag{2.24}$$

4.3 Recall

$$recall = \frac{TP}{TP + FN} \tag{2.25}$$

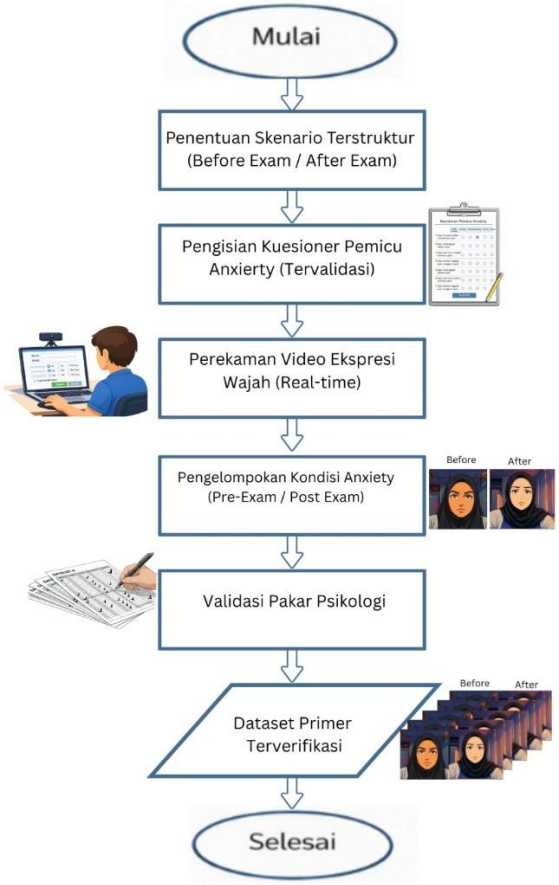
BAB III. METODOLOGI PENELITIAN

3.1 Waktu dan Tempat Penelitian

Penelitian ini dilaksanakan pada rentang waktu Agustus 2025 sampai Juni 2026 di Politeknik Negeri Malang, khususnya di Laboratorium IVSS (*Intelligent Vision and Smart System*) pada Gedung Jurusan Teknik Informatika.

3.2 Teknik Pengumpulan Data

Penelitian ini menggunakan data ekspresi wajah pada kondisi normal dan kondisi individu yang mengalami gangguan kecemasan (*anxiety*) sebagai data primer. Data diperoleh melalui proses pengumpulan langsung yang melibatkan subjek penelitian sebagai sumber utama. Subjek penelitian pada data latih merupakan mahasiswa dan mahasiswi semester V Program Studi Teknik Informatika pada tahun 2025 dengan rentang usia 20–22 tahun yang sedang atau akan menjalani Ujian. Sementara itu, data uji diperoleh dari mahasiswa dan mahasiswi semester IV Program Studi Teknik Informatika pada tahun 2026 dengan rentang usia 19–20 tahun yang juga sedang atau akan menjalani Ujian. Partisipan mencakup mahasiswa yang memiliki tingkat kecemasan tinggi maupun rendah, sehingga variasi kondisi psikologis dapat terwakili. Penelitian ini difokuskan pada mahasiswa yang menghadapi situasi evaluasi akademik, dengan tujuan memperoleh gambaran kondisi kecemasan yang muncul dalam konteks tersebut. Pengambilan data dilakukan dengan berkoordinasi bersama pihak-pihak yang terlibat secara langsung dalam proses akuisisi data. Secara garis besar, proses pengumpulan data dirancang dan dilaksanakan dalam tiga tahapan utama, sebagaimana yang ditunjukkan pada Gambar 3.1.



Gambar 3.1 Alur Pengumpulan Data Primer Penelitian

3.2.1 Tahapan Awal

Tahapan awal adalah tahapan persiapan sebelum dilakukan pengambilan data ekspresi wajah. Pada tahapan ini akan dilakukan.

a. Penentuan skenario pengambilan data dilakukan dengan menetapkan dua kondisi pengambilan, yaitu sebelum (*before*) dan sesudah (*after*) responden mengikuti Ujian Akhir Semester. Penetapan dua kondisi ini bertujuan untuk menangkap perbedaan respons kecemasan pada fase pra dan pasca ujian, sesuai dengan karakteristik kecemasan yang dapat muncul sebelum maupun setelah peristiwa pemicu.
b. Penyusunan kuesioner dilakukan dengan merancang pertanyaan yang berkaitan dengan pemicu munculnya perasaan cemas. Kuesioner ini disusun dengan mengacu pada penelitian *Faces*

Anxiety Scale as a Screening Measure of Preoperative Anxiety: Validation and Diagnostic Accuracy Study yang dilakukan oleh Jana Turzáková, Tomáš Sollár, dan Andrea Solgajová, yang menggunakan instrumen *Faces Anxiety Scale* (FAS). Selain itu, proses validasi kuesioner juga melibatkan penggunaan instrumen *Spielberger State-Trait Anxiety Inventory* (STAI) dan *Hospital Anxiety and Depression Scale* (HADS). Seluruh instrumen tersebut diterjemahkan ke dalam Bahasa Indonesia sebelum digunakan. Proses penyusunan dan validasi kuesioner dilaksanakan dengan pendampingan pakar, yaitu Prof. Dr. Adi Atmoko, M.Pd., dosen Fakultas Ilmu Pendidikan Universitas Negeri Malang dengan kepakaran di bidang bimbingan dan konseling.

c. Pengisian kuesioner tahap pertama menggunakan enam pernyataan awal. Hasil pengisian tersebut digunakan untuk mengevaluasi kejelasan pemahaman responden serta kesesuaian setiap butir terhadap konstruk kecemasan. Berdasarkan hasil evaluasi dan masukan pakar, satu butir pernyataan dieliminasi karena disusun dalam bentuk pernyataan positif mengenai kondisi rileks, sedangkan konstruk yang diukur berfokus pada kecemasan. Perbedaan arah makna tersebut dinilai berpotensi menimbulkan bias pemahaman dan inkonsistensi jawaban responden, sehingga lima pertanyaan akhir digunakan pada tahap berikutnya.
d. Pengisian kuesioner tahap kedua dengan menggunakan lima pertanyaan terpilih oleh calon objek penelitian sekaligus penentuan kelompok jenis kecemasannya. Tabel 3.1 merupakan contoh pertanyaan pada kuesioner.

Tabel 3.1 Pertanyaan Kuesioner *Faces Anxiety Scale* (FAS)

No.	Pertanyaan	1	2	3	4
1.	Saya merasakan semacam perasaan takut seperti ulat di perut				
2.	Saya merasakan semacam perasaan				

	takut, seolah-olah sesuatu yang buruk akan terjadi				
3.	Saya merasa gelisah karena saya harus terus bergerak				
4.	Pikiran-pikiran yang mengkhawatirkan muncul di benak saya				
5.	Saya tiba-tiba merasa panik				

Skala yang digunakan pada pengisian kuesioner adalah sebagai berikut:

- 1 = Sangat Tidak Setuju
- 2 = Tidak setuju
- 3 = Setuju
- 4 = Sangat Setuju

e. Pelabelan data dilakukan berdasarkan hasil kuesioner pada tingkat responden. Skor total kuesioner diperoleh dari penjumlahan nilai seluruh pernyataan, dengan rentang skor 5-20. Rentang skor 5-10 dikategorikan sebagai *anxiety* rendah, sedangkan rentang skor 11-20 dikategorikan sebagai *anxiety* tinggi. Seluruh video ekspresi wajah yang direkam dari satu responden diberi label yang sama sesuai kategori tingkat kecemasan tersebut. Apabila hasil kuesioner menunjukkan tingkat kecemasan tinggi, maka seluruh video responden tersebut diberi label *anxiety* tinggi, dan sebaliknya untuk *anxiety* rendah.

f. Sebelum melakukan pengumpulan data, dikembangkan sebuah aplikasi kuesioner yang digunakan sebagai media penelitian. Aplikasi ini dirancang untuk mendukung proses pengisian kuesioner sekaligus memfasilitasi perekaman video ekspresi wajah responden selama proses pengisian berlangsung. Melalui aplikasi tersebut, proses pengambilan data dapat dilakukan secara terstruktur dan terintegrasi, sehingga data kuesioner dan data visual yang

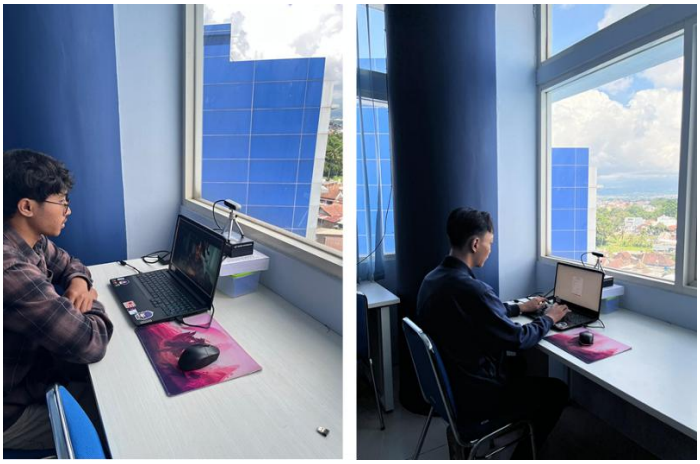
dihasilkan memiliki keterkaitan langsung dengan setiap pertanyaan yang dijawab oleh responden.

3.2.2 Tahapan Perekaman

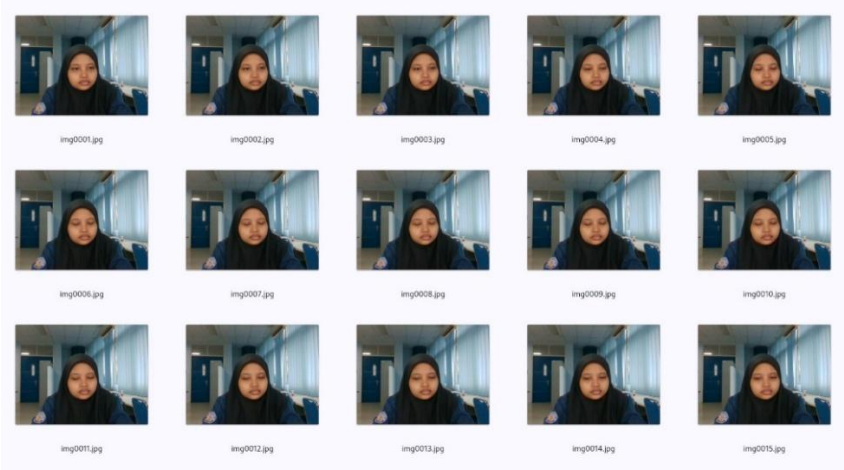
Tahapan perekaman merupakan tahap pengambilan data ekspresi wajah dari subjek penelitian yang telah ditentukan. Pada tahap ini, responden diminta untuk mengisi serta memberikan persetujuan melalui lembar informed consent terkait penggunaan data yang diperoleh untuk keperluan penelitian. Perekaman ekspresi wajah dilakukan sesuai dengan skenario pengambilan data yang telah ditetapkan pada tahap sebelumnya. Pengumpulan data dilakukan dalam bentuk rekaman video yang diambil secara langsung saat responden mengisi kuesioner melalui aplikasi yang dikembangkan, sehingga proses perekaman berlangsung secara natural sesuai dengan respons responden.

Setiap pertanyaan pada kuesioner menghasilkan satu rekaman video, sehingga satu kuesioner lengkap menghasilkan beberapa video yang tersegmentasi berdasarkan masing-masing pertanyaan. Seluruh hasil perekaman selanjutnya dikelompokkan berdasarkan kondisi waktu pengambilan data, yaitu before dan after, untuk mendukung analisis perbedaan ekspresi wajah pada kedua kondisi tersebut. Aktivitas perekaman ini dilakukan menggunakan dua perangkat kamera yang telah terintegrasi ke dalam aplikasi, yaitu:

- Intel *Real*Sense D415, digunakan sebagai kamera utama dengan pengaturan dengan resolusi 640 × 480 piksel dan *frame* rate 60 *frame* per detik (FPS), yang diharapkan mampu menangkap perubahan ekspresi wajah secara lebih halus.
- Webcam bawaan laptop Lenovo IdeaPad Gaming (HD 720p *with Privacy Shutter* dan 30 fps), digunakan sebagai kamera cadangan apabila terjadi kendala teknis pada kamera utama.



Gambar 3.2 Proses Pengambilan Data



Gambar 3.3 Contoh *Dataset* Dalam Bentuk *Frame*

3.2.3 Tahapan Validasi

Tahapan validasi merupakan tahap pelabelan data berdasarkan indikator kecemasan yang telah ditetapkan dengan memanfaatkan hasil perekaman video yang sebelumnya telah tersegmentasi secara otomatis berdasarkan setiap pertanyaan kuesioner dan kondisi pengambilan data (*before* dan *after*). Seluruh video hasil perekaman divalidasi oleh pakar, yaitu Prof. Dr. Adi Atmoko, M.Pd., dosen Fakultas Ilmu Pendidikan Universitas Negeri Malang dengan kepakaran di bidang bimbingan dan konseling, dengan mempertimbangkan hasil pengisian kuesioner responden sebagai dasar penentuan label kondisi kecemasan. Berdasarkan hasil validasi tersebut, data dikelompokkan ke dalam kategori *anxiety* tinggi dan *anxiety* rendah

sesuai label yang ditetapkan, kemudian digunakan sebagai *dataset* akhir pada tahap pemodelan dan analisis.

3.3 Teknik Pengolahan Data

Teknik pengolahan data dalam penelitian ini dilaksanakan melalui dua tahap utama yang saling berkaitan guna memastikan kualitas *dataset* yang digunakan.

3.3.1 Tahap Pengolahan Data Kuesioner dan Validasi Video

Tahap pertama mencakup pengolahan data kuesioner serta validasi data video untuk menjamin kesesuaian antara label kecemasan dan data visual. Rangkaian kegiatan pada tahap ini adalah sebagai berikut.

1. **Pengolahan Data Kuesioner**

   Data hasil pengisian kuesioner dihitung menggunakan sistem skoring yang telah ditetapkan oleh pakar psikologi, di mana setiap respons dijumlahkan untuk menghasilkan skor total. Berdasarkan skor total tersebut, tingkat kecemasan diklasifikasikan ke dalam empat kategori, yaitu kecemasan sangat rendah, rendah, tinggi, dan sangat tinggi. Penentuan kategori kecemasan ini dilakukan berdasarkan skema scoring yang telah ditetapkan dan divalidasi oleh pakar, sebagaimana disajikan pada Tabel 3.2.

   Tabel 3.2 Kategorisasi dan skor

Kategori	**Rentang Skor**
Anxiety Tinggi	11-20
Anxiety Rendah	1-10

2. **Validasi dan Seleksi Data Video**

   Seluruh video yang diperoleh dari responden melalui proses perekaman divalidasi oleh Prof. Dr. Adi Atmoko, M.Pd., sebagai pakar psikologi yang memiliki kompetensi dalam menilai ekspresi emosi dan kecemasan. Proses validasi ini dilakukan untuk memastikan bahwa ekspresi wajah yang terekam dalam video selaras dengan tingkat kecemasan yang diindikasikan berdasarkan hasil

pengisian kuesioner. Data video yang tidak menunjukkan konsistensi antara ekspresi visual dan skor kecemasan yang diperoleh akan dikeluarkan dari dataset guna menjaga kualitas dan integritas data penelitian.

Kriteria eliminasi ditetapkan untuk menyaring data yang tidak layak digunakan, meliputi ketidaksesuaian ekspresi wajah dengan indikator kecemasan serta kualitas video yang tidak memenuhi standar. Hanya video yang telah tervalidasi dan memenuhi seluruh persyaratan kualitas yang digunakan sebagai dataset pengujian dalam pengembangan model klasifikasi.

3.3.2 Tahap Preprocessing Citra Wajah dan Ekstraksi Fitur

Tahap kedua merupakan rangkaian pemrosesan citra wajah dan ekstraksi fitur untuk mengubah data video mentah menjadi representasi numerik yang terstruktur. Langkah-langkahnya adalah sebagai berikut.

1. ***Preprocessing* Citra Wajah**



Gambar 3.4 *Frame* Video

Proses standardisasi data mentah video meliputi deteksi wajah pada tiap *frame*, pemotongan area latar belakang, dan penyesuaian ukuran gambar untuk memastikan sistem hanya memproses area wajah subjek secara konsisten. *Input* dari tahap ini berupa video wajah subjek yang diperoleh dari proses perekaman pada kondisi sebelum dan sesudah Ujian. Video tersebut kemudian diubah menjadi rangkaian *frame* agar setiap citra wajah dapat diproses secara terpisah. *Output* dari tahap *preprocessing* berupa citra wajah yang telah terdeteksi, dipotong

pada area wajah, dan disesuaikan ukurannya sehingga memiliki format yang seragam untuk digunakan pada tahap ekstraksi fitur berikutnya.

2. **Deteksi Area Komponen Wajah (ROI)**

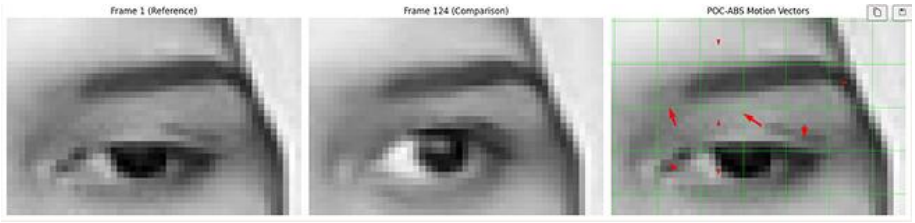


Gambar 3.5 Area Wajah Yang Diambil

Tahapan ini dilakukan untuk menentukan *Region of Interest* (ROI) dengan memanfaatkan *facial landmarks* pada citra wajah yang telah melalui proses *preprocessing*. *Input* dari tahap ini berupa citra wajah hasil *preprocessing* yang telah terdeteksi dan dipotong pada area wajah.

Selanjutnya, sistem mendeteksi titik-titik *landmark* wajah untuk menentukan posisi komponen wajah yang digunakan dalam penelitian, yaitu area alis dan mulut. Area tersebut dipilih karena memiliki peran penting dalam merepresentasikan perubahan ekspresi wajah, khususnya *micro-expression* atau pergerakan otot wajah yang muncul secara singkat dan tidak disadari. *Output* dari tahap ini berupa citra ROI pada area alis dan mulut yang telah dipisahkan dari wajah utama, sehingga dapat digunakan sebagai masukan pada tahap ekstraksi fitur.

3. **Ekstraksi Fitur (POC-ABS)**



Gambar 3.6 Pergerakan Yang Ditangkap POC-ABS

Tahapan ekstraksi fitur dilakukan dengan menerapkan metode *Phase Only Correlation with All Block Search* (POC-ABS) pada ROI yang telah diperoleh dari tahap sebelumnya. *Input* dari tahap ini berupa rangkaian *frame* ROI yang telah memiliki informasi *onset*, *apex*, dan *offset*. POC-ABS digunakan untuk menghitung pergeseran piksel antar

dua *frame* berurutan sehingga diperoleh nilai pergerakan mikro pada area wajah yang diamati. Hasil perhitungan tersebut menghasilkan fitur pergerakan berupa dx, dy, theta, dan *magnitude* yang merepresentasikan arah serta besar pergeseran pada area ROI. *Output* dari tahap ini berupa data fitur numerik pergerakan wajah yang digunakan sebagai masukan pada tahap pembentukan *dataset* dan klasifikasi.

4. ***Spotting Micro-expression***

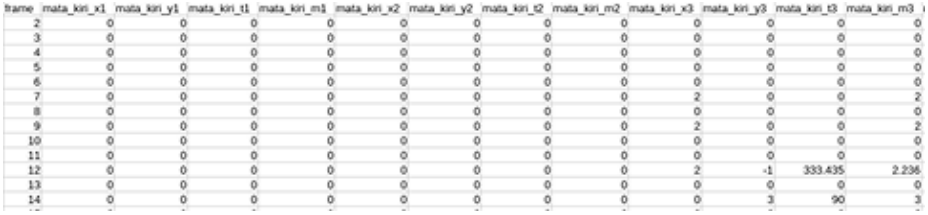


Gambar 3.7 Contoh Tiap Fase *Onset*, *Apex*, Dan *Offset*

Tahapan *spotting micro-expression* dilakukan untuk menentukan durasi kemunculan ekspresi mikro pada rangkaian *frame* video. *Input* dari tahap ini berupa citra ROI pada area alis dan mulut. Sistem menggunakan algoritma *Phase Only Correlation with All Block Search* (POC-ABS) untuk menghitung nilai pergerakan pada setiap *frame*.

Nilai pergerakan tersebut direpresentasikan dalam bentuk rata-rata magnitude sebagai intensitas pergerakan pada area ROI. Batas *threshold* ditentukan secara adaptif dari nilai rata-rata sinyal pergerakan yang ditambahkan dengan satu standar deviasi pada setiap video. Nilai mean magnitude yang melewati batas *threshold* tersebut digunakan untuk menentukan kandidat *event micro-expression*. Pada *event* tersebut, *onset* ditentukan sebagai awal pergerakan, *apex* sebagai titik puncak pergerakan, dan *offset* sebagai akhir pergerakan. *Output* dari tahap ini berupa rangkaian *event* yang telah memiliki informasi *onset*, *apex*, dan *offset* untuk digunakan sebagai input model.

5. **Pembentukan Data Tabular**



frame	mata_kiri_x1	mata_kiri_y1	mata_kiri_t1	mata_kiri_m1	mata_kiri_x2	mata_kiri_y2	mata_kiri_t2	mata_kiri_m2	mata_kiri_x3	mata_kiri_y3	mata_kiri_t3	mata_kiri_m3
2	0	0	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	0	0
5	0	0	0	0	0	0	0	0	0	0	0	0
6	0	0	0	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	2	0	0	2
8	0	0	0	0	0	0	0	0	0	0	0	0
9	0	0	0	0	0	0	0	0	2	0	0	2
10	0	0	0	0	0	0	0	0	0	0	0	0
11	0	0	0	0	0	0	0	0	0	0	0	0
12	0	0	0	0	0	0	0	0	2	-1	333.435	2.236
13	0	0	0	0	0	0	0	0	0	0	0	0
14	0	0	0	0	0	0	0	0	0	3	90	3

Gambar 3.8 Contoh Data Yang Dihasilkan

Setelah proses ekstraksi fitur dilakukan, nilai-nilai pergerakan wajah disusun menjadi data tabular. Tujuannya adalah agar hasil pergerakan dari video wajah dapat diubah menjadi angka yang siap digunakan oleh model. Fitur diambil dari tiga area wajah, yaitu mulut, mata-alis kanan, dan mata-alis kiri. Area mulut menghasilkan 50 blok, sedangkan mata-alis kanan dan mata-alis kiri masing-masing menghasilkan 24 blok. Jadi, total blok yang digunakan adalah:

$$50 + 24 + 24 = 98 \text{ blok}$$

Setiap blok menghasilkan empat nilai fitur, yaitu X, Y, theta, dan magnitude. Dengan demikian, jumlah fitur yang dihasilkan adalah:

$$98 \times 4 = 392 \text{ fitur}$$

Jadi, setiap data hasil ekstraksi memiliki 392 fitur numerik yang digunakan sebagai masukan untuk model TabR.

3.4 Desain Sistem

Sistem pengenalan *micro-expression* untuk mendeteksi tingkat *anxiety* dirancang secara konseptual sebagai sebuah arsitektur terdistribusi yang memisahkan antara antarmuka layanan pengguna, layanan basis data, dan AI. Secara keseluruhan, desain sistem ini berfokus pada alur metodologi ekstraksi fitur wajah dan klasifikasi menggunakan model *deep learning* yang dijalankan di atas tiga komponen utama berikut.

3.4.1 Arsitektur Sistem

Arsitektur sistem dibangun oleh tiga komponen utama yang saling terintegrasi:

1. **Client dan Web Application**

   Berfungsi sebagai antarmuka utama bagi pengguna. Komponen ini menangani interaksi langsung seperti pengisian kuesioner, perekaman video ekspresi wajah, serta menampilkan visualisasi hasil akhir analisis kepada partisipan atau penguji.

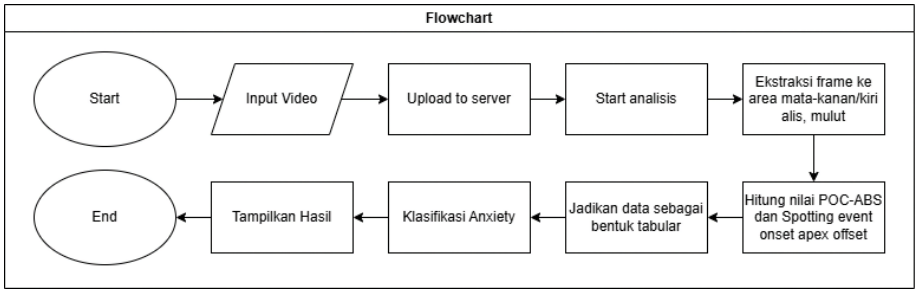
2. **Layanan Basis Data**

   Komponen pendukung yang terintegrasi dengan aplikasi web dan AI Server, bertanggung jawab untuk mengelola sesi pengguna, menyimpan data kuesioner, merekam metadata video, menyimpan hasil prediksi klasifikasi, serta mencatat log proses dari setiap tahapan analisis untuk keperluan audit dan evaluasi sistem.

3. **AI Server**

   Merupakan pusat pemrosesan mandiri yang bertugas menerima data video dari aplikasi web, melakukan *preprocessing* citra, ekstraksi fitur *micro-expression*, hingga menjalankan proses inferensi klasifikasi menggunakan model *deep learning*. Hasil prediksi dari AI Server kemudian dikembalikan ke Web *Application* dan tersimpan ke Layanan Basis Data.

3.4.2 Alur Proses Sistem



Gambar 3.9 Diagram Alur

Berdasarkan *flowchart* pada Gambar 3.12, alur proses sistem menggambarkan serangkaian proses yang dilakukan secara berurutan mulai dari pengumpulan data hingga penampilan hasil analisis. Adapun tahapan alur kerja sistem adalah sebagai berikut.

1. ***Input* Video**

   Proses dimulai ketika pengguna memasukkan video wajah ke dalam sistem. Video ini menjadi data awal yang akan digunakan untuk menganalisis pola pergerakan wajah. Pada tahap ini, sistem hanya menerima *file* video yang nantinya akan diproses lebih lanjut oleh server.

2. **Up*load* Video ke Server**

   Setelah video dimasukkan, sistem mengunggah video tersebut ke server. Proses *upload* ini diperlukan karena tahap analisis dilakukan pada sisi server. Setelah video berhasil diterima, server akan memulai proses analisis terhadap isi video.

3. **Start Analisis**

   Pada tahap ini, server mulai menjalankan proses analisis video. Video yang telah diunggah tidak langsung diklasifikasikan, tetapi harus melalui beberapa tahap pengolahan terlebih dahulu agar informasi pergerakan wajah dapat diperoleh secara lebih terstruktur.

4. **Ekstraksi *Frame* dan Area Wajah**

   Video kemudian diubah menjadi rangkaian frame. Dari frame tersebut, sistem mengambil area wajah yang menjadi fokus penelitian, yaitu mata-alis kanan, mata-alis kiri, dan mulut. Ketiga area tersebut digunakan karena memiliki pergerakan halus yang dapat merepresentasikan perubahan ekspresi wajah.

5. **Perhitungan POC-ABS dan *Spotting Event***

   Setelah area wajah berhasil diekstraksi, sistem menghitung nilai pergerakan menggunakan metode POC-ABS. Metode ini digunakan untuk memperoleh nilai pergeseran mikro pada setiap blok area wajah. Selanjutnya, dilakukan proses *spotting event* untuk menentukan bagian penting dari pergerakan wajah, yaitu fase *onset, apex,* dan *offset*. Tahap ini bertujuan agar sistem hanya memproses bagian pergerakan yang dianggap relevan.

6. Pembentukan Data Tabular

Nilai hasil perhitungan POC-ABS kemudian disusun menjadi data tabular. Data ini berisi nilai numerik dari hasil pergerakan wajah yang telah diekstraksi. Dengan bentuk tabular, data menjadi lebih terstruktur dan dapat digunakan sebagai masukan untuk proses klasifikasi

7. Klasifikasi *Anxiety*

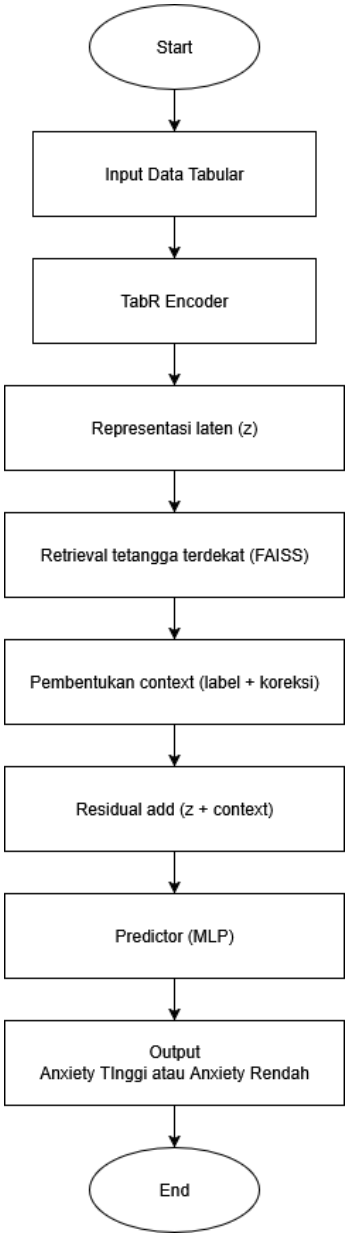
Data tabular yang telah terbentuk kemudian digunakan untuk melakukan klasifikasi *anxiety*. Pada tahap ini, model mempelajari pola dari fitur pergerakan wajah untuk menentukan hasil klasifikasi berdasarkan data yang telah diproses sebelumnya

8. Menampilkan Hasil

Setelah proses klasifikasi selesai, sistem menampilkan hasil analisis kepada pengguna. Hasil yang ditampilkan merupakan keluaran dari proses klasifikasi *anxiety* berdasarkan pola pergerakan wajah pada video yang telah diunggah. Setelah hasil ditampilkan, alur proses sistem berakhir.

3.5 Pembuatan Model

Pada tahap ini dilakukan pembuatan model klasifikasi kondisi *anxiety* menggunakan *Tabular Retrieval Network* (TabR). Model TabR digunakan untuk mengklasifikasikan kondisi kecemasan berdasarkan data tabular hasil ekstraksi fitur *micro-expression* yang telah melalui tahap *preprocessing*, validasi, dan pembentukan *dataset*. Alur kerja model TabR yang digunakan dalam penelitian ini ditunjukkan pada Gambar 3.13.



Gambar 3.10 Alur Model TabR

3.5.1 Persiapan dan Pra-pemrosesan Data

Data yang digunakan dalam pembuatan model berupa data tabular hasil proses merge dan flatten dari seluruh fitur *micro-expression*. Sebelum digunakan pada proses pelatihan dan inferensi, data terlebih dahulu dibagi menjadi data *training*, validation, dan testing menggunakan skema *question-wise split*. Skema ini membagi data berdasarkan nomor pertanyaan, sehingga data dari pertanyaan tertentu digunakan sebagai data

pelatihan, sedangkan pertanyaan lain digunakan sebagai data validasi dan pengujian.

Pembagian ini bertujuan untuk menguji kemampuan generalisasi model terhadap data dari pertanyaan yang berbeda dari data pelatihan. Selain itu, dilakukan penyeimbangan jumlah data antar kelas untuk mengurangi bias distribusi kelas. Selanjutnya, data diproses menggunakan imputer untuk menangani nilai kosong serta *scale*r untuk menormalkan skala fitur. Tahapan ini bertujuan untuk memastikan bahwa distribusi kelas dan skala fitur berada dalam kondisi yang konsisten. Hasil dari tahap ini digunakan sebagai masukan langsung ke encoder model.

3.5.2 Pembentukan Model TabR dan Mekanisme Context

Model TabR dibangun dengan memanfaatkan jaringan saraf *feed-forward* sebagai *encoder* untuk memetakan data tabular ke dalam representasi fitur laten. Representasi laten ini digunakan digunakan sebagai dasar pengukuran kemiripan dengan data latih.

Selain memanfaatkan informasi dari data masukan, model juga menggunakan mekanisme *context* yang berasal dari data latih. *Context* berfungsi sebagai sumber informasi tambahan yang memungkinkan model mempertimbangkan kemiripan antar sampel, sehingga proses klasifikasi tidak hanya bergantung pada informasi individual, tetapi juga pada pola lokal dalam data pelatihan.

3.5.3 Retrieval Berbasis FAISS

Untuk membantu model membuat prediksi, setiap data masukan tidak diproses secara terpisah, tetapi dibandingkan dengan data latih yang memiliki pola fitur serupa. Proses pencarian data yang paling mirip dilakukan menggunakan FAISS agar lebih cepat dan efisien. Pada tahap ini, representasi fitur dari data masukan digunakan sebagai query untuk mencari beberapa sampel terdekat dari data latih. Sampel yang terpilih kemudian digunakan sebagai informasi context dan digabungkan dengan representasi fitur utama. Dengan cara ini, model dapat mempertimbangkan pola dari data-data yang mirip sebelum menentukan kelas akhir.

3.5.4 Konfigurasi Model

Pelatihan model dilakukan menggunakan arsitektur *Tabular Retrieval Network* (TABR) dengan mekanisme *retrieval* berbasis konteks. Konfigurasi model diperoleh melalui proses *hyperparameter tuning* sebanyak 15 *trial* untuk menentukan kombinasi parameter terbaik berdasarkan performa validasi. Ruang pencarian yang digunakan mencakup *context size*, *d main*, jumlah blok *encoder* dan *predictor*, nilai *dropout*, *learning rate*, serta *weight decay*.

Setiap konfigurasi dilatih menggunakan data pelatihan dan dievaluasi menggunakan data validasi. Pada proses pelatihan, *batch size* ditetapkan sebesar 128, *optimizer* yang digunakan adalah AdamW, dan proses pelatihan dikontrol menggunakan *early stopping* dengan nilai *patience* sebesar 6. Mekanisme ini digunakan agar pelatihan berhenti ketika performa validasi tidak lagi mengalami peningkatan.

Tabel 3.3 Ruang Pencarian *Hyperparameter*

No	Hyperparameter	Rentang / Nilai	Keterangan
1	context size	4-16	Jumlah sampel konteks terdekat yang digunakan pada proses *retrieval*
2	d main	96-256	Dimensi representasi utama pada model
3	encoder blocks	0-2	Jumlah blok *encoder* sebelum proses *retrieval*
4	predictor blocks	1-2	Jumlah blok *predictor* setelah proses *retrieval*
5	context dropout	0.00-0.35	Nilai *dropout* pada representasi konteks
6	dropout0	0.05-0.40	Nilai *dropout* pada blok *feed-forward* pertama
7	dropout1	0.00-0.30	Nilai *dropout* pada blok *feed-forward* kedua

8	learning rate	1e-5-8e-4	Laju pembelajaran pada *optimizer* AdamW
9	weight decay	1e-6-5e-4	Regularisasi bobot pada *optimizer* AdamW

Berdasarkan hasil *hyperparameter tuning*, konfigurasi terbaik diperoleh pada *seed* 5 dengan *context size* sebesar 9, *d main* sebesar 133, tanpa blok *encoder*, dan menggunakan dua blok *predictor*. Nilai *dropout* terbaik yang digunakan terdiri atas *context dropout* sebesar 0,0099, *dropout0* sebesar 0,3967, dan *dropout1* sebesar 0,2990. Model dilatih menggunakan *optimizer* AdamW dengan *learning rate* sebesar 0,0007403 dan *weight decay* sebesar 0,00000842. Konfigurasi terbaik dicapai pada *epoch* ke-22.

Tabel 3.4 Konfigurasi Akhir Model TabR

No	Parameter	Nilai Akhir
1	Seed	5
2	Batch size	128
3	Patience	6
4	Best epoch	22
5	Context size	9
6	D main	133
7	Encoder blocks	0
8	Predictor blocks	2
9	Context dropout	0,0099
10	Dropout0	0,3967
11	Dropout1	0,2990
12	Optimizer	AdamW
13	Learning rate	0,0007403
14	Weight decay	0,00000842
15	Normalization	LayerNorm
16	Activation	ReLU

Model dilatih secara iteratif menggunakan data pelatihan, kemudian performanya diamati melalui data validasi. Hasil validasi digunakan untuk menentukan konfigurasi terbaik serta memastikan bahwa model mampu mempelajari pola data secara stabil. Setelah konfigurasi terbaik diperoleh, model disimpan dan digunakan pada tahap uji coba sistem untuk mengklasifikasikan kondisi *anxiety* tinggi dan *anxiety* rendah berdasarkan fitur *POC-ABS* yang telah diekstraksi.

3.6 Uji Coba Sistem

Pada tahap ini dilakukan pengujian terhadap model yang telah dilatih untuk menilai performa klasifikasi kondisi *anxiety* tinggi dan *anxiety* rendah.

3.6.1 Skema Pengujian

Skema pengujian pada penelitian ini terdiri atas pengujian internal dan pengujian eksternal. Pengujian internal dilakukan menggunakan data validasi yang berasal dari pembagian *dataset* pelatihan untuk memantau performa model selama proses *training* dan mendeteksi potensi *overfitting*. Sementara itu, pengujian eksternal dilakukan menggunakan 20 data video uji berbasis subjek yang tidak dilibatkan dalam proses pelatihan maupun validasi model, terdiri atas 10 video kondisi *anxiety* tinggi dan 10 video kondisi *anxiety* rendah dengan variasi kondisi *before* ujian dan *after* ujian. Seluruh data pengujian diproses menggunakan tahapan *preprocessing* yang sama dengan data pelatihan, meliputi penyeimbangan kelas, imputasi nilai kosong, dan normalisasi fitur untuk menjaga konsistensi distribusi data.

Pada proses pengujian eksternal, setiap *event micro-expression* hasil *spotting onset–apex–offset* diperlakukan sebagai sampel independen yang kemudian diklasifikasikan ke dalam kategori *anxiety* tinggi atau *anxiety* rendah menggunakan model TabR. Hasil prediksi selanjutnya dibandingkan dengan label aktual untuk menghitung metrik evaluasi berupa *accuracy*, *precision*, *recall*, *F1-score*, dan *confusion matrix*.

3.6.2 Confusion Matrix

Confusion matrix digunakan sebagai alat utama untuk mengevaluasi kinerja model *Tabular Retrieval Network* (TabR) dalam mengklasifikasikan

kondisi *anxiety* tinggi dan *anxiety* rendah. Matriks ini membandingkan hasil prediksi model dengan label sebenarnya yang terdiri dari *true positive*, *true negative*, *false positive*, dan *false negative*, sehingga memungkinkan analisis terhadap pola kesalahan klasifikasi yang terjadi. Berdasarkan *confusion matrix* tersebut, dihitung metrik evaluasi yang meliputi *accuracy*, *precision*, *recall*, dan *F1-score* untuk memberikan gambaran menyeluruh mengenai performa model dalam mengklasifikasikan masing-masing kelas.

3.6.3 Interpretasi Hasil Evaluasi

Hasil evaluasi dianalisis untuk menilai tingkat akurasi model serta keseimbangan performa antar kelas. Analisis terhadap nilai *precision* dan *recall* dilakukan untuk menilai kemampuan model dalam mendeteksi kondisi *anxiety* secara tepat, sedangkan nilai *F1-score* digunakan sebagai indikator performa keseluruhan yang mempertimbangkan keseimbangan antara *precision* dan *recall*.

BAB IV. ANALISA DAN PERANCANGAN SISTEM

4.1 Analisis Sistem

Analisis sistem dilakukan untuk mengidentifikasi kebutuhan teknis yang diperlukan dalam membangun sistem web pengenalan *micro expression* penderita *anxiety* sesuai dengan rancangan yang telah ditetapkan. Fokus analisis mencakup aspek perangkat keras yang digunakan selama proses pengembangan, pelatihan model *deep learning*, dan pengujian sistem, serta perangkat lunak yang mendukung implementasi algoritma *deep learning* dan pengembangan aplikasi web. Untuk membangun sistem tersebut, dibutuhkan spesifikasi teknis yang sesuai, baik dari sisi perangkat keras seperti komputer pengembang dengan spesifikasi tinggi untuk proses *training* model, server untuk *deployment* aplikasi web, dan perangkat *capturing* video untuk pengujian, maupun dari sisi perangkat lunak seperti bahasa pemrograman *python*, *framework*, *library* pendukung, dan platform pengembangan yang digunakan.

4.2 Kebutuhan Perangkat Keras

Implementasi deteksi *micro expression* penderita *anxiety* membutuhkan dukungan perangkat keras yang memadai. Kebutuhan perangkat keras mencakup spesifikasi dari perangkat yang digunakan selama proses pengembangan dan pengujian sistem. Tabel 4.1 menunjukkan spesifikasi perangkat keras yang digunakan dalam penelitian ini, baik untuk proses pengembangan maupun pengujian aplikasi. Perangkat laptop digunakan untuk mendukung proses pengembangan serta pelatihan model, pengujian hingga implementasi aplikasi akhir, seperti yang ditunjukkan pada Table 4.1 berikut:

Tabel 4.1 Perangkat yang digunakan

No.	Perangkat Keras	Keterangan
1.	Laptop	Lenovo Ideapad Gaming
2.	Processor	Amd Ryzen 7 5800H

3.	VGA/GPU	Nvidia Gforce RTX 3050 4GB Vram
4.	Ram	24GB DDR4
5.	SSD	1.5TB M.2 NVMe

4.3 Kebutuhan Perangkat Lunak

Selain perangkat keras, penelitian ini juga membutuhkan perangkat lunak yang meliputi *tools* serta platform yang digunakan mulai dari pengolahan data, pelatihan model, hingga pembuatan sistem web. Rincian kebutuhan perangkat lunak yang digunakan ditampilkan pada Tabel 4.2 di bawah ini :

Tabel 4.2 Rincian Kebutuhan Perangkat Lunak

No.	Perangkat Lunak	Keterangan
1.	Fedora Linux 43	Sistem operasi yang digunakan untuk membuat dan menjalankan sistem
2.	Visual Studio Code	*Text editor* untuk mengolah kode yang berguna dalam pembuatan sistem dan mengolah data dari berbagai bahasa pemrograman
3.	Pytorch python	*Framework machine learning open-source* berbasis *Python* untuk *deep learning* dan penelitian AI
4.	Tanstack Start	*Framework full-stack* modern berbasis *JavaScript/TypeScript* dirancang untuk membangun aplikasi web
5.	Pi	AI *harness open-source* yang minimalis dan *opinionated*, dirancang untuk membantu *coding* melalui antarmuka terminal (TUI)

4.4 Analisis Pengguna

Analisis pengguna bertujuan untuk mengidentifikasi pihak atau aktor yang akan berinteraksi langsung dengan sistem serta memahami peran dan

aktivitas mereka dalam penggunaan aplikasi web. Pengguna utama dari sistem ini terdiri dari dua pihak, yakni Tenaga Medis atau Psikolog dan Peneliti. Melalui sistem web yang dikembangkan, tenaga medis dapat langsung mengunggah video testing pasien ke dalam sistem, lalu sistem akan melakukan proses analisis dan klasifikasi *micro expression* secara otomatis dengan menghasilkan *output* berupa label yang menunjukkan tingkat *anxiety* yang terdeteksi.

Sistem dirancang agar mudah digunakan oleh pengguna dengan latar belakang medis namun non-teknis dalam bidang *deep learning*, serta dapat diakses melalui web browser tanpa memerlukan instalasi *software* khusus. *Interface* yang sederhana dan *user friendly* memungkinkan tenaga medis untuk fokus pada interpretasi hasil analisis tanpa perlu memahami detail teknis sistem yang kompleks, sehingga dapat mendukung proses diagnosis dan evaluasi kondisi *anxiety* pasien secara lebih objektif dan terstandar.

4.5 Analisis Kebutuhan Sistem

4.5.1 Kebutuhan Fungsional

Kebutuhan fungsional adalah fitur utama yang harus tersedia agar sistem dapat menjalankan seluruh alur proses, meliputi *input* data, *preprocessing*, deteksi wajah, ekstraksi fitur, *Spotting*, seleksi fitur, hingga klasifikasi. Fitur-fitur ini dirancang untuk mendukung interaksi langsung antara pengguna dan aplikasi berbasis web, serta memastikan sistem bekerja sesuai dengan tujuan yang telah ditetapkan. Rincian kebutuhan fungsional tersebut disajikan pada Tabel 4.3.

Tabel 4.3 Rincian kebutuhan fungsional

No.	Fungsionalitas	Keterangan
1.	Up*load* Video	Sistem menyediakan fitur untuk mengunggah *file* video testing dalam format yang didukung (MP4, AVI, WEBM).
2.	*Preprocessing* Video	Sistem melakukan *preprocessing* pada video yang diunggah untuk mempersiapkan data sebelum analisis.

3.	Deteksi Wajah	Sistem mendeteksi area wajah dalam setiap *frame* video untuk fokus analisis *micro expression*. Area yang diambil yaitu alis dan mulut.
4.	Ekstraksi Fitur	Ekstraksi fitur menggunakan metode POC-ABS dengan block size 7x7.
5.	*Spotting onset-apex-offset*	Sistem menentukan fase *onset*, *apex*, dan *offset* untuk mengambil bagian pergerakan *micro-expression* yang paling relevan sebelum proses klasifikasi.
6.	Labeling Klasifikasi *Anxiety*	Sistem mengklasifikasikan *micro expression* menggunakan model *deep learning* untuk menentukan tingkat *anxiety*.
7.	Menampilkan Hasil	Hasil klasifikasi ditampilkan dalam bentuk label atau kategori *anxiety* yang terdeteksi.
8.	Menyimpan Hasil	Sistem menyimpan hasil analisis beserta informasi video.

4.5.2 Kebutuhan Non Fungsional

Kebutuhan non-fungsional menggambarkan karakteristik kualitas yang harus dipenuhi sistem selama digunakan, mencakup aspek performa, kemudahan penggunaan, keandalan, serta dukungan sistem. Kebutuhan ini tidak berfokus pada fitur utama, melainkan pada bagaimana sistem bekerja secara optimal dan konsisten dalam berbagai kondisi. Rincian kebutuhan non-fungsional tersebut disajikan pada Tabel 4.4.

Tabel 4.4 Rincian kebutuhan non-fungsional

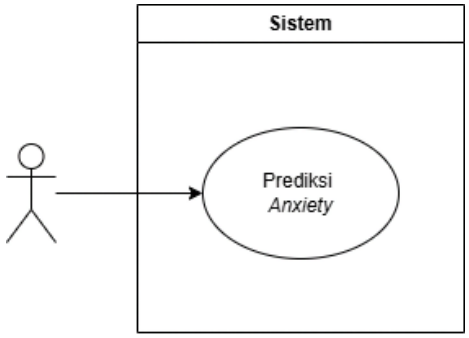
No.	Jenis	Keterangan
1.	Usability	Aplikasi memiliki antarmuka sederhana dan mudah digunakan oleh peneliti, psikolog, atau tenaga medis tanpa latar belakang teknis yang mendalam dalam *computer vision*.

2.	Performance	Sistem mampu memproses data video *micro expression* secara efisien mulai dari tahap *preprocessing* hingga klasifikasi menggunakan model *deep learning*, dengan waktu pemrosesan yang cepat.
3.	Compatibility	Aplikasi dapat berjalan pada berbagai Sistem Operasi serta mendukung perangkat dengan spesifikasi minimum RAM 4GB dan prosesor Intel i5 atau setara agar proses pengolahan video dan inferensi model dapat berjalan dengan baik.
4.	Reliability	Aplikasi dapat beroperasi secara stabil selama proses analisis video berlangsung tanpa mengalami *crash*, *error*, atau penurunan performa yang signifikan.

4.6 Perancangan Sistem

4.6.1 Perancangan *Use Case*

Perancangan *use case* dilakukan untuk menggambarkan interaksi antara pengguna dengan sistem deteksi *micro expression* penderita *anxiety*. Gambar 4.1 menunjukkan aktivitas utama yang dapat dilakukan oleh *user* sebagai pengguna, yaitu meng*input*kan data testing. *Use case* diagram ini menjadi dasar pengembangan fitur sistem web yang memungkinkan pengguna berinteraksi secara langsung dengan sistem deteksi *micro expression* penderita *anxiety*.



Gambar 4.1 *Use Case* Diagram

Penjelasan lebih detail mengenai *use case* diagram pada Gambar 4.1 dijabarkan pada Tabel 4.5 yang masing-masing mendeskripsikan *use case* mengisi kuesioner, submit data, dan melihat hasil deteksi. Tabel-tabel tersebut memuat informasi penting seperti aktor, deskripsi, prekondisi, alur utama, serta postkondisi yang menggambarkan interaksi antara pengguna dengan sistem secara ringkas.

Deskripsi *use case* ini digunakan sebagai acuan dalam perancangan dan implementasi sistem agar setiap fungsi utama dapat berjalan sesuai dengan kebutuhan. Rincian dari masing-masing *use case* dapat dilihat pada Tabel 4.5.

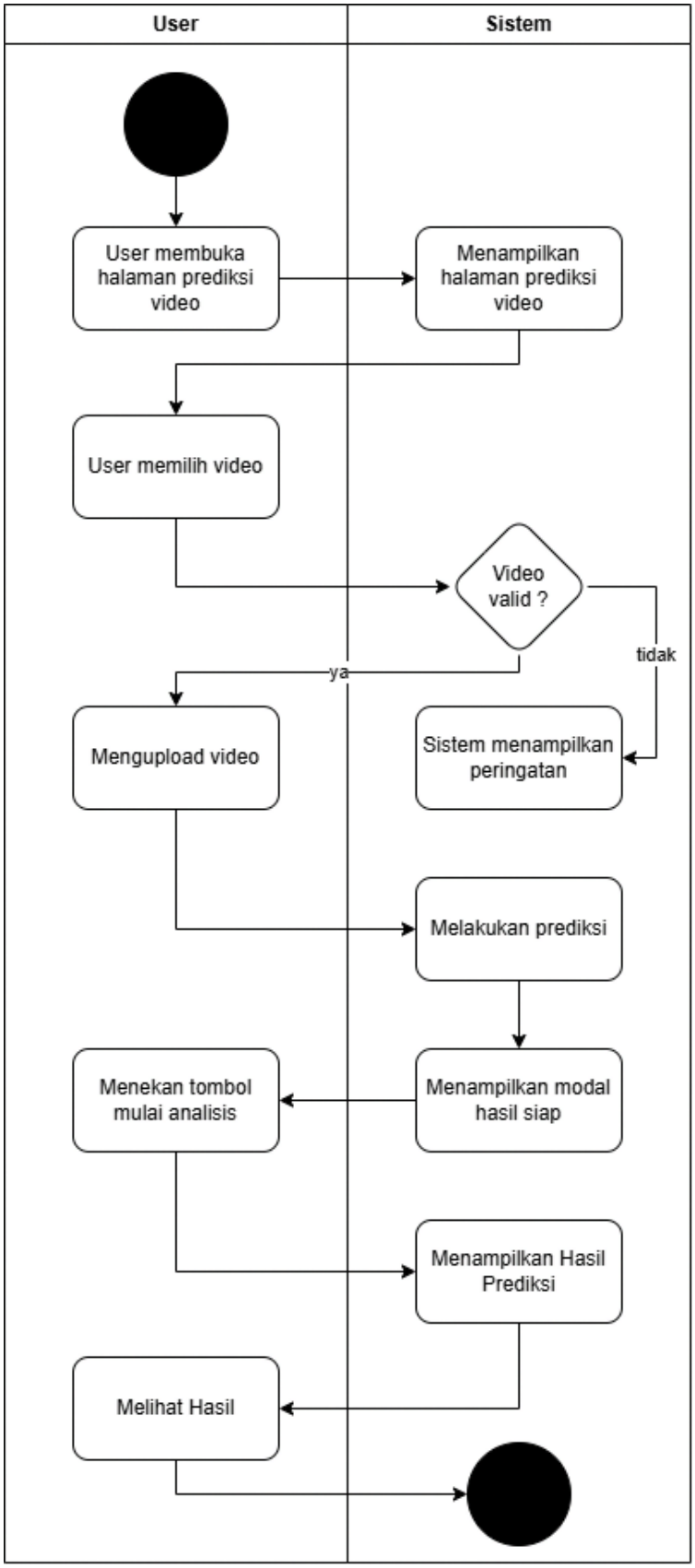
Tabel 4.5 Rincian *Use case* Prediksi *Anxiety*

Atribut	Keterangan
Nama *Use Case*	Prediksi *Anxiety*
Aktor	*User*
Deskripsi	*User* mengunggah video untuk diproses oleh system sehingga menghasilkan prediksi tingkat kecemasan
Prekondisi	Aplikasi telah dibuka dan *user* berada di halaman prediksi video.
Alur Utama	*User* memilih video → *user* up*load* video → *system* menyiapkan hasil prediksi → menekan tombol mulai → *system* menampilkan hasil prediksi dan *event* pada video.
Alternatif	Jika video tidak valid atau up*load* gagal → *system* menampilkan peringatan. Jika pratinjau video tidak tersedia → video tetap dapat di up*load* dan dilihat pada halaman analisis
Postkondisi	Hasil prediksi *anxiety*, daftar *event*, dan *chart* analisis berhasil ditampilkan

4.6.2 Activity Diagram

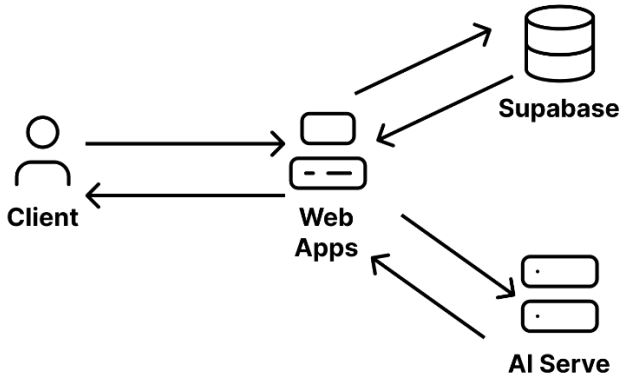
Untuk menggambarkan alur proses dari setiap *use case* secara lebih rinci dan terstruktur. Diagram ini menunjukkan urutan aktivitas yang dilakukan oleh pengguna dan sistem, mulai dari *input* hingga menghasilkan *output* berupa hasil deteksi *anxiety*.

Pada Gambar 4.2 disajikan *activity* diagram yang merepresentasikan alur aktivitas dari masing-masing fungsi utama sistem, yaitu mengisi kuesioner. Setiap diagram memperlihatkan interaksi antara pengguna dan sistem serta proses yang terjadi di dalam sistem hingga menghasilkan *output* yang diharapkan.



Gambar 4.2 *Activity* Diagram Prediksi *Anxiety*

4.6.3 Perancangan Arsitektur Sistem



Gambar 4.3 Arsitektur Sistem

Perancangan arsitektur sistem pengenalan *micro-expression* penderita *anxiety* ini dibangun menggunakan pendekatan *de-coupled architecture* (arsitektur terdistribusi). Seperti yang ditunjukkan pada Gambar 4.3 , arsitektur ini memisahkan beban kerja operasional ke dalam beberapa entitas untuk memastikan bahwa proses ekstraksi fitur *deep learning* yang berat tidak mengganggu responsivitas antarmuka aplikasi.

Arsitektur ini terdiri dari empat komponen utama yang saling berinteraksi:

1. Client Merupakan sisi antarmuka pengguna (*frontend*) yang berinteraksi langsung dengan *user*. Klien dibangun dengan memanfaatkan *library* React, TypeScript, dan Tailwind CSS. Pada komponen ini, sistem memfasilitasi interaksi seperti pengisian kuesioner, perekaman video ekspresi wajah pasien secara *real-time* menggunakan antarmuka *browser*, serta visualisasi hasil persentase prediksi klasifikasi.
2. Web Apps Berperan sebagai *application server* utama yang dikembangkan menggunakan *framework fullstack* TanStack Start. Komponen ini bertindak sebagai pusat kendali (jembatan) yang mengelola logika bisnis aplikasi, melakukan validasi data dari *Client*, serta mengatur komunikasi data melalui API menuju layanan basis data maupun server AI.
3. Supabase Berfungsi sebagai sistem manajemen basis data relasional dan layanan *backend as a service*. Komponen ini mengelola autentikasi

manajemen sesi pengguna serta menyimpan seluruh data persisten berbasis PostgreSQL. Data yang disimpan mencakup rekapitulasi skor kuesioner pengguna, *metadata* video, serta log riwayat hasil klasifikasi *anxiety*.

4. AI Serve Merupakan *backend* yang berfokus penuh pada pemrosesan *computer vision* dan inferensi model. Dibangun menggunakan *framework* FastAPI berbasis Python, server ini bertanggung jawab untuk mengeksekusi tahapan analisis data yang berat. Proses yang ditangani meliputi: *preprocessing* citra video, pelacakan *facial landmark* menggunakan Dlib, kalkulasi pergerakan mikro dengan algoritma *Phase Only Correlation with All Block Search* (POC-ABS), pembentukan data tabular, hingga tahap penentuan kelas *anxiety* menggunakan model *Tabular Retrieval Network* (TabR) yang telah dilatih.

4.6.4 Perancangan Database



Gambar 4.4 *Entity Relationship Diagram*

Perancangan *database* sistem pengenalan *micro-expression* untuk deteksi tingkat kecemasan dirancang menggunakan pendekatan relasional yang terdiri dari lima tabel utama. Struktur *database* ini mengakomodasi seluruh alur data mulai dari informasi pengguna, daftar pertanyaan kuesioner, pilihan jawaban, rekaman respons, hingga detail jawaban per pertanyaan. Penjelasan masing-masing tabel adalah sebagai berikut.

1. **Tabel Video *Predictions***

   Tabel ini berfungsi untuk menyimpan informasi utama dari video yang dianalisis serta hasil prediksi *anxiety* yang dihasilkan oleh sistem. Tabel ini memiliki relasi *one-to-many* ke tabel video_prediction_*event*s melalui *field* prediction_id. Rincian kolom pada tabel video_predictions adalah sebagai berikut.

Tabel 4.6 Deskripsi Tabel Video *Predictions*

No	Nama Field	Tipe Data	Keterangan
1	id	uuid	*Primary key*, identitas unik data prediksi video
2	video_*path*	text	Lokasi atau *path file* video yang diunggah
3	video_format	text	Format *file* video yang digunakan
4	video_mime_type	text	Tipe MIME dari *file* video
5	video_size_bytes	int4	Ukuran *file* video dalam satuan *byte*
6	access_token_hash	text	Hash token akses untuk membuka hasil prediksi
7	access_token_expires_at	timestamp	Waktu kedaluwarsa token akses
8	label	text	Label hasil klasifikasi *anxiety*
9	probability_*anxiety*_tinggi	float8	Nilai probabilitas untuk kelas *anxiety* tinggi
10	threshold	float8	Nilai ambang batas yang digunakan dalam klasifikasi
11	aggregation	text	Metode agregasi hasil prediksi

12	model_exp_name	text	Nama eksperimen model yang digunakan
13	model_version	text	Versi model yang digunakan
14	*frame*_count	int4	Jumlah *frame* pada video
15	duration_seconds	float8	Durasi video dalam satuan detik
16	fps	float8	Jumlah *frame* per detik pada video
17	*event*_count	int4	Jumlah *event micro-expression* yang terdeteksi
18	*error*_message	text	Pesan *error* apabila proses analisis gagal
19	status	text	Status proses analisis video
20	created_at	timestamp	Waktu pembuatan data prediksi
21	updated_at	timestamp	Waktu pembaruan data prediksi
22	playback_video_*path*	text	Lokasi *file* video hasil *playback*
23	*spotting*_signal	jsonb	Data sinyal *spotting* dalam format JSON

2. Tabel Video *Prediction Event*s

Tabel ini berfungsi untuk menyimpan detail *event micro-expression* yang terdeteksi dari setiap video. Setiap data *event* terhubung dengan tabel video_predictions melalui *field* prediction_id. Rincian kolom pada tabel video_prediction_*event*s adalah sebagai berikut.

Tabel 4.7 Deskripsi Video *Prediction Event*s

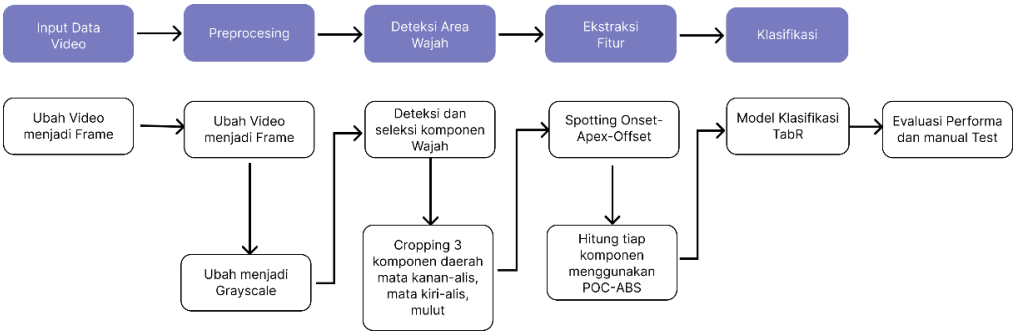
No	Nama Field	Tipe Data	Keterangan
1	id	uuid	*Primary key*, identitas unik *event* prediksi
2	prediction_id	uuid	*Foreign key* yang mengacu pada tabel video_predictions
3	*event*_no	int4	Nomor urut *event micro-expression*
4	onset_*frame*	int4	Nomor *frame* awal munculnya *event*
5	apex_*frame*	int4	Nomor *frame* puncak pergerakan *event*
6	offset_*frame*	int4	Nomor *frame* akhir *event*
7	onset_time_seconds	float8	Waktu *onset* dalam satuan detik
8	apex_time_seconds	float8	Waktu *apex* dalam satuan detik

9	offset_time_seconds	float8	Waktu *offset* dalam satuan detik
10	duration_*frame*s	int4	Durasi *event* berdasarkan jumlah *frame*
11	duration_seconds	float8	Durasi *event* dalam satuan detik
12	probability_*anxiety*_tinggi	float8	Nilai probabilitas *anxiety* tinggi pada *event*
13	label	text	Label hasil klasifikasi pada *event*

4.6.5 Alur Proses Sistem

Alur proses sistem pengenalan *micro-expression anxiety* pada penelitian ini dirancang untuk menganalisis data video wajah dan mengidentifikasi pola *micro-expression* yang mengindikasikan kondisi kecemasan. Sistem ini mengintegrasikan teknik *computer vision* dan *deep learning* untuk menghasilkan klasifikasi kondisi *anxiety*.

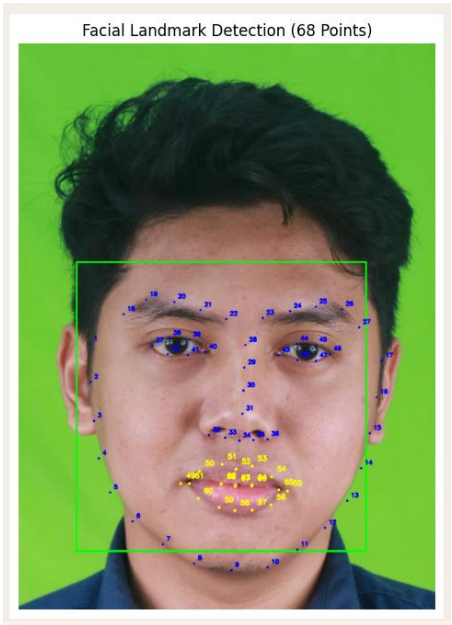
Sistem yang diusulkan terdiri atas enam tahap utama, yaitu *input* data video, *preprocessing*, deteksi area komponen wajah, ekstraksi fitur, seleksi fitur, dan klasifikasi. Alur kerja sistem pengenalan *micro-expression* penderita *anxiety* pada penelitian ini mengikuti tahapan yang ditunjukkan pada Gambar



Gambar 4.5 Alur Perancangan Sistem

Tahap pertama yaitu *input* data video yang dilanjutkan dengan proses *preprocessing*, di mana video yang di *input*kan akan diubah menjadi *frame*. Setiap *frame* kemudian dikonversi ke dalam format gray*scale* agar memudahkan proses pengolahan citra.

Pada tahap deteksi area komponen wajah, sistem memanfaatkan pustaka Dlib untuk mendeteksi wajah serta menentukan 68 titik *facial landmark* sebagai acuan penentuan lokasi komponen wajah. Berdasarkan titik-titik *landmark* tersebut, sistem mengidentifikasi area wajah yang relevan untuk analisis *micro-expression*. Visualisasi hasil deteksi *facial landmark* dan penentuan area wajah ditunjukkan pada Gambar 4.6.



Gambar 4.6 Deteksi 68 *Facial Landmark* menggunakan Dlib

Setelah wajah berhasil terdeteksi, dilakukan proses *cropping* pada tiga area komponen utama yang berperan penting dalam ekspresi wajah, yaitu daerah

mata kanan-alis, mata kiri-alis, dan mulut. Penentuan area *cropping* dilakukan berdasarkan indeks *facial landmark* 68 titik yang digunakan sebagai acuan lokasi komponen wajah. Konfigurasi area komponen wajah pada penelitian ini ditetapkan sebagai berikut:

- Area mulut ditentukan oleh *landmark* 48-67.
- Area mata-alis kiri ditentukan oleh *landmark* 17-21 (alis) dan 36-41(mata).
- Area mata-alis kanan ditentukan oleh *landmark* 22-26 (alis) dan 42-47 (mata).
- Setiap *region of interest* (ROI) diperluas menggunakan *padding* horizontal sebesar 6 piksel dan *padding* vertikal sebesar 8 piksel untuk memastikan area ekspresi tercakup secara optimal.
- Hasil *cropping* setiap ROI kemudian dinormalisasi ke ukuran tetap, yaitu 70 × 35 piksel untuk area mulut serta 48 × 32 piksel untuk area mata–alis kiri dan mata–alis kanan.

Contoh hasil *cropping* pada masing-masing komponen wajah ditunjukkan pada Gambar 4.7.



Gambar 4.7 *Cropping* Komponen Wajah

Gambar Hasil *cropping* area mata-alis kanan, mata-alis kiri, dan mulut

Tahap berikutnya yaitu ekstraksi fitur, di mana dilakukan *spotting* untuk mencari *frame onset-apex-offset* lalu hasil *spotting* dihitung nilai fiturnya menggunakan metode *Phase Only Correlation with All Block Search* (POC-ABS). Proses ini dilakukan pada masing-masing komponen wajah untuk menangkap pergerakan halus yang merepresentasikan *micro-expression*. Seluruh fitur hasil ekstraksi kemudian digabungkan dan diratakan melalui proses *merge* dan *flatten* sehingga membentuk data tabular yang siap

digunakan pada tahap klasifikasi. Contoh struktur data yang dihasilkan oleh metode POC-ABS dapat dilihat pada Tabel berikut:

Tabel 4.8 Contoh bentuk hasil data POC-ABS

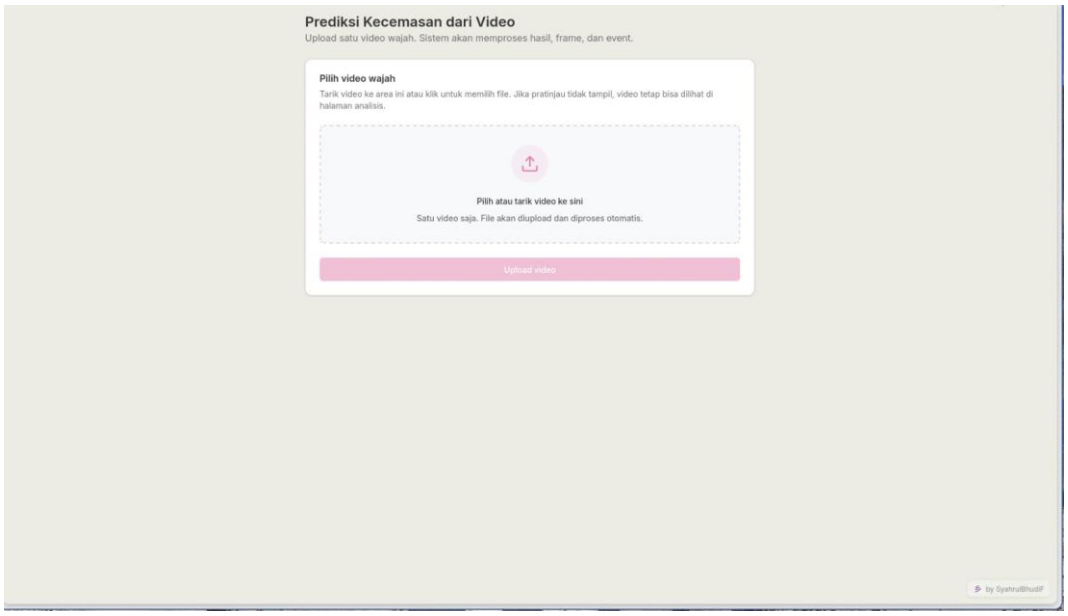
Sample	**ROI Mata-Alis Kanan**	**ROI Mata-Alis Kiri**	**ROI Mulut**	**Label**
S1	(X_1, Y_1, θ_1, M_1)	(X_2, Y_2, θ_2, M_2)	(X_3, Y_3, θ_3, M_3)	*Anxiety*
S2	(X_1, Y_1, θ_1, M_1)	(X_2, Y_2, θ_2, M_2)	(X_3, Y_3, θ_3, M_3)	Normal

Data terdiri dari tiga kelompok fitur berdasarkan *region of interest* (ROI), yaitu mata-alis kanan, mata-alis kiri, dan mulut. Setiap *region of interest* (ROI) direpresentasikan oleh beberapa parameter fitur sebagai berikut:

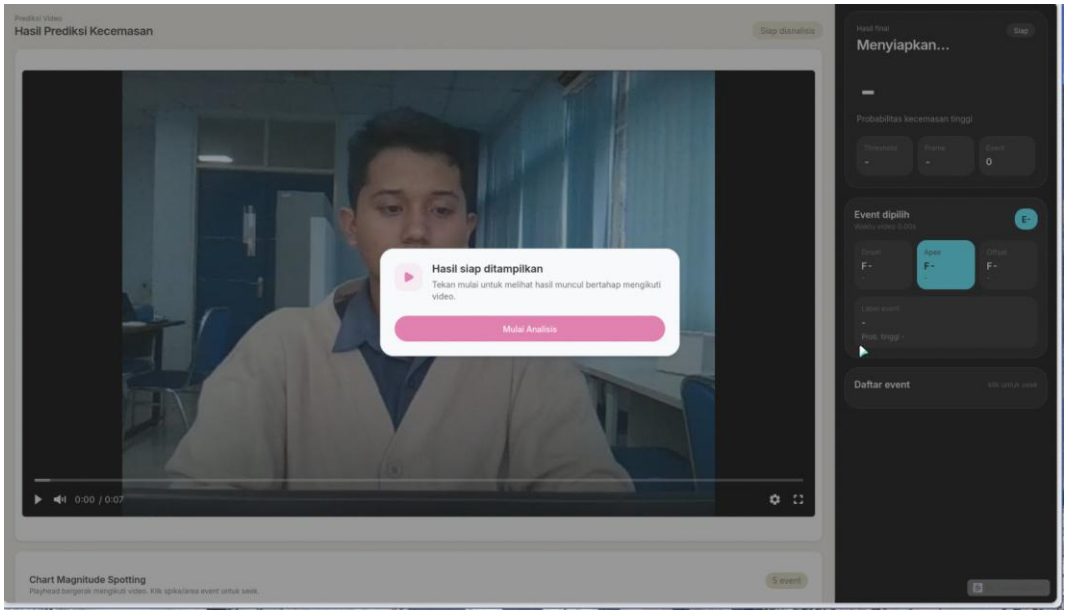
- X: pergeseran horizontal antar-*frame* yang merepresentasikan gerakan mikro ke arah kiri atau kanan.
- Y: pergeseran vertikal antar-*frame* yang merepresentasikan gerakan mikro ke arah atas atau bawah.
- θ (theta): arah dominan pergerakan wajah yang menunjukkan orientasi vektor gerak.
- Magnitudo: besar intensitas pergerakan yang menggambarkan kekuatan gerakan mikro hasil POC-ABS.

Karena seluruh fitur bersifat numerik dan terstruktur dalam bentuk vektor satu dimensi, data hasil ekstraksi POC-ABS sangat sesuai untuk diproses menggunakan model *Tabular Retrieval Network* (TabR) yang memang dirancang untuk menangani data tabular dan memodelkan relasi kompleks antar fitur secara efektif. Pada tahap selanjutnya, data tabular hasil ekstraksi digunakan sebagai masukan ke dalam model klasifikasi TabR untuk melakukan klasifikasi kondisi *anxiety*.

4.6.6 Perancangan Antar Muka Sistem



Gambar 4.8 Halaman *Input* Video



Gambar 4.9 Tampillan Sebelum Hasil Muncul



Gambar 4.10 Tampilan Setelah Hasil Muncul



Gambar 4.11 Tampilan *Chart Spotting*

BAB V. IMPLEMENTASI DAN PENGUJIAN

Bab ini membahas mengenai penerapan dan pengujian sistem "Pengenalan Wajah *Anxiety* Menggunakan Metode Phase Only Correlation with All Block Search dan Model Arsitektur Tabular Retrieval Network". Langkah-langkah teknis yang telah dirancang pada tahap sebelumnya akan diimplementasikan untuk membangun sistem secara menyeluruh, mulai dari pengolahan *dataset micro-expression*, *preprocessing* citra wajah, deteksi *facial landmark* menggunakan Dlib dengan 68 titik fitur wajah, *spotting micro-expression* untuk menentukan fase *onset*, *apex*, dan *offset*, ekstraksi fitur pergerakan mikro menggunakan *Phase Only Correlation with All Block Search* (POC-ABS), pembentukan data tabular, hingga klasifikasi menggunakan model *Tabular Retrieval Network* (TabR).

Sistem mengintegrasikan kamera Intel *Real*Sense D415 dengan resolusi 640 × 480 piksel dan *frame rate* 60 FPS, serta webcam bawaan laptop sebagai kamera cadangan. Data hasil perekaman video ekspresi wajah dilakukan *spotting* untuk mengidentifikasi durasi kemunculan ekspresi mikro, kemudian diekstraksi menggunakan metode POC-ABS dengan *block size* 7 × 7 pada tiga area komponen wajah, yaitu mata–alis kanan, mata–alis kiri, dan mulut, untuk menangkap pergeseran piksel mikroskopis antar *frame*. Hasil ekstraksi fitur kemudian ditransformasikan ke dalam format data tabular yang terstruktur dan diolah menggunakan model TabR dengan mekanisme *retrieval-augmented learning* berbasis FAISS. Hasil pengujian akan menggambarkan akurasi kinerja sistem dalam mengklasifikasikan tingkat kecemasan (*anxiety*) pada mahasiswa berusia 20–22 tahun, dengan evaluasi berdasarkan metrik *accuracy*, *precision*, *recall*, dan *F1-score.*

5.1 Pengolahan *Dataset* dan Implementasi Model

Implementasi pengolahan *dataset* merupakan tahap awal dalam membangun sistem pengenalan wajah *anxiety*. Tahap ini mencakup serangkaian proses mulai dari pengambilan data primer, perekaman video ekspresi wajah, validasi data oleh pakar, hingga pembentukan *dataset* final yang siap digunakan untuk pelatihan model. Seluruh proses dilaksanakan dengan mengacu pada metodologi yang telah dirancang pada Bab III.

5.1.1 Implementasi Pengolahan Data

1. Konversi video ke gambar (*Frame* per Second)

Tahap pertama dalam pengolahan data adalah mengubah video hasil perekaman menjadi kumpulan *frame* gambar. Proses ini dilakukan karena metode POC-ABS membutuhkan data citra berurutan untuk membandingkan perubahan antar *frame*. Dengan demikian, video tidak langsung diproses sebagai satu *file* utuh, tetapi dipecah menjadi beberapa *frame* berformat .jpg.

Alur utama proses konversi video ke *frame* terdiri atas tiga bagian, yaitu membaca video, menyimpan *frame* ke direktori *output*, dan menjalankan *pipeline* konversi sesuai struktur *dataset*.

Membaca Video Menjadi *Frame*

```
def load_video_frames(video_path: Path) -> list:
    frames = []
    cap = cv2.VideoCapture(str(video_path))

    if not cap.isOpened():
        raise ValueError(f"Gagal membuka video: {video_path}")

    while True:
        ret, frame = cap.read()

        if not ret:
            break

        frames.append(frame)

    cap.release()
    return frames
```

Fungsi *load*_video_*frame*s() digunakan untuk membaca *file* video dari direktori *dataset* dan mengubahnya menjadi kumpulan *frame* yang tersimpan secara berurutan.

- Video dibuka menggunakan *cv2.VideoCapture().*
- Sistem membaca *frame* satu per satu menggunakan *cap.read().*
- Nilai *ret* digunakan untuk mengecek apakah *frame* masih tersedia.
- Jika *ret* bernilai *False*, proses pembacaan dihentikan.
- Setiap *frame* yang berhasil dibaca disimpan ke dalam *list frames*.

- Urutan *frame* tetap dipertahankan karena POC-ABS membutuhkan perbandingan antara *frame* sebelumnya dan *frame* berikutnya.

Dengan proses ini, video berhasil diubah menjadi kumpulan *frame* yang siap digunakan pada tahap analisis pergerakan mikro antar *frame*.

Menyimpan *Frame* ke Direktori *Output*

```
        def save_frames_to_disk(frames: list, output_dir: Path, prefix: str = 
"frame"):
            output_dir.mkdir(parents=True, exist_ok=True)
            saved_paths = []

            for idx, frame in enumerate(frames, start=1):
                frame_path = output_dir / f"{prefix}_{idx:05d}.jpg"
                cv2.imwrite(str(frame_path), frame)
                saved_paths.append(frame_path)

            return saved_paths
```

Fungsi *save_frames_to_disk()* digunakan untuk menyimpan *frame* yang sudah berhasil dibaca ke dalam direktori *output*.

- Direktori *output* dibuat otomatis menggunakan *mkdir(parents=True, exist_ok=True).*
- Parameter *parents=True* memastikan *folder* induk ikut dibuat jika belum tersedia.
- Parameter *exist_ok=True* mencegah *error* jika *folder output* sudah ada.
- Setiap *frame* disimpan dalam format .jpg menggunakan *cv2.imwrite().*
- Nama *file* dibuat berurutan, seperti *frame*_00001.jpg, *frame*_00002.jpg, dan seterusnya.
- Penamaan berurutan digunakan untuk memudahkan pengecekan data dan menjaga urutan *frame* sesuai video asli.

Dengan proses ini, seluruh *frame* hasil ekstraksi tersimpan dalam *folder output* dan siap digunakan pada tahap *preprocessing* berikutnya.

Pipeline* Konversi Video ke *Frame

```
        def process_video_to_frames(video_path: Path, dataset_root: Path) -> 
list[Path]:
            relative_path = video_path.relative_to(dataset_root)
            sample_name = video_path.stem

            output_dir = dataset_root / "frames" / relative_path.parent / 
sample_name
```

```
        output_dir.mkdir(parents=True, exist_ok=True)

        frames = load_video_frames(video_path)
        frame_paths = save_frames_to_disk(frames, output_dir)

        print(f"Video {video_path.name}: {len(frames)} frame diekstrak")
        return frame_paths
```

Fungsi *process_video_to_frames()* berperan sebagai *pipeline* utama untuk mengatur proses konversi video menjadi *frame*.

- Fungsi ini menerima video_*path* sebagai lokasi *file* video.
- Parameter *dataset_root* digunakan sebagai direktori utama *dataset*.
- *relative_path* digunakan untuk menjaga struktur *folder* sesuai lokasi asal video.
- *sample_name* diambil dari nama *file* video tanpa ekstensi.
- *Folder output* dibuat berdasarkan kategori dan nama sampel video.
- Fungsi *load*_video_*frame*s() dipanggil untuk membaca seluruh *frame* dari video.
- Fungsi *save_frame*s_*to_disk()* dipanggil untuk menyimpan *frame* ke direktori *output*.
- Hasil akhir fungsi ini adalah daftar *path* dari *frame* yang berhasil diekstrak.

Output dari tahap ini adalah kumpulan *frame* berformat .jpg yang tersimpan secara berurutan. *Frame* tersebut kemudian digunakan pada tahap *preprocessing* wajah, deteksi *Region of Interest* (ROI), dan ekstraksi fitur pergerakan mikro menggunakan POC-ABS.

2. Deteksi Wajah dan *Facial Landmark*



Gambar 5.1 Contoh *Cropping* ROI

Tahap deteksi wajah dilakukan untuk mengidentifikasi posisi wajah dan titik-titik *facial landmark* pada setiap *frame*. Proses ini menggunakan

library Dlib dengan model 68 *facial landmark*. Titik *landmark* tersebut digunakan sebagai acuan untuk mengambil *Region of Interest* (ROI) pada area mata-alis kiri, mata-alis kanan, dan mulut.

Alur utama proses deteksi wajah dan ekstraksi ROI terdiri atas lima bagian, yaitu menyimpan informasi sampel video, menentukan konfigurasi ROI, menginisialisasi *detector* dan *predictor*, mendeteksi *landmark* wajah, serta mengekstraksi ROI dari *frame*.

Struktur Data Sampel Video

```
@dataclass(slots=True)
class VideoSample:
    video_path: Path
    relative_parent: Path
    stem: str

    @property
    def relative_video_key(self) -> Path:
        return self.relative_parent / self.stem
```

Class *VideoSample* digunakan untuk menyimpan informasi dasar dari video yang akan diproses.

- video_*path* digunakan untuk menyimpan lokasi *file* video.
- *relative_parent* digunakan untuk menyimpan lokasi relatif *folder* asal video.
- *stem* digunakan untuk menyimpan nama *file* video tanpa ekstensi.
- Properti *relative_video_key* digunakan untuk membentuk identitas video berdasarkan *folder* relatif dan nama *file*.

Dengan struktur ini, setiap video dapat dilacak berdasarkan lokasi asal dan nama sampelnya selama proses pengolahan data.

Konfigurasi *Region of Interest*

```
class FaceDetector:
    REGIONS = {
        "mulut": list(range(48, 68)),
        "mata_kiri": list(range(17, 22)) + list(range(36, 42)),
        "mata_kanan": list(range(22, 27)) + list(range(42, 48)),
    }

    TARGET_SIZE = {
        "mulut": (70, 35),
```

```
            "mata_kiri": (48, 32),
            "mata_kanan": (48, 32),
        }
```

Class *FaceDetector* menjadi komponen utama dalam proses deteksi wajah dan ekstraksi ROI. Pada class ini, area wajah yang diamati ditentukan melalui variabel *REGIONS*.

- Area mulut menggunakan indeks *landmark* 48 sampai 67.
- Area mata_kiri menggunakan gabungan *landmark* alis kiri dan mata kiri.
- Area mata_kanan menggunakan gabungan *landmark* alis kanan dan mata kanan.
- Variabel *TARGET_SIZE* digunakan untuk menentukan ukuran standar setiap ROI.
- ROI mulut diubah menjadi ukuran 70 × 35 piksel.
- ROI mata kiri dan mata kanan diubah menjadi ukuran 48 × 32 piksel.

Penentuan ROI ini dilakukan agar proses ekstraksi fitur hanya difokuskan pada area wajah yang digunakan dalam analisis *micro-expression anxiety*.

Inisialisasi *Detector* dan *Predictor*

```
    def __init__(self, predictor_path: Path):
        if not predictor_path.exists():
            raise FileNotFoundError(
                f"Predictor tidak ditemukan: {predictor_path}"
            )

        self._detector = dlib.get_frontal_face_detector()
        self._predictor = dlib.shape_predictor(str(predictor_path))
        self.padding_x = 6
        self.padding_y = 8
```

Fungsi *__init__()* digunakan untuk menyiapkan komponen utama deteksi wajah dan *facial landmark*. Pada fungsi ini, sistem memuat model *shape_predictor_68_face_landmarks.dat* sebagai *predictor landmark*, kemudian membuat *detector* wajah menggunakan *dlib.get_frontal_face_detector()* dan *predictor* menggunakan *dlib.shape_predictor()*.

- *predictor_path* digunakan sebagai lokasi *file* model *facial landmark*.
- Sistem mengecek ketersediaan *file predictor* sebelum proses dijalankan.
- *Detector* wajah digunakan untuk menemukan area wajah pada *frame*.
- *Predictor landmark* digunakan untuk menghasilkan 68 titik *landmark* wajah.
- Nilai *padding*_x dan *padding*_y digunakan untuk memperluas area ROI agar bagian ekspresi tidak terpotong terlalu sempit.

Deteksi *Facial Landmark*

```
def detect_landmarks(
    self,
    gray_image: np.ndarray
) -> dlib.full_object_detection | None:
    faces = self._detector(gray_image)

    if len(faces) == 0:
        return None

    face = max(
        faces,
        key=lambda rect: rect.width() * rect.height()
    )

    landmarks = self._predictor(gray_image, face)

    return landmarks
```

Fungsi *detect_landmark*s() digunakan untuk mendeteksi wajah dan menghasilkan 68 titik *facial landmark*.

- *Input* fungsi berupa *frame* dalam bentuk *grayscale*.
- Deteksi wajah dilakukan menggunakan *detector* dari Dlib.
- Jika tidak ada wajah yang terdeteksi, fungsi mengembalikan nilai *None.*
- Jika terdapat lebih dari satu wajah, sistem memilih wajah dengan ukuran terbesar.
- Pemilihan wajah terbesar dilakukan agar proses tetap fokus pada subjek utama.
- Setelah wajah dipilih, *predictor* Dlib digunakan untuk menghasilkan titik *landmark*.

Hasil dari fungsi ini adalah titik *landmark* wajah yang menjadi dasar untuk menentukan batas area ROI.

Ekstraksi *Region of Interest*

```
    def extract_region(
        self,
        image: np.ndarray,
        landmarks,
        indices: list[int]
    ) -> np.ndarray:
        pts = [
            (landmarks.part(i).x, landmarks.part(i).y)
            for i in indices
        ]

        xs, ys = zip(*pts)

        left = max(0, min(xs) - self.padding_x)
        top = max(0, min(ys) - self.padding_y)
        right = min(image.shape[1], max(xs) + self.padding_x)
        bottom = min(image.shape[0], max(ys) + self.padding_y)

        return image[top:bottom, left:right]
```

Fungsi *extract_region()* digunakan untuk mengambil potongan area wajah berdasarkan indeks *landmark* tertentu.

- Fungsi menerima citra asli, titik *landmark*, dan daftar indeks *landmark*.
- Koordinat X dan Y diambil dari *landmark* sesuai area yang dipilih.
- Sistem menghitung batas kiri, atas, kanan, dan bawah dari area ROI.
- *Padding* ditambahkan agar area ekspresi tidak terpotong terlalu sempit.
- Nilai batas ROI dibatasi agar tidak keluar dari ukuran citra.
- *Output* fungsi berupa potongan citra sesuai area ROI.

Dengan fungsi ini, setiap area wajah dapat dipotong berdasarkan posisi *landmark* yang terdeteksi pada *frame*.

Ekstraksi Seluruh ROI

```
    def extract_all_rois(
        self,
        image: np.ndarray
    ) -> dict[str, np.ndarray]:
        gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
        landmarks = self.detect_landmarks(gray)

        if landmarks is None:
            raise ValueError("Tidak ada wajah terdeteksi dalam frame")

        output = {}

        for region_name, indices in self.REGIONS.items():
            roi = self.extract_region(image, landmarks, indices)

            if roi.size == 0:
                raise ValueError(
                    f"ROI kosong untuk region: {region_name}"
                )

            target = self.TARGET_SIZE[region_name]
            output[region_name] = cv2.resize(roi, target)

        return output
```

Fungsi *extract_all_rois()* digunakan untuk mengambil seluruh ROI yang dibutuhkan dalam penelitian.

- *Frame* dikonversi dari format BGR ke *grayscale* menggunakan *cv2.cvtColor()*.
- Fungsi *detect_landmark*s() dipanggil untuk mendeteksi titik *landmark* wajah.
- Jika *landmark* tidak ditemukan, sistem menampilkan *error* bahwa wajah tidak terdeteksi.
- Setiap area pada *REGIONS* diproses satu per satu.
- Fungsi extract_region() digunakan untuk mengambil ROI dari setiap area.
- Jika ROI kosong, proses dihentikan dan sistem menampilkan *error* sesuai nama *region*.
- Setiap ROI yang berhasil diambil diubah ukurannya sesuai *TARGET_SIZE*.
- Hasil akhir disimpan dalam bentuk *dictionary* dengan nama area sebagai *key* dan citra ROI sebagai *value*.

Output dari tahap ini adalah tiga ROI wajah, yaitu mata-alis kiri, mata-alis kanan, dan mulut. Ketiga ROI tersebut sudah memiliki ukuran seragam sehingga dapat digunakan pada tahap ekstraksi fitur pergerakan mikro menggunakan metode POC-ABS.

3. *Spotting Micro-Expression* dan Segmentasi *Event*

Tahap *spotting* dilakukan untuk mengidentifikasi durasi kemunculan ekspresi mikro dalam rangkaian *frame video*. Proses ini bertujuan untuk menentukan tiga fase utama, yaitu *onset* sebagai awal pergerakan, *apex* sebagai puncak intensitas pergerakan, dan *offset* sebagai akhir pergerakan. Pada tahap ini, *spotting* dilakukan berdasarkan analisis sinyal *magnitudo* pergerakan antar-*frame* berbasis POC.

Alur utama proses *spotting micro-expression* terdiri atas empat bagian, yaitu konfigurasi parameter deteksi fase, *smoothing* sinyal magnitudo, pembentukan sinyal magnitudo dari rangkaian *frame*, dan deteksi *event micro-expression.*

Konfigurasi Deteksi Fase *Micro-Expression*

```
@dataclass
class ApexPhase:
    distance_threshold: int = 5
    prominence_threshold: float = 0.5
    cutoff_ratio: float = 0.3
```

Class ApexPhase digunakan untuk menyimpan parameter yang dipakai dalam proses deteksi fase *micro-expression.*

- distance_threshold digunakan untuk menentukan jarak minimal antar puncak agar *apex* yang terdeteksi tidak terlalu berdekatan.
- prominence_threshold digunakan untuk menentukan tingkat kemunculan puncak agar hanya pergerakan yang cukup menonjol yang dipilih.
- cutoff_ratio digunakan untuk membantu menentukan batas *onset* dan *offset* dari *event micro-expression.*

Dengan konfigurasi ini, proses *spotting* tidak hanya mencari puncak tertinggi, tetapi juga mempertimbangkan jarak dan kejelasan puncak pergerakan.

***Smoothing* Sinyal Magnitudo**

```
class ApexSmoother:
    @staticmethod
    def calculate_window_length(signal_length: int) -> int:
        window = int(signal_length * 0.05)
        return max(5, window if window % 2 == 1 else window + 1)

    @staticmethod
    def calculate_polyorder(window_length: int) -> int:
        return min(3, window_length - 2)

    @classmethod
    def smooth(cls, magnitudes: list[float]) -> list[float]:
        if len(magnitudes) < 5:
            return magnitudes

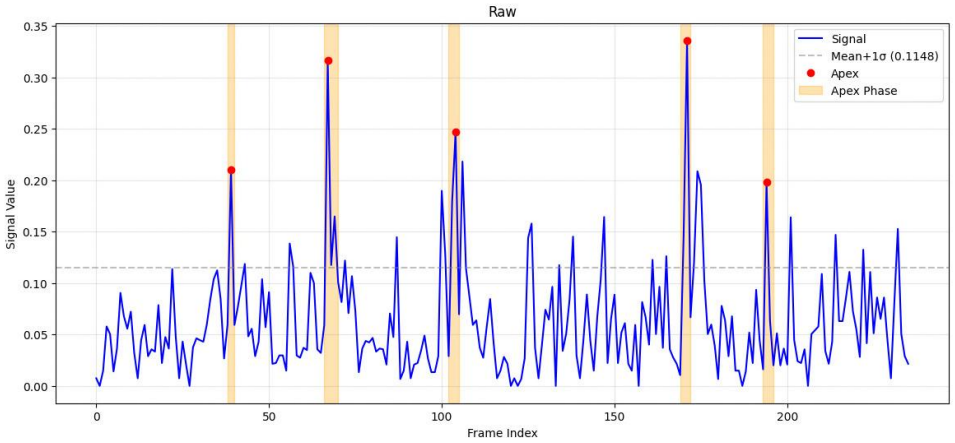
        window_length = cls.calculate_window_length(len(magnitudes))
        polyorder = cls.calculate_polyorder(window_length)

        return savgol_filter(
            magnitudes,
            window_length,
            polyorder
        ).tolist()
```

Class *ApexSmoother* digunakan untuk menghaluskan sinyal magnitudo sebelum proses deteksi *event* dilakukan. Proses *smoothing* bertujuan untuk mengurangi noise pada sinyal agar pola perubahan pergerakan wajah lebih mudah dianalisis.

- Fungsi *calculate_window_length()* digunakan untuk menentukan panjang *window* berdasarkan jumlah data magnitudo.
- Panjang *window* dihitung sekitar 5% dari panjang sinyal.
- Fungsi *calculate_polyorder()* digunakan untuk menentukan orde polinomial pada filter.
- Fungsi *smooth()* menjalankan proses *smoothing* menggunakan *Savitzky-Golay filter*.
- Jika jumlah data magnitudo kurang dari 5, sinyal tidak dihaluskan dan langsung dikembalikan.

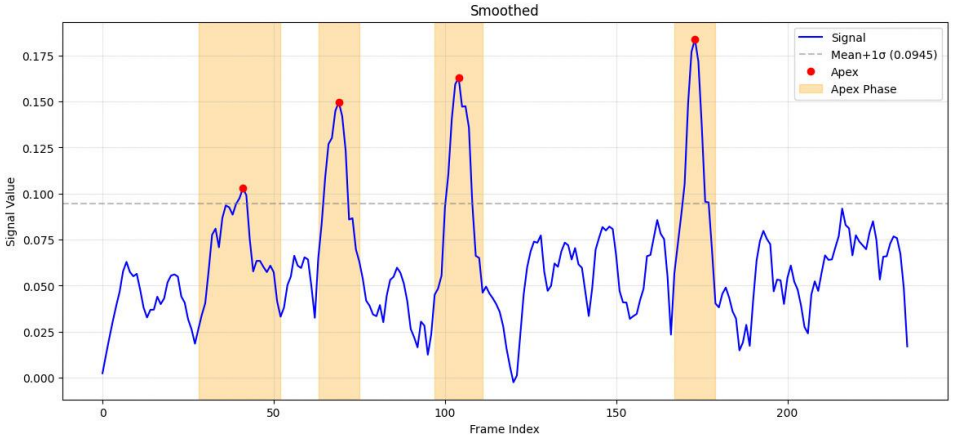
Dengan proses ini, sinyal magnitudo menjadi lebih stabil sebelum digunakan untuk mencari puncak pergerakan.



Gambar 5.2 Gambar Sinyal magnitude *raw*

Gambar 5.2 menunjukkan sinyal magnitudo *raw* yang diperoleh dari perubahan pergerakan antar *frame*. Pada sinyal tersebut terlihat beberapa lonjakan tajam dalam rentang *frame* tertentu. Lonjakan ini menunjukkan adanya perubahan pergerakan wajah, tetapi belum seluruhnya dapat dianggap sebagai *event micro-expression* karena masih terdapat kemungkinan noise akibat perubahan piksel kecil, ketidakstabilan *landmark*, pencahayaan, atau gerakan wajah yang tidak relevan.

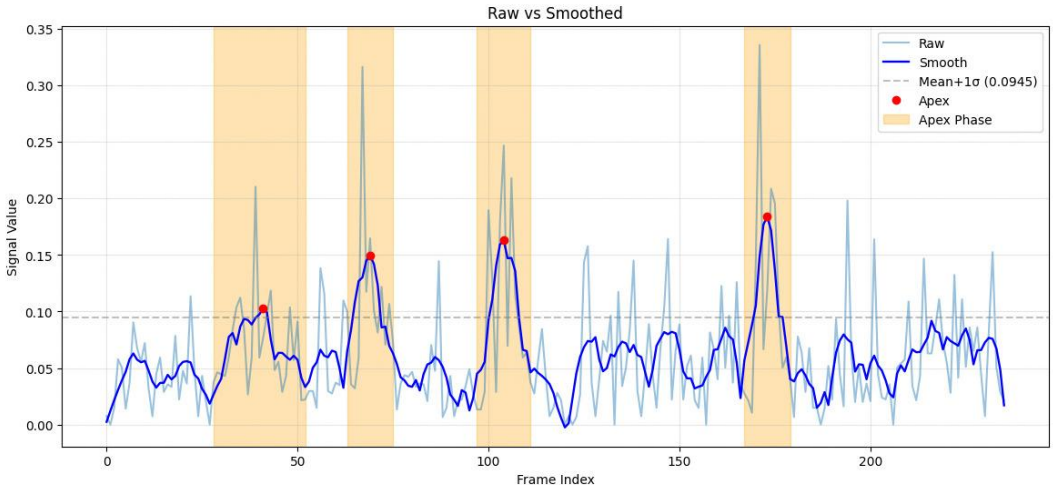
Jika sinyal *raw* langsung digunakan untuk menentukan *apex*, sistem dapat terlalu sensitif terhadap lonjakan sesaat. Hal ini dapat menyebabkan *frame* yang sebenarnya hanya mengalami perubahan kecil terdeteksi sebagai puncak pergerakan. Oleh karena itu, sinyal *raw* perlu diproses terlebih dahulu sebelum digunakan dalam tahap *spotting*.



Gambar 5. 3 Gambar Sinyal magnitude hasil *smoothing*

Gambar 5.3 menunjukkan sinyal magnitudo setelah dilakukan proses smoothing menggunakan *Savitzky-Golay filter*. Setelah *smoothing*, bentuk sinyal menjadi lebih stabil dan fluktuasi kecil pada sinyal *raw* dapat dikurangi. Pola kenaikan dan penurunan sinyal juga terlihat lebih jelas sehingga lebih mudah digunakan untuk mengamati perubahan intensitas pergerakan wajah.

Savitzky-Golay *filter* digunakan karena mampu menghaluskan sinyal tanpa menghilangkan bentuk puncak utama. Hal ini penting karena proses *spotting* membutuhkan informasi temporal berupa fase *onset*, *apex*, dan *offset*. Dengan sinyal yang lebih stabil, proses deteksi *peak* dapat lebih fokus pada puncak pergerakan yang dominan.



Gambar 5. 4 Gambar Perbandingan Sinyal *Raw* dan *Smoothed*

Gambar 5.4 menunjukkan perbandingan antara sinyal *raw* dan sinyal hasil *smoothing*. Pada sinyal *raw*, terdapat banyak lonjakan tajam yang muncul dalam durasi singkat. Setelah *smoothing* dilakukan, lonjakan kecil menjadi lebih terkendali, sedangkan pola puncak utama tetap terlihat.

Berdasarkan perbandingan tersebut, *smoothing* digunakan untuk membantu sistem membedakan antara *noise* dan kandidat *event micro-expression*. Hasil *smoothing* kemudian digunakan sebagai dasar dalam proses deteksi *peak* untuk menentukan *apex*, serta penentuan rentang *onset* dan *offset* pada setiap *event*.

Pembentukan Sinyal Magnitudo

```
def build_magnitude_signal(
    frames: list[np.ndarray],
    detector,
    predictor,
    config
) -> list[float]:
    if len(frames) < 3:
        return []

    grays = [
        cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        for frame in frames
    ]

    magnitudes = []

    prev_gray = grays[0]
    prev_lm = detect_landmarks(
        prev_gray,
        detector,
        predictor
    )

    for curr_gray in grays[1:]:
        curr_lm = detect_landmarks(
            curr_gray,
            detector,
            predictor
        )

        if prev_lm is not None and curr_lm is not None:
            mag = compute_frame_magnitude(
                prev_gray,
                prev_lm,
                curr_gray,
                curr_lm,
                config
            )
        else:
            mag = 0.0

        magnitudes.append(float(mag))
        prev_gray = curr_gray
        prev_lm = curr_lm
```

```
        return magnitudes
```

Fungsi *build_magnitude_signal()* digunakan untuk membentuk sinyal magnitudo dari rangkaian *frame video*. Sinyal ini menunjukkan besar perubahan gerakan wajah dari satu *frame* ke *frame* berikutnya.

- *Input* fungsi berupa daftar *frame video*.
- Jika jumlah *frame* kurang dari 3, fungsi mengembalikan list kosong.
- Setiap *frame* dikonversi ke *grayscale* agar proses perhitungan lebih sederhana.
- *Landmark* wajah pada *frame* sebelumnya dan *frame* saat ini dideteksi terlebih dahulu.
- Jika *landmark* pada kedua *frame* berhasil ditemukan, magnitudo dihitung menggunakan compute*_frame_magnitude().*
- Jika *landmark* tidak ditemukan pada salah satu *frame*, nilai magnitudo diberi nilai 0.0.
- Nilai magnitudo disimpan secara berurutan ke dalam list *magnitudes*.

Hasil dari fungsi ini adalah sinyal magnitudo yang merepresentasikan perubahan pergerakan wajah sepanjang video.

Deteksi *Event Micro-Expression*

```
    def detect_events(
        magnitudes: list[float],
        config
    ) -> tuple[list[float], list[dict], dict]:
        smoothed = ApexSmoother.smooth(magnitudes)
        signal_arr = np.asarray(smoothed, dtype=float)

        height_threshold = float(
            signal_arr.mean() + signal_arr.std()
        )

        detector = ApexPhase(
            distance_threshold=config.distance_threshold,
            prominence_threshold=config.prominence_threshold,
            cutoff_ratio=config.cutoff_ratio,
        )

        apex_indices = detector.find_top_k_apex(
            smoothed,
```

```
            k=config.top_k_apex,
            height=height_threshold
        )

        phases = detector.find_phase(
            smoothed,
            apex_indices
        ) if apex_indices else {}

        events = []

        for event_no, apex_idx in enumerate(apex_indices, start=1):
            phase = phases[apex_idx]
            duration = int(phase["end"] - phase["start"])

            if duration < config.min_phase_duration:
                continue

            events.append({
                "event_no": event_no,
                "onset_signal": int(phase["start"]),
                "apex_signal": int(apex_idx),
                "offset_signal": int(phase["end"]),
                "duration": duration,
            })

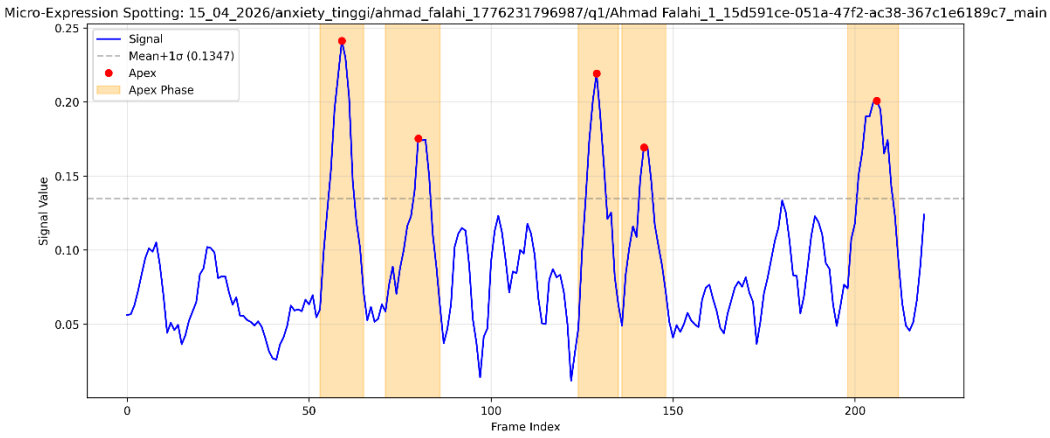
        meta = {
            "window_length": ApexSmoother.calculate_window_length(
                len(magnitudes)
            ),
            "polyorder": ApexSmoother.calculate_polyorder(
                ApexSmoother.calculate_window_length(
                    len(magnitudes)
                )
            ),
            "height_threshold": height_threshold,
        }

        return smoothed, events, meta
```

Fungsi *detect_event*s() digunakan untuk mendeteksi *event micro-expression* berdasarkan sinyal magnitudo yang telah dibentuk.

- Sinyal magnitudo terlebih dahulu dihaluskan menggunakan *ApexSmoother.smooth()*.

- Nilai ambang batas dihitung menggunakan rumus *mean* + *standard deviation*.
- Nilai *mean* berasal dari rata-rata seluruh sinyal magnitudo hasil *smoothing*.
- Nilai *mean* digunakan sebagai acuan dasar untuk merepresentasikan rata-rata intensitas pergerakan wajah pada satu video.
- Nilai di atas ambang batas dianggap sebagai kandidat pergerakan yang signifikan.
- Sistem mencari titik *apex* sebagai puncak pergerakan dari sinyal magnitudo.
- Setelah *apex* ditemukan, sistem menentukan fase *onset* dan *offset* di sekitar *apex* tersebut.
- *Event* dengan durasi kurang dari batas minimal akan diabaikan agar hasil tidak terlalu dipengaruhi *noise*.
- *Metadata* seperti *window_length*, *polyorder*, dan *height_threshold* disimpan sebagai informasi pendukung.



Gambar 5.5 Grafik Hasil *Spotting*

Grafik di atas menunjukkan hasil *spotting micro-expression* dari rangkaian *frame* video. Sumbu horizontal menunjukkan urutan *frame*, sedangkan sumbu vertikal menunjukkan besar magnitudo pergerakan wajah antar *frame*.

Garis putus-putus abu-abu menunjukkan ambang batas deteksi yang dihitung dari *mean* + 1σ, yaitu rata-rata nilai magnitudo hasil *smoothing*

ditambah satu standar deviasi. Pada grafik ini, nilai ambang batas yang diperoleh adalah 0,1347. Nilai magnitudo yang melewati garis tersebut dianggap sebagai pergerakan yang signifikan.

Titik merah menunjukkan *frame apex*, yaitu puncak intensitas dari *event micro-expression* yang terdeteksi. Kotak oranye menunjukkan rentang satu *event*, mulai dari *onset, apex,* hingga *offset*. Berdasarkan grafik, sistem berhasil mendeteksi beberapa *event micro-expression* secara otomatis dari sinyal pergerakan wajah.

4. Implementasi Ekstraksi Fitur POC-ABS

Tahap ekstraksi fitur dilakukan menggunakan metode *Phase Only Correlation with All Block Search* (POC-ABS). Metode ini digunakan untuk mengidentifikasi pergeseran piksel mikroskopis antar-*frame* pada area wajah. Proses ekstraksi dilakukan dengan membagi setiap ROI menjadi blok berukuran 7×7 piksel, menghitung korelasi fase pada setiap pasangan blok, lalu mengubah hasil korelasi menjadi vektor gerak.

Jumlah blok pada setiap ROI diperoleh dari pembagian ukuran ROI terhadap block size 7×7 piksel. Area mulut yang berukuran 70×35 piksel menghasilkan 10×5 blok, sehingga diperoleh 50 blok. Area mata-alis kiri dan mata-alis kanan yang masing-masing berukuran 48×32 piksel menghasilkan 6×4 blok, sehingga masing-masing ROI menghasilkan 24 blok. Dengan demikian, total blok dari ketiga ROI adalah 98 blok. Setiap blok menghasilkan empat fitur, yaitu X, Y, *theta*, dan magnitude, sehingga total fitur yang diperoleh adalah 98×4 = 392 fitur.

Alur utama ekstraksi fitur POC-ABS terdiri atas lima bagian, yaitu inisialisasi pasangan citra ROI, pembentukan *Hanning window*, perhitungan korelasi fase, pemrosesan seluruh blok ROI, serta ekstraksi vektor gerak dan arah pergerakan.

Inisialisasi Pasangan Citra ROI

```
class POC:
    def __init__(
        self,
        img_block_ref: np.ndarray,
        img_block_cur: np.ndarray,
        block_size: int = 7
```

```
        ):
            self.img_block_ref = img_block_ref.astype(np.float64)
            self.img_block_cur = img_block_cur.astype(np.float64)
            self.block_size = block_size
```

Class POC digunakan untuk memproses dua citra ROI yang akan dibandingkan.

- img_block_ref digunakan sebagai citra ROI dari *frame* referensi atau *frame* sebelumnya.
- img_block_cur digunakan sebagai citra ROI dari *frame* saat ini.
- Kedua citra dikonversi ke tipe float64 agar proses perhitungan numerik lebih stabil.
- Parameter block_size digunakan untuk menentukan ukuran blok, yaitu 7×7 piksel.

Dengan inisialisasi ini, sistem menyiapkan dua citra ROI yang akan dibandingkan untuk mencari pergeseran mikro antar *frame*.

Pembentukan *Hanning Window*

```
    def hann_window(self) -> np.ndarray:
        window_1d = np.hanning(self.block_size)
        window_2d = np.outer(window_1d, window_1d)

        return window_2d
```

Fungsi hann_window() digunakan untuk membuat *Hanning window* dua dimensi sebelum proses korelasi fase dilakukan.

- *Hanning window* dibentuk dari *window* satu dimensi menggunakan np.hanning().
- *Window* dua dimensi dibuat menggunakan np.outer().
- *Window* ini diterapkan pada blok citra sebelum proses FFT.
- Tujuannya adalah mengurangi efek tepi pada blok citra agar hasil korelasi lebih stabil.

Dengan proses ini, setiap blok citra dipersiapkan agar perhitungan POC tidak terlalu dipengaruhi oleh batas tepi blok.

Perhitungan Korelasi Fase

```
    def calc_poc(
        self,
```

```
            block_ref: np.ndarray,
            block_cur: np.ndarray,
            window: np.ndarray,
            mb_x: int,
            mb_y: int
        ) -> np.ndarray:
            fft_ref = fft2(block_ref * window, (mb_x, mb_y))
            fft_cur = fft2(block_cur * window, (mb_x, mb_y))

            R1 = fft_ref * np.conj(fft_cur)
            R2 = np.abs(R1)

            R2[R2 == 0] = 1e-31

            R = R1 / R2

            r = ifft2(R)
            r = np.abs(r)
            r = fftshift(r)

            return r
```

Fungsi calc_poc() digunakan untuk menghitung nilai korelasi fase pada satu pasangan blok citra.

- Blok referensi dan blok saat ini dikalikan terlebih dahulu dengan *Hanning window*.
- Kedua blok kemudian diubah ke domain frekuensi menggunakan fft2().
- Sistem menghitung *cross-power spectrum* dari hasil FFT kedua blok.
- Nilai amplitudo dinormalisasi agar informasi yang dipertahankan hanya informasi fase.
- Nilai 1e-31 digunakan untuk menghindari pembagian dengan nol.
- Hasil normalisasi dikembalikan ke domain spasial menggunakan ifft2().
- Fungsi fftshift() digunakan untuk memusatkan hasil korelasi.

Hasil dari fungsi ini adalah matriks korelasi POC yang menunjukkan posisi puncak korelasi pada pasangan blok.

Pemrosesan Seluruh Blok ROI

```
        def get_poc(self) -> list:
            window = self.hann_window()

            img0 = self.img_block_ref
```

```
            img1 = self.img_block_cur

            rows, cols = img0.shape

            cols_y = int(np.floor(cols / self.block_size))
            rows_x = int(np.floor(rows / self.block_size))

            mod_y = cols % self.block_size
            mod_x = rows % self.block_size

            BlocksCurr = np.empty((cols_y, rows_x), dtype=object)
            BlocksRef = np.empty((cols_y, rows_x), dtype=object)

            poc = np.zeros(
                (self.block_size, self.block_size, cols_y * rows_x)
            )

            coor_awal = np.zeros((cols_y * rows_x, 2))
            rect = np.zeros((cols_y * rows_x, 4))

            nm = 0

            for y in range(0, cols - mod_y, self.block_size):
                n_x = 0
                n_xx = 1

                for x in range(0, rows - mod_x, self.block_size):
                    BlocksCurr[n_y, n_x] = img0[
                        y:y + self.block_size,
                        x:x + self.block_size
                    ]

                    BlocksRef[n_y, n_x] = img1[
                        y:y + self.block_size,
                        x:x + self.block_size
                    ]

                    rect[nm, :] = [
                        x,
                        y,
                        self.block_size,
                        self.block_size
                    ]

                    block_ref = BlocksRef[n_y, n_x]
                    block_curr = BlocksCurr[n_y, n_x]
```

```
                    r = self.calc_poc(
                        block_ref,
                        block_curr,
                        window,
                        self.block_size,
                        self.block_size
                    )

                    poc[:, :, nm] = r

                    coor_awal[nm, 0] = n_xx * self.block_size
                    coor_awal[nm, 1] = n_yy * self.block_size

                    n_x += 1
                    n_xx += 1
                    nm += 1

                n_y += 1
                n_yy += 1

            return [poc, coor_awal, rect]
```

Fungsi get_poc() digunakan untuk memproses seluruh blok pada ROI.

- ROI dibagi menjadi beberapa blok kecil berdasarkan nilai block_size.
- Area mulut berukuran 70×35 menghasilkan 50 blok.
- Area mata-alis kiri dan kanan berukuran 48×32 menghasilkan sekitar 24 blok pada masing-masing ROI.
- Setiap blok dari *frame* referensi dipasangkan dengan blok pada posisi yang sama dari *frame* saat ini.
- Fungsi calc_poc() dipanggil untuk menghitung korelasi fase pada setiap pasangan blok.
- Nilai korelasi setiap blok disimpan dalam variabel poc.
- Koordinat awal blok disimpan dalam coor_awal.
- Informasi posisi dan ukuran blok disimpan dalam rect.

Output dari fungsi ini berupa nilai POC, koordinat awal blok, dan bounding box blok. Data tersebut menjadi dasar untuk menghitung arah dan besar pergerakan mikro.

Ekstraksi Vektor Gerak

```
        class Vektor:
            def __init__(self, poc_output: list, block_size: int):
                self.poc = poc_output[0]
                self.coor_awal = poc_output[1]
                self.block_size = block_size

            def get_vektor(self) -> np.ndarray:
                mb = self.block_size
                nil_tengah = int(np.median(np.arange(mb)))
                med = nil_tengah + 1

                rep_x = np.arange(-nil_tengah, med)
                rep_y = np.arange(nil_tengah, -med, -1)

                cur_x = np.arange(mb)
                cur_y = np.arange(mb)

                output = np.zeros((len(self.coor_awal), 6))
                val_poc = self.poc

                for i in range(val_poc.shape[2]):
                    r = val_poc[:, :, i]

                    val_max = np.max(r)
                    temp_y, temp_x = np.where(r == val_max)

                    if len(temp_y) > 1 or len(temp_x) > 1:
                        temp_x = [nil_tengah]
                        temp_y = [nil_tengah]

                    tx = temp_x[0]
                    ty = temp_y[0]

                    if tx != nil_tengah or ty != nil_tengah:
                        cor_x = self.coor_awal[i][0]
                        cor_y = self.coor_awal[i][1]

                        t_x = cor_x - med
                        t_y = cor_y - med

                        o_x = rep_x[cur_x[tx]]
                        o_y = rep_y[cur_y[ty]]

                        m_x = cor_x - (mb - tx)
                        m_y = cor_y - (mb - ty)
```

```
                    p1 = np.array([t_x, t_y])
                    p2 = np.array([m_x, m_y])
                    V = p2 - p1

                    output[i, 0] = p1[0]
                    output[i, 1] = p1[1]
                    output[i, 2] = V[0]
                    output[i, 3] = V[1]
                    output[i, 4] = o_x
                    output[i, 5] = o_y

            return output
```

Class Vektor digunakan untuk mengambil informasi pergerakan dari hasil POC.

- *Input* class berupa *output* POC dan ukuran blok yang digunakan.
- Nilai poc berisi matriks korelasi dari setiap blok.
- Nilai coor_awal berisi koordinat awal setiap blok.
- Fungsi get_vektor() mencari titik puncak korelasi pada setiap blok.
- Titik puncak korelasi menunjukkan posisi pergeseran paling dominan.
- Jika puncak berada di tengah blok, maka blok dianggap tidak mengalami pergerakan.
- Jika puncak bergeser dari tengah, sistem menghitung perubahan posisi sebagai vektor gerak.

Output dari fungsi ini berupa posisi awal blok, nilai perpindahan pada sumbu X dan Y, serta arah relatif pergerakan.

Klasifikasi Arah Pergerakan

```
class Quadran:
    def __init__(self, coor_data: np.ndarray):
        self.data_x = coor_data[:, 4]
        self.data_y = coor_data[:, 5]

    def get_quadran(self) -> np.ndarray:
        quadran_data = np.empty(
            (len(self.data_x), 6),
            dtype=object
        )

        for i in range(len(self.data_x)):
            X = int(self.data_x[i])
```

```
                Y = int(self.data_y[i])

                theta_rad = np.arctan2(Y, X)
                theta_deg = np.degrees(theta_rad)

                if Y < 0:
                    theta_deg += 360

                magnitude = np.sqrt(X**2 + Y**2)

                if X == 0 and Y == 0:
                    label = "No Quadran"
                elif 0 <= theta_deg < 90:
                    label = "Q1"
                elif 90 <= theta_deg < 180:
                    label = "Q2"
                elif 180 <= theta_deg < 270:
                    label = "Q3"
                elif 270 <= theta_deg < 360:
                    label = "Q4"
                else:
                    label = "No Quadran"

                quadran_data[i, :] = [
                    str(i),
                    X,
                    Y,
                    round(theta_deg, 2),
                    round(magnitude, 4),
                    label
                ]

            return quadran_data
```

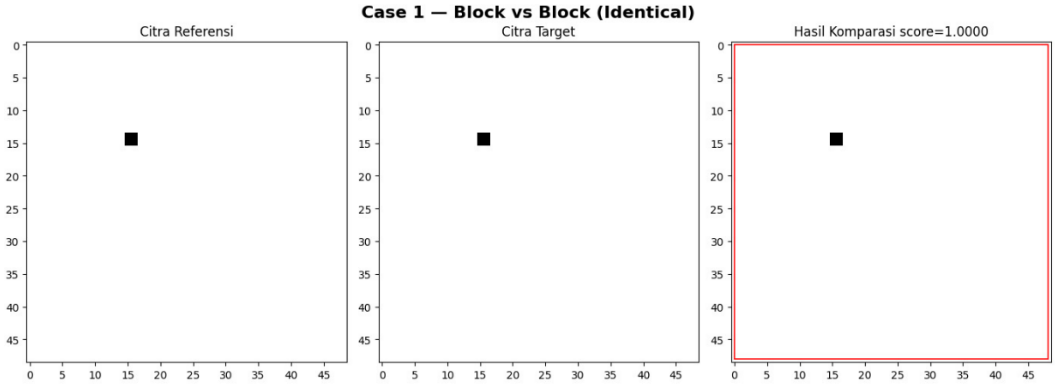
Class Quadran digunakan untuk mengubah vektor gerak menjadi fitur arah dan besar pergerakan.

- Class ini mengambil nilai arah X dan Y dari hasil vektor gerak.
- Fungsi get_quadran() menghitung nilai *theta*, magnitude, dan label kuadran.
- Nilai *theta* menunjukkan sudut arah pergerakan dalam satuan derajat.
- Nilai magnitude menunjukkan besar atau panjang pergerakan pada blok.
- Jika X dan Y bernilai 0, blok diberi label *No Quadran*.
- Jika terdapat pergerakan, arah diklasifikasikan ke dalam Q1, Q2, Q3, atau Q4 berdasarkan sudutnya.

- Hasil akhir disusun dalam bentuk data berisi block_id, X, Y, *theta*, magnitude, dan label kuadran.

Output dari tahap ini adalah fitur pergerakan mikro pada setiap blok wajah. Fitur tersebut merepresentasikan arah dan besar pergeseran yang kemudian digunakan sebagai data numerik untuk proses analisis dan klasifikasi berikutnya.

***Case* 1: Block vs Block (Identical)**

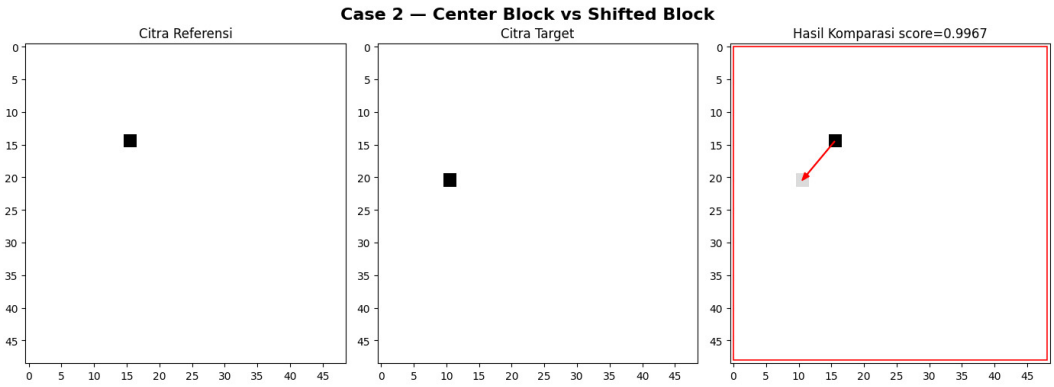


Gambar 5.6 *Case* 1 validasi algoritma POC-ABS

Tujuan gambar ini adalah validasi baseline metode POC-ABS pada kondisi paling sederhana.

- **Citra Referensi** menampilkan satu blok hitam pada posisi tertentu (sekitar koordinat [15, 15]).
- **Citra Target** menampilkan blok hitam yang persis identik, bentuk, dan lokasi pikselnya sama persis dengan citra referensi.
- **Hasil Komparasi** menunjukkan skor 1,0000 (nilai maksimum). Tidak ada panah atau tanda pergeseran karena sistem tidak mendeteksi perbedaan antar-blok sama sekali.
- **Maknanya:** Metode POC-ABS mampu mengenali kecocokan sempurna. Ini menjadi patokan bahwa skor 1,0000 berarti tidak ada gerakan atau deformasi pada blok yang diuji.

Case 2: Center Block vs Shifted Block

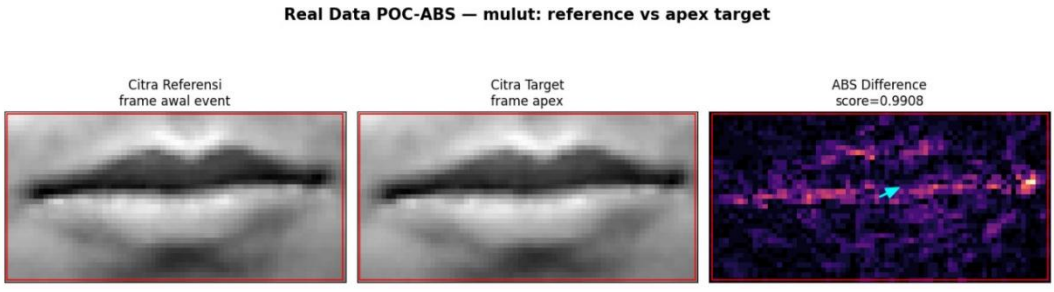


Gambar 5.7 *Case* 2 validasi algoritma POC-ABS

Tujuan gambar ini adalah demonstrasi deteksi pergeseran mikro pada level blok.

- Citra Referensi memiliki blok hitam di posisi tengah (sekitar [15, 15]).
- Citra Target memiliki blok hitam yang sama, tetapi telah bergeser sedikit ke arah kiri dan ke bawah (sekitar [20, 10]).
- Hasil Komparasi menampilkan dua hal penting:
  - Skor 0,9967 — masih sangat tinggi, menandakan blok tersebut sangat mirip (hanya berbeda posisi, bukan bentuk).
  - Panah merah — merepresentasikan vektor gerak (*motion vector*) hasil pencarian *All Block Search*. Panah ini menunjukkan arah dan magnitudo pergeseran piksel dari posisi referensi (titik abu-abu) ke posisi target (titik hitam).
- Maknanya: POC-ABS tidak hanya menghitung kemiripan, tetapi juga mampu mengestimasi arah pergeseran spasial antar-blok 7×7.

Real Data POC-ABS Mulut: Reference vs Apex Target



Gambar 5.8 Visualisasi Data *Real* POC-ABS

Tujuan gambar ini adalah penerapan pada data wajah nyata untuk ekstraksi fitur gerak.

- **Citra Referensi (*frame* awal *event*):** Menampilkan area mulut (ROI) pada kondisi netral atau *onset* bibir dalam posisi awal sebelum ekspresi mencapai puncak.
- **Citra Target (*frame* apex):** Menampilkan area mulut yang sama pada *frame apex*, yaitu saat ekspresi mencapai intensitas maksimum. Terlihat perubahan tipis pada bentuk bibir (misalnya, sedikit membuka atau meregang).
- **ABS Difference (score=0,9908):**
    - Menampilkan peta perbedaan absolut antar-blok dalam format *heatmap* (warna ungu hingga oranye).
    - **Area oranye/merah** menandakan lokasi blok-blok 7×7 yang mengalami perubahan intensitas piksel akibat gerakan wajah.
    - **Panah biru** menunjukkan arah dominan deformasi pada area bibir, mengindikasikan vektor gerak terkuat yang terdeteksi oleh POC-ABS.
    - **Skor 0,9908** menunjukkan bahwa secara keseluruhan masih ada kemiripan tinggi (karena hanya perubahan mikro), namun perbedaan yang terekstrak cukup signifikan untuk dijadikan fitur diskriminatif.
- **Maknanya:** Pada implementasi sesungguhnya, setiap komponen wajah (mulut 70×35, mata-kiri 48×32, mata-kanan 48×32) dibagi menjadi ratusan blok 7×7. Gambar ini membuktikan bahwa POC-ABS dapat menangkap pergeseran piksel mikroskopis pada data nyata dan merubahnya menjadi vektor fitur gerak untuk analisis *micro-expression* lebih lanjut.

5. Pembentukan Data Tabular dan Ekspor Fitur

Setelah *event micro-expression* terdeteksi, sistem menyimpan potongan *frame* berdasarkan rentang *onset* sampai *offset*. *Frame apex* diberi penanda khusus agar posisi puncak ekspresi dapat dikenali kembali. *Metadata* dari setiap *event* kemudian dikumpulkan dan digunakan sebagai dasar pembentukan *dataset* fitur.

Alur utama tahap ini terdiri atas tiga bagian, yaitu penyimpanan *frame event*, pemrosesan satu sampel video, dan ekspor *dataset* fitur.

Menyimpan *Frame Event*

```
def save_event_clip_frames(frames, sample_output_dir, events):
    sample_output_dir.mkdir(parents=True, exist_ok=True)
    rows = []

    for event in events:
        onset = event["onset_signal"] + 1
        apex = event["apex_signal"] + 1
        offset = event["offset_signal"] + 1

        clip_dir = sample_output_dir / f"event_{onset:05d}-{offset:05d}"
        clip_dir.mkdir(parents=True, exist_ok=True)

        for idx, frame_idx in enumerate(range(onset, offset + 1)):
            suffix = "-apex.png" if frame_idx == apex else ".jpg"
            out_path = clip_dir / f"frame_{idx:05d}{suffix}"
            cv2.imwrite(str(out_path), frames[frame_idx])

        rows.append({
            "event_no": event["event_no"],
            "onset_frame": onset,
            "apex_frame": apex,
            "offset_frame": offset,
            "duration": event["duration"],
            "clip_dir": str(clip_dir),
        })

    return rows
```

Fungsi *save_event*_clip_*frame*s() digunakan untuk menyimpan *frame* dari setiap *event* yang terdeteksi.

- *Event* disimpan berdasarkan rentang *onset* sampai *offset*.
- *Frame apex* diberi nama khusus dengan label *apex*.
- *Metadata* seperti *onset*, *apex*, *offset*, durasi, dan lokasi clip disimpan.

Output dari fungsi ini adalah daftar *metadata event* yang berhasil disimpan.

Pemrosesan Satu Sampel Video

```
def process_sample(sample, config, detector, predictor):
    frames = load_video_frames(sample.video_path)

    if len(frames) < 3:
```

```
            return {"status": "skip", "reason": "too few frames"}

        magnitudes = build_magnitude_signal(
            frames,
            detector,
            predictor,
            config
        )

        if len(magnitudes) < 5:
            return {"status": "skip", "reason": "too few magnitudes"}

        smoothed, events, meta = detect_events(magnitudes, config)

        output_dir = config.output_dataset_root / sample.relative_video_key
        rows = save_event_clip_frames(frames, output_dir, events)

        return {
            "status": "processed",
            "relative_key": str(sample.relative_video_key),
            "rows": rows,
        }
```

Fungsi process_sample() digunakan untuk menjalankan proses utama pada satu video.

- Video dibaca menjadi rangkaian *frame*.
- Sistem membentuk sinyal magnitudo dari *frame* video.
- *Event micro-expression* dideteksi dari sinyal magnitudo.
- *Frame event* disimpan dan *metadata* hasil proses dikumpulkan.
- Sampel dilewati jika jumlah *frame* atau magnitudo tidak mencukupi.

Output fungsi ini berupa status proses dan *metadata event* dari video.

Ekspor *Dataset* Fitur

```
    def export_dataset_slice(spec, config):
        name = spec["name"]
        dataset_root = spec["dataset_root"]

        output_path = config.feature_root / name / "poc_abs_test.xlsx"
        errors_path = config.feature_root / name / "errors.xlsx"

        clips, parse_errors = discover_event_clips(dataset_root, name)

        rows = []
        errors = list(parse_errors)
```

```
    for clip in clips:
        result = process_event_clip(asdict(clip))

        if result["ok"]:
            rows.extend(result["rows"])
        else:
            errors.append((result["sample_id"], result["error"]))

    df = pd.DataFrame(rows)
    df.to_excel(output_path, index=False)

    error_df = pd.DataFrame(errors, columns=["sample_id", "error"])
    error_df.to_excel(errors_path, index=False)

    return df, error_df
```

Fungsi export_*dataset*_slice() digunakan untuk mengekspor hasil fitur ke dalam *file* Excel.

- Sistem mengambil daftar *event clip* dari direktori *dataset*.
- Setiap *clip* diproses menjadi baris data fitur.
- Data hasil ekstraksi disimpan menjadi poc_abs_test.xlsx.
- *Error* selama proses ekstraksi disimpan ke *error*s.xlsx.

Output dari tahap ini adalah *dataset* fitur dalam format Excel yang digunakan sebagai masukan pada tahap klasifikasi model TabR.

5.1.2 Implementasi Model TabR

1. Persiapan *Dataset* dan Konfigurasi

Implementasi model TabR mengikuti *pipeline* standar dari repositori *official* tabular-dl-tabr. *Dataset* hasil ekstraksi fitur POC-ABS diekspor ke format yang kompatibel dengan *framework* TabR melalui fungsi export_official_*dataset*.

```
def export_official_dataset(df, feature_cols, cfg):
    out_dir = DATA_ROOT / cfg["dataset_name"]

    df_tr, df_val, df_test = split_by_questions(
        df,
        cfg["train_questions"],
        cfg["val_questions"],
        cfg["test_questions"]
    )
```

```
            imputer = SimpleImputer(strategy="mean")
            scaler = StandardScaler()

            X_train                                                    =
scaler.fit_transform(imputer.fit_transform(df_tr[feature_cols]))
            X_val = scaler.transform(imputer.transform(df_val[feature_cols]))
            X_test = scaler.transform(imputer.transform(df_test[feature_cols]))

            y_train = df_tr["target"].to_numpy()
            y_val = df_val["target"].to_numpy()
            y_test = df_test["target"].to_numpy()

            out_dir.mkdir(parents=True, exist_ok=True)

            np.save(out_dir / "X_num_train.npy", X_train.astype(np.float32))
            np.save(out_dir / "X_num_val.npy", X_val.astype(np.float32))
            np.save(out_dir / "X_num_test.npy", X_test.astype(np.float32))

            np.save(out_dir / "Y_train.npy", y_train.astype(np.int64))
            np.save(out_dir / "Y_val.npy", y_val.astype(np.int64))
            np.save(out_dir / "Y_test.npy", y_test.astype(np.int64))

            (out_dir / "info.json").write_text(json.dumps({
                "task_type": "binclass",
                "name": cfg["dataset_name"]
            }, indent=2))

            return out_dir
```

Fungsi export_official_*dataset*() digunakan untuk menyiapkan *dataset* agar sesuai dengan struktur *input* model TabR.

- *Dataset* dibagi menjadi *train*, *validation*, dan *test* menggunakan split_by_*questions*().
- Kolom fitur diambil dari feature_cols, sedangkan label diambil dari kolom target.
- Nilai kosong pada fitur ditangani menggunakan SimpleImputer.
- Fitur dinormalisasi menggunakan Standard*Scale*r.
- Data fitur disimpan dalam format .npy sebagai X_num_*train*, X_num_val, dan X_num_test.
- Label disimpan dalam format .npy sebagai Y_*train*, Y_val, dan Y_test.

- *Metadata dataset* disimpan dalam info.json, sedangkan daftar fitur disimpan dalam feature_cols.json.
- Data asli setiap split juga disimpan dalam format .csv untuk kebutuhan analisis hasil.

Output dari tahap ini adalah *folder dataset* TabR yang berisi fitur numerik, label, *metadata*, daftar fitur, dan *file* split data. *Dataset* ini kemudian digunakan pada proses pelatihan dan evaluasi model TabR.

2. Konfigurasi *Dataset Question-Wise Split*

Skema *Question-Wise Split* ini memastikan evaluasi yang robust dengan memisahkan data berdasarkan pertanyaan kuesioner, bukan secara random, sehingga menguji generalisasi model terhadap subjek dan kondisi yang belum pernah dilihat.

```
FOLDS = [
    {
        'dataset_name': 'convat_apex_anxiety_qwalk_q12_q3_q4',
        'train_questions': (1, 2),      # Q1, Q2 untuk training
        'val_questions': (3,),          # Q3 untuk validasi
        'test_questions': (4,),         # Q4 untuk testing
    },
    {
        'dataset_name': 'convat_apex_anxiety_qwalk_q123_q4_q5',
        'train_questions': (1, 2, 3),   # Q1-Q3 untuk training
        'val_questions': (4,),          # Q4 untuk validasi
        'test_questions': (5,),         # Q5 untuk testing
    },
]
```

Detail konfigurasi fold adalah sebagai berikut:

- Variabel FOLDS berisi daftar skenario pembagian *dataset*.
- Setiap fold memiliki *dataset*_name sebagai nama *folder dataset* yang akan dibuat.
- Pada fold pertama, Q1 dan Q2 digunakan sebagai data *training*.
- Q3 digunakan sebagai data *validation*.
- Q4 digunakan sebagai data *testing*.
- Pada fold kedua, Q1 sampai Q3 digunakan sebagai data *training*.
- Q4 digunakan sebagai data *validation*.
- Q5 digunakan sebagai data *testing*.

- Skema ini digunakan untuk melihat performa model pada data pertanyaan yang berbeda dari data *training*.

Output dari tahap ini adalah konfigurasi pembagian *dataset* yang digunakan oleh fungsi ekspor *dataset* TabR.

3. Konfigurasi Hyperparameter dan Tuning

TabR memerlukan konfigurasi TOML untuk tuning hyperparameter. Fungsi prepare_tuning_config menghasilkan *file* konfigurasi yang mengacu pada *dataset* yang telah diekspor.

```
        def prepare_tuning_config(dataset_name: str, exp_name: str,
                                  overwrite_existing:   bool   =   True)   ->
tuple[dict, Path]:
            """
            Menyiapkan konfigurasi TOML untuk hyperparameter tuning TabR.
            Menggunakan template base tuning dari repositori official.
            """
            # Load template konfigurasi TabR
            config = lib.load_config(BASE_TUNING_TOML)
            config = copy.deepcopy(config)

            # Set path dataset mengikuti konvensi TabR (:data/<dataset_name>)
            config['space']['data']['path'] = f':data/{dataset_name}'

            # Direktori eksperimen
            exp_dir = EXP_ROOT / exp_name
            tuning_toml = exp_dir / '0-tuning.toml'

            if tuning_toml.exists() and not overwrite_existing:
                raise   FileExistsError(f'Experiment   path   already   exists:
{exp_dir}')

            exp_dir.mkdir(parents=True, exist_ok=True)
            lib.dump_config(config, tuning_toml)

            return config, tuning_toml
```

Parameter	Nilai	Keterangan
n_external	9	Jumlah context eksternal untuk retrieval
batch_size	128	Ukuran batch training
max_epoch	Early stopping, best epoch 31	Training tidak memakai batas 50; berhenti berdasarkan performa validasi
lr	6.44e-4	Learning rate terbaik
weight_decay	2.85e-5	Regularisasi L2 / AdamW weight decay
dropout	context=0.330, dropout0=0.399, dropout1=0.123	Dropout hasil tuning TABR
n_blocks	encoder=0, predictor=2	Jumlah block model TABR
d_main	256	Dimensi hidden layer utama

Gambar 5.9 Parameter yang digunakan

4. Inisialisasi dan Arsitektur Model TabR

```
        model = Model(
            n_num_features=dataset.n_num_features,
            n_bin_features=dataset.n_bin_features,
            cat_cardinalities=dataset.cat_cardinalities(),
            n_classes=dataset.n_classes(),
            **C.model,
        )
```

Potongan kode tersebut menunjukkan proses inisialisasi model TabR berdasarkan struktur *dataset* yang telah disiapkan. Parameter n_num_features digunakan untuk menentukan jumlah fitur numerik hasil ekstraksi POC-ABS, sedangkan n_classes digunakan untuk menentukan jumlah kelas *output* pada proses klasifikasi. Pada penelitian ini, data yang digunakan berupa fitur numerik, sehingga fitur utama yang diproses model berasal dari nilai X, Y, *theta*, dan magnitude.

Pembentukan Dimensi *Input* Model

```
d_in = (
    n_num_features
    * (1 if num_embeddings is None else num_embeddings["d_embedding"])
    + n_bin_features
    + sum(cat_cardinalities)
)
```

Potongan kode tersebut digunakan untuk menghitung jumlah fitur *input* yang masuk ke model. Nilai d_in dibentuk dari jumlah fitur numerik, fitur biner, dan fitur kategorikal. Karena data pada penelitian ini berupa fitur numerik hasil POC-ABS, maka bagian utama yang digunakan adalah n_num_features.

Proyeksi Fitur ke Dimensi Utama Model

```
self.linear = nn.Linear(d_in, d_main)
```

Layer self.linear digunakan untuk mengubah ukuran *input* fitur dari d_in menjadi d_main. Tahap ini diperlukan agar seluruh fitur numerik hasil POC-ABS diproyeksikan ke ukuran representasi utama yang digunakan oleh model TabR. Dengan demikian, fitur *input* yang awalnya berupa data tabular dapat diproses dalam bentuk representasi laten.

Encoder Block

```
self.blocks0 = nn.ModuleList(
    [make_block(i > 0) for i in range(encoder_n_blocks)]
)
```

Bagian blocks0 merupakan kumpulan *encoder block* yang digunakan untuk memproses representasi fitur setelah melewati *layer linear*. *Encoder block* membantu model membentuk representasi fitur yang lebih informatif sebelum digunakan pada tahap *retrieval*. Jumlah *block* yang digunakan ditentukan oleh parameter encoder_n_blocks.

Pembentukan *Key* untuk *Retrieval*

```
self.K = nn.Linear(d_main, d_main)
```

Layer self.K digunakan untuk membentuk key dari representasi fitur. *Key* ini digunakan pada proses *retrieval* untuk membandingkan data *input* dengan kandidat sampel dari data *training*. Dengan *key* tersebut, model dapat mencari sampel *training* yang memiliki kedekatan representasi dengan data *input*.

Proses *Encoding* Fitur

```
x = torch.cat(x, dim=1)

x = self.linear(x)
for block in self.blocks0:
    x = x + block(x)

k = self.K(x if self.normalization is None else self.normalization(x))
```

```
return x, k
```

Fungsi _encode() digunakan untuk mengubah fitur *input* menjadi dua bentuk representasi, yaitu x dan k. Representasi x digunakan sebagai fitur utama yang diproses model, sedangkan k digunakan sebagai *key* untuk mencari *context sample*. Proses ini menjadi tahap awal sebelum model melakukan *retrieval* terhadap data *training*.

Pencarian *Context* Menggunakan FAISS

```
self.search_index.reset()
self.search_index.add(candidate_k)

distances, context_idx = self.search_index.search(
    k, context_size + (1 if is_train else 0)
)
```

Potongan kode tersebut menunjukkan proses pencarian *context* menggunakan FAISS. Representasi kandidat dari *data training* dimasukkan ke dalam *index*, kemudian model mencari sejumlah sampel yang paling dekat dengan representasi *input*. Sampel terdekat tersebut digunakan sebagai *context* dalam proses prediksi.

Pencegahan Data *Leakage* Saat *Training*

```
if is_train:
    distances[
        context_idx == torch.arange(batch_size, device=device)[:, None]
    ] = torch.inf
    context_idx = context_idx.gather(-1, distances.argsort()[:, :-1])
```

Pada saat *training*, model menghapus *data input* dari daftar *context* jika data tersebut muncul sebagai tetangga terdekat untuk dirinya sendiri. Tahap ini dilakukan untuk mencegah *data leakage*, sehingga model tidak menggunakan sampel yang sama sebagai acuan prediksi dirinya sendiri.

Perhitungan *Similarity Context*

```
similarities = (
    -k.square().sum(-1, keepdim=True)
    + (2 * (k[..., None, :] @ context_k.transpose(-1, -2))).squeeze(-2)
    - context_k.square().sum(-1)
)

probs = F.softmax(similarities, dim=-1)
```

Bagian ini digunakan untuk menghitung tingkat kemiripan antara representasi *input* dan *context* yang diperoleh dari FAISS. Nilai *similarity*

kemudian diubah menjadi probabilitas menggunakan fungsi *softmax*. Probabilitas tersebut menunjukkan seberapa besar pengaruh setiap *context* terhadap proses prediksi.

Penggabungan *Context* dengan Representasi *Input*

```
context_y_emb = self.label_encoder(candidate_y[context_idx][..., None])
values = context_y_emb + self.T(k[:, None] - context_k)
context_x = (probs[:, None] @ values).squeeze(1)
x = x + context_x
```

Label dari sampel *context* dikodekan menggunakan label_encoder, kemudian digabungkan dengan informasi selisih antara representasi *input* dan context. Hasil penggabungan ini digunakan untuk memperkaya representasi *input*. Dengan mekanisme tersebut, TabR tidak hanya memproses fitur dari satu sampel, tetapi juga mempertimbangkan sampel lain yang memiliki karakteristik serupa.

***Output* Prediksi Model**

```
for block in self.blocks1:
    x = x + block(x)

x = self.head(x)
return x
```

Tahap akhir model dilakukan melalui *predictor block* dan *output head*. *Predictor block* memproses representasi akhir yang telah digabungkan dengan *context*, sedangkan *output head* menghasilkan nilai prediksi. Pada penelitian ini, *output* model digunakan untuk menentukan kelas *anxiety* rendah atau *anxiety* tinggi.

5. Proses *Training* dan Evaluasi

Pipeline training TabR dijalankan melalui script bin/go.py yang mengotomatisasi tiga tahap: *tuning*, *evaluation*, dan *ensemble*.

Alur *Training* Lengkap:

```
# 1. Tuning Hyperparameter
python bin/tune.py exp/tabr/convat_apex_anxiety/0-tuning.toml

# 2. Evaluasi dengan 15 random seeds
python bin/evaluate.py exp/tabr/convat_apex_anxiety/0-tuning

# 3. Ensemble 5 model terbaik
python bin/ensemble.py exp/tabr/convat_apex_anxiety/0-evaluation
```

Detail proses pelatihan dan evaluasi adalah sebagai berikut:

- Perintah tune.py digunakan untuk mencari konfigurasi *hyperparameter* terbaik berdasarkan *file* eksperimen .toml.
- Hasil tuning disimpan pada *folder* eksperimen yang telah ditentukan.
- Setelah *tuning* selesai, perintah evaluate.py digunakan untuk mengevaluasi model menggunakan 15 random seeds.
- Penggunaan beberapa random seeds bertujuan untuk melihat kestabilan performa model pada inisialisasi yang berbeda.
- Setelah evaluasi selesai, perintah ensemble.py digunakan untuk menggabungkan 5 model terbaik.
- Ensemble dilakukan agar hasil prediksi lebih stabil dibandingkan hanya menggunakan satu model.
- *Output* dari tahap ini berupa hasil evaluasi model TabR yang kemudian digunakan untuk melihat performa klasifikasi *anxiety* dan tidak *anxiety*.

Tahap ini menjadi proses utama dalam pengujian model, karena hasil akhirnya digunakan untuk memperoleh metrik evaluasi seperti *accuracy*, *precision*, *recall*, dan *F1-score*.

Kumpulan Metrik per Epoch:

```
        def collect_epoch_metrics(run_dir: Path) -> pd.DataFrame:
            """
            Mengumpulkan metrik training, validation, dan test per epoch.
            """
            checkpoint = lib.load_checkpoint(run_dir, map_location='cpu')
            rows = []
            for item in checkpoint['training_log']:
                metrics = item['metrics']
                rows.append({
                    'epoch': len(rows),
                    'val_loss':       float(metrics['val'].get('cross-entropy',
np.nan)),
                    'test_loss':     float(metrics['test'].get('cross-entropy',
np.nan)),
                    'val_acc': float(metrics['val']['score']),
                    'test_acc': float(metrics['test']['score']),
                })

            epoch_df = pd.DataFrame(rows)

            # Merge dengan training loss dari TensorBoard
```

```
            train_loss = load_scalar_series(run_dir, 'loss_train', 'loss')
            epoch_df = epoch_df.merge(
                train_loss[['epoch',         'value']].rename(columns={'value':
'train_loss'}),
                on='epoch', how='left'
            )
            return epoch_df
```

Detail implementasinya adalah sebagai berikut:

- Fungsi collect_epoch_metrics() membaca *checkpoint* dari *folder* hasil eksperimen.
- Data metrik diambil dari *training*_log yang tersimpan pada *checkpoint*.
- Setiap epoch menyimpan nilai val_loss, test_loss, val_acc, dan test_acc.
- Nilai loss diambil dari metrik *cross-entropy*.
- Nilai akurasi diambil dari *field score* pada data *validation* dan *test*.
- Data metrik kemudian disusun ke dalam *dataframe*.
- Nilai *train*_loss ditambahkan dari log TensorBoard menggunakan fungsi *load*_scalar_series().
- Hasil akhirnya adalah data*frame* berisi metrik *training*, *validation*, dan *test* untuk setiap *epoch*.

Output dari tahap ini digunakan untuk menganalisis perkembangan performa model selama proses pelatihan.

6. Evaluasi Model Terbaik

Setelah *training* selesai, model terbaik dipilih berdasarkan akurasi validasi tertinggi dari 15 random seeds.

```
        def evaluate_experiment(cfg: dict) -> tuple[pd.DataFrame, dict]:
            """
            Mengevaluasi eksperimen dan memilih seed terbaik berdasarkan val
accuracy.
            """
            dataset_dir = DATA_ROOT / cfg['dataset_name']
            split_info = load_split_info_from_dataset(dataset_dir)
            eval_dir = EXP_ROOT / cfg['exp_name'] / '0-evaluation'

            # Pilih seed terbaik berdasarkan val score
            best_seed = max(
                range(N_EVAL_SEEDS),
                key=lambda s: json.loads(
                    (eval_dir / str(s) / 'report.json').read_text()
```

```
                )['metrics']['val']['score'],
            )

            best_run_dir = eval_dir / str(best_seed)
            result = evaluate_best_run(best_run_dir, split_info)
            summary = summarize_result(cfg, result, best_seed)

            return summary, {
                'cfg': cfg,
                'best_seed': best_seed,
                'best_run_dir': best_run_dir,
                'result': result,
            }
```

Detail implementasinya adalah sebagai berikut:

- Fungsi evaluate_experiment() menerima konfigurasi eksperimen sebagai *input*.
- Sistem membaca *folder dataset* dan informasi pembagian data *train*, *validation*, dan *test*.
- Hasil evaluasi diambil dari *folder* 0-evaluation.
- Setiap *seed* memiliki *file* report.json yang berisi metrik hasil evaluasi.
- Sistem memilih seed terbaik berdasarkan nilai *val score* tertinggi.
- Setelah *seed* terbaik ditemukan, sistem membaca hasil evaluasi dari *folder seed* tersebut.
- Hasil evaluasi kemudian diringkas menggunakan fungsi summarize_result().

Output dari tahap ini adalah ringkasan hasil eksperimen dan informasi *seed* terbaik yang digunakan sebagai hasil evaluasi utama.

Perhitungan Akurasi *Training* Final:

```
        def infer_train_accuracy_from_predictions(run_dir: Path, y_train:
np.ndarray) -> float:
            """
            Menghitung akurasi training dari prediksi model (bukan dari log).
            """
            preds = lib.load_predictions(run_dir)
            train_logits = preds['train']
            train_prob = expit(train_logits)
            train_pred = (train_prob >= 0.5).astype(int)
            return float((train_pred == y_train).mean())
```

Detail implementasinya adalah sebagai berikut:

- Fungsi infer_*train_accuracy*_from_predictions() menerima *folder* hasil eksperimen dan label *training* sebagai *input*.
- Prediksi model dibaca menggunakan lib.*load*_predictions().
- Nilai prediksi *training* diambil dari bagian preds['*train*'].
- *Output* model masih berbentuk logit, sehingga dikonversi menjadi probabilitas menggunakan fungsi expit().
- Probabilitas kemudian diubah menjadi kelas prediksi dengan *threshold* 0,5.
- Jika probabilitas lebih besar atau sama dengan 0,5, data diprediksi sebagai kelas 1.
- Jika probabilitas kurang dari 0,5, data diprediksi sebagai kelas 0.
- Akurasi dihitung dengan membandingkan hasil prediksi terhadap label asli y_*train*.

Output dari tahap ini adalah nilai akurasi *training* yang diperoleh langsung dari prediksi model.

5.2 Pengujian

Pengujian sistem dilakukan untuk mengevaluasi kinerja model *Tabular Retrieval Network* (TabR) yang telah dilatih dalam mengklasifikasikan kondisi *anxiety* tinggi dan *anxiety* rendah berdasarkan data tabular hasil ekstraksi fitur *micro-expression*. Pengujian ini menggunakan 20 data test yang terdiri dari video ekspresi wajah subjek yang tidak dilibatkan dalam proses pelatihan maupun pembentukan *context*. Data test ini telah melalui proses *preprocessing* yang sama dengan data pelatihan, meliputi konversi video ke *frame*, deteksi wajah menggunakan Dlib, ekstraksi *Region of Interest* (ROI) pada area mata-alis kanan, mata-alis kiri, dan mulut, *spotting micro-expression* untuk menentukan fase *onset-apex-offset*, serta ekstraksi fitur menggunakan metode *Phase Only Correlation with All Block Search* (POC-ABS) dengan *block size* 7×7.

5.2.1 Persiapan Data Test

Data test sebanyak 20 sampel berasal dari rekaman video ekspresi wajah mahasiswa Program Studi Teknik Informatika semester IV dengan rentang usia 19-20 tahun. Setiap sampel merupakan hasil perekaman selama pengisian kuesioner *Faces Anxiety Scale* (FAS) pada kondisi *before* atau *after* ujian. Tingkat *anxiety* setiap sampel ditentukan berdasarkan skor kuesioner yang dihitung menggunakan skema penilaian yang telah ditetapkan dan divalidasi oleh pakar. Berdasarkan hasil validasi tersebut, setiap sampel kemudian dikelompokkan ke dalam kategori *anxiety* rendah dan *anxiety* tinggi sesuai rentang skor yang telah ditentukan. Komposisi data uji disusun sebagai berikut:

- 10 data video: Kondisi *anxiety* tinggi (skor kuesioner 11–20)
- 10 data video: Kondisi *anxiety* rendah (skor kuesioner 1–10)

Dalam proses pengujian, data video yang digunakan bersifat independen terhadap data pelatihan sehingga model tidak pernah melihat sampel tersebut sebelumnya. Pendekatan ini dilakukan untuk mengukur kemampuan generalisasi model dalam mengenali pola *micro-expression anxiety* pada data baru. Dengan demikian, hasil evaluasi yang diperoleh dapat merepresentasikan performa sistem secara lebih objektif terhadap kondisi nyata.

5.2.2 Skenario Pengujian

- **Pengujian per *event*:** Setiap *event micro-expression* yang terdeteksi dari proses *spotting* diuji secara individual untuk menghasilkan satu prediksi. Satu data video dapat menghasilkan beberapa *event*, dan setiap *event* menjadi satu sampel uji yang independen.
- **Pengujian batch:** Seluruh hasil prediksi per *event* dari 20 data video dikumpulkan untuk perhitungan metrik evaluasi secara agregat.
- **Level prediksi:** Prediksi dilakukan pada level *event*, bukan pada level subjek maupun level video.

5.2.3 Metrik Evaluasi

Evaluasi performa model dilakukan menggunakan *confusion matrix, accuracy, precision, recall,* dan *F1-score*. *Confusion matrix* digunakan untuk menggambarkan distribusi hasil klasifikasi antara label aktual dan label prediksi pada kelas *anxiety* tinggi dan *anxiety* rendah, sehingga dapat diketahui jumlah prediksi benar maupun prediksi salah yang dihasilkan model. *Accuracy* digunakan untuk mengukur proporsi keseluruhan prediksi yang berhasil diklasifikasikan dengan benar terhadap seluruh data uji.

Selain itu, *precision* digunakan untuk mengukur tingkat ketepatan model dalam memprediksi kelas *anxiety*, sedangkan *recall* digunakan untuk mengukur kemampuan model dalam mendeteksi seluruh data *anxiety* yang sebenarnya ada pada data uji. *F1-score* digunakan sebagai kombinasi antara *precision* dan *recall* untuk memberikan evaluasi performa model yang lebih seimbang, terutama pada kondisi distribusi data yang tidak sepenuhnya seimbang antar kelas.

BAB VI. HASIL DAN PEMBAHASAN

6.1 Hasil

Hasil ini memaparkan seluruh tahapan penelitian yang telah dilaksanakan, mulai dari pengolahan dataset video wajah hingga pengujian kinerja model. Pemaparan hasil disusun secara berurutan agar alur penelitian dapat terlihat secara utuh, mulai dari pembentukan data awal, pengolahan frame, deteksi area wajah, ekstraksi fitur pergerakan mikro, pembentukan event micro-expression, penyusunan data tabular, pembentukan model, sampai evaluasi hasil klasifikasi.

Dataset yang digunakan pada penelitian ini berupa data video wajah responden yang telah diproses menjadi fitur pergerakan wajah berbasis landmark pada *level frame* dan *event onset-apex-offset*. Pada pembentukan dataset internal, penelitian ini menggunakan skema *question-wise split* berdasarkan kelompok pertanyaan kuesioner. Eksperimen menggunakan data pertanyaan pertama sampai pertanyaan ketiga sebagai data latih, pertanyaan keempat sebagai data validasi, dan pertanyaan kelima sebagai data uji internal. Dataset pelatihan terdiri dari 333 video atau 1.663 *event* dengan total 35.863 *frame*. Data validasi terdiri dari 111 video atau 550 *event* dengan total 11.356 frame, sedangkan data uji internal terdiri dari 111 video atau 550 *event* dengan total 10.437 *frame*. Ringkasan komposisi dataset internal ditunjukkan pada Tabel 6.1.

Tabel 6.1 Komposisi *Dataset*

Jenis Data	Kelompok Pertanyaan	Jumlah Video	Jumlah *Event*	Jumlah *Frame*
Data latih	Q1–Q3	333	1.663	35.863
Data validasi	Q4	111	550	11.356
Data uji internal	Q5	111	550	10.437

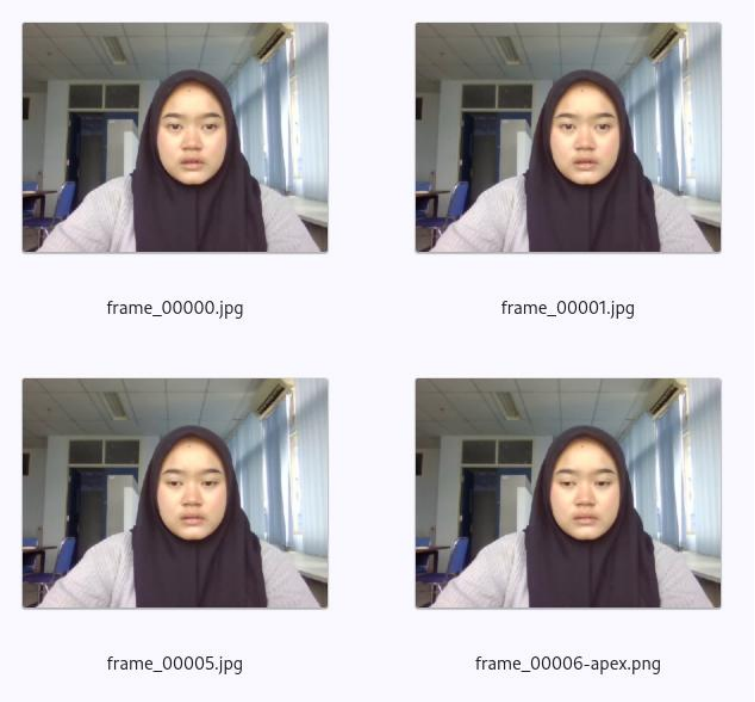
Selain pengujian internal, penelitian ini juga melakukan pengujian akhir menggunakan *datatest*. *Datatest* terdiri dari 20 subjek yang tidak digunakan dalam proses *training* model, yaitu 10 subjek *anxiety* tinggi dan 10 subjek

anxiety rendah. Data tersebut menghasilkan 175 video, 692 *event onset-apex-offset*, dan 8.223 *frame*. Berdasarkan label aktualnya, *datatest* terdiri dari 370 *event anxiety* tinggi dan 322 *event anxiety* rendah. Ringkasan komposisi *datatest* ditunjukkan pada Tabel 6.2.

Tabel 6.1 Tabel Komposisi *Datatest*

Komponen	Jumlah
Subjek *anxiety* rendah	10
Subjek *anxiety* tinggi	10
Total subjek	20
Total video	175
Total *event*	692
Total *frame*	8.223
Event *anxiety* rendah	322
Event *anxiety* tinggi	370

Tahap awal pemrosesan data dilakukan dengan memecah setiap video wajah menjadi rangkaian *frame*. Proses ini dilakukan agar data video dapat diubah menjadi unit citra yang lebih mudah diproses pada tahap berikutnya. Setiap *frame* yang dihasilkan disusun secara berurutan sesuai urutan waktu pada video asli, sehingga informasi temporal tetap terjaga. Gambar 6.1 menunjukkan contoh hasil pemecahan video menjadi beberapa *frame* berurutan. Hasil dari tahap ini berupa kumpulan *frame* wajah yang kemudian digunakan sebagai masukan pada proses *preprocessing*, yaitu deteksi wajah, pemotongan area wajah, dan penyesuaian ukuran citra.



Gambar 6.1 Gambar Hasil video menjadi *frame*

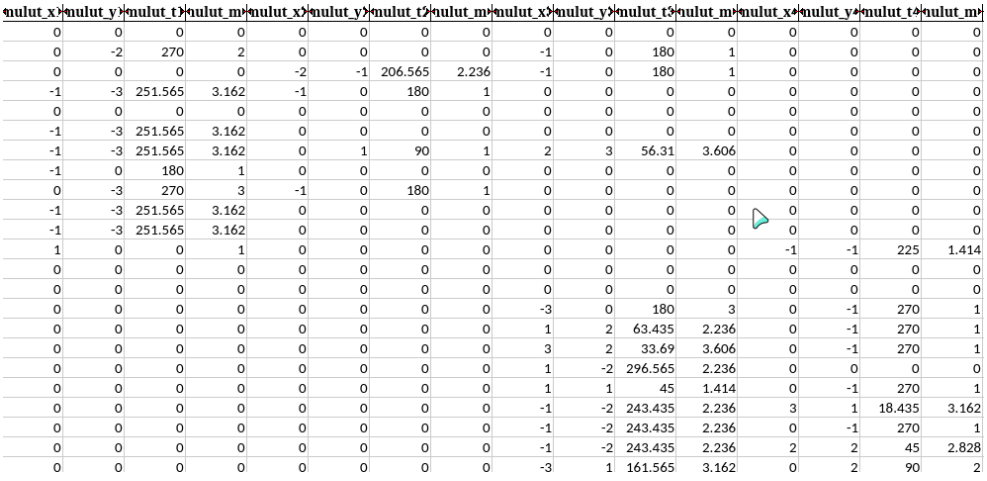
Setelah frame diperoleh, tahap berikutnya adalah mengambil *Region of Interest* (ROI) pada area wajah yang digunakan dalam penelitian. ROI yang dihasilkan terdiri dari area mata-alis kiri, mata-alis kanan, dan mulut. Hasil cropping ROI ditunjukkan pada Gambar 6.2. Berdasarkan gambar tersebut, area wajah yang digunakan untuk analisis berhasil dipisahkan dari wajah utama, sehingga proses ekstraksi fitur pada tahap berikutnya dapat difokuskan pada bagian wajah yang relevan terhadap pergerakan *micro-expression*.



Gambar 6.2 Gambar Contoh Hasil *Cropping Region of Interest* (ROI)

Setelah ROI diperoleh, tahap berikutnya adalah ekstraksi fitur pergerakan mikro menggunakan metode POC-ABS. Pada tahap ini, setiap ROI diproses dengan membandingkan pasangan *frame* berurutan untuk memperoleh nilai pergeseran pada area wajah. Proses perhitungan dilakukan dengan membagi

ROI ke dalam blok berukuran 7×7 piksel. Setiap blok menghasilkan empat fitur pergerakan, yaitu X, Y, *theta*, dan *magnitude*. Fitur X dan Y merepresentasikan arah pergeseran, *theta* merepresentasikan sudut arah gerak, sedangkan *magnitude* merepresentasikan besar pergerakan pada blok tersebut.



mulut_x	mulut_y	mulut_t	mulut_m	mulut_x	mulut_y	mulut_t	mulut_m	mulut_x	mulut_y	mulut_t	mulut_m	mulut_x	mulut_y	mulut_t	mulut_m
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	-2	270	2	0	0	0	0	-1	0	180	1	0	0	0	0
0	0	0	0	-2	-1	206.565	2.236	-1	0	180	1	0	0	0	0
-1	-3	251.565	3.162	-1	0	180	1	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
-1	-3	251.565	3.162	0	0	0	0	0	0	0	0	0	0	0	0
-1	-3	251.565	3.162	0	1	90	1	2	3	56.31	3.606	0	0	0	0
-1	0	180	1	0	0	0	0	0	0	0	0	0	0	0	0
0	-3	270	3	-1	0	180	1	0	0	0	0	0	0	0	0
-1	-3	251.565	3.162	0	0	0	0	0	0	0	0	0	0	0	0
-1	-3	251.565	3.162	0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	0	0	-1	-1	225	1.414
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	-3	0	180	3	0	-1	270	1
0	0	0	0	0	0	0	0	1	2	63.435	2.236	0	-1	270	1
0	0	0	0	0	0	0	0	3	2	33.69	3.606	0	-1	270	1
0	0	0	0	0	0	0	0	1	-2	296.565	2.236	0	0	0	0
0	0	0	0	0	0	0	0	1	1	45	1.414	0	-1	270	1
0	0	0	0	0	0	0	0	-1	-2	243.435	2.236	3	1	18.435	3.162
0	0	0	0	0	0	0	0	-1	-2	243.435	2.236	0	-1	270	1
0	0	0	0	0	0	0	0	-1	-2	243.435	2.236	2	2	45	2.828
0	0	0	0	0	0	0	0	-3	1	161.565	3.162	0	2	90	2

Gambar 6.3 Contoh Bentuk Data Fitur Hasil Ekstraksi POC-ABS

Tabel 6.3 Contoh Fitur Hasil Ekstraksi POC-ABS per Blok

Blok	X	Y	Magnitude	Theta
10	10	10	2.00	0° (kanan)
22	17	17	2.24	-63° (kanan bawah)
41	10	31	2.00	90° (bawah)
46	45	31	1.41	-45° (kanan bawah)
50	63	31	2.00	180° (kiri)

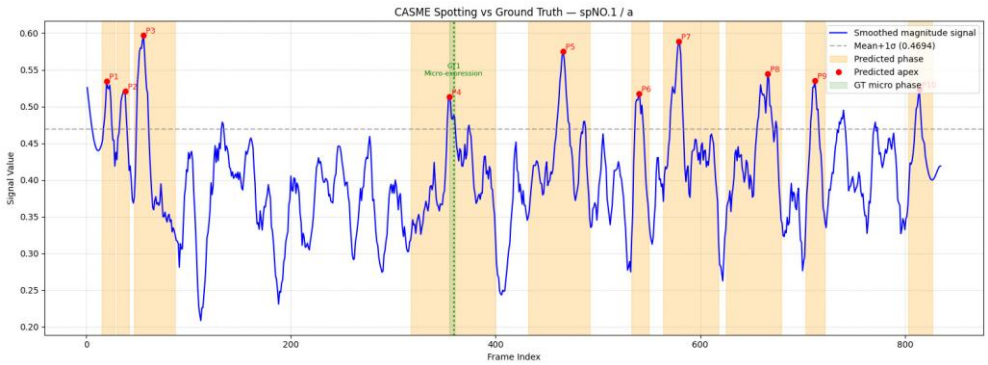
Berdasarkan contoh tersebut, setiap blok menghasilkan nilai numerik yang menggambarkan arah dan besar pergerakan pada area ROI. Nilai *magnitude* yang lebih besar menunjukkan adanya pergerakan yang lebih kuat pada blok tersebut, sedangkan nilai *theta* menunjukkan arah pergerakan. Bentuk data ini menjadi dasar pembentukan dataset tabular yang digunakan sebagai acuan untuk *spotting*.

Setelah *Region of Interest* (ROI) pada area mata-alis kiri, mata-alis kanan, dan mulut diperoleh, tahap berikutnya adalah melakukan *spotting micro-expression* untuk menentukan rentang kemunculan pergerakan wajah yang

relevan. Pada tahap ini, sistem menghitung nilai pergerakan antar *frame* pada setiap ROI menggunakan metode *Phase Only Correlation with All Block Search* (POC-ABS). Nilai pergerakan tersebut kemudian direpresentasikan dalam bentuk rata-rata *magnitude* sebagai intensitas pergerakan wajah pada setiap *frame*.

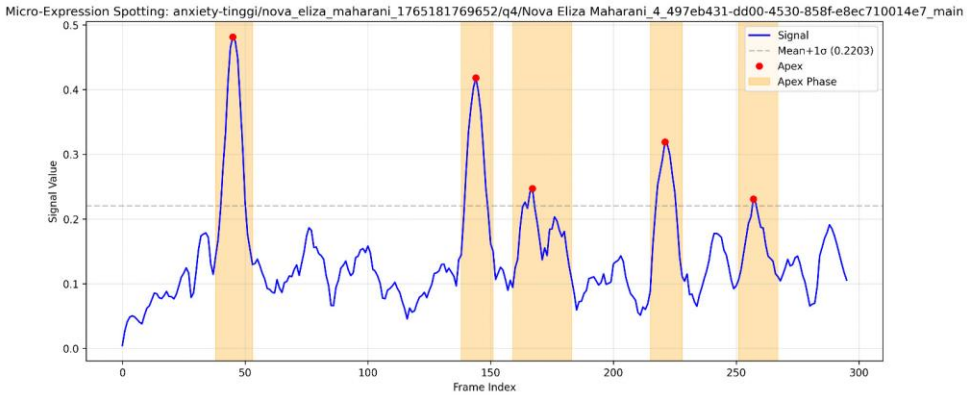
Berdasarkan sinyal pergerakan yang dihasilkan, sistem menentukan threshold adaptif pada setiap video menggunakan nilai rata-rata sinyal yang ditambahkan dengan satu standar deviasi. *Frame* dengan nilai *mean magnitude* yang melewati *threshold* tersebut digunakan sebagai kandidat *event onset-apex-offset*. Dari kandidat event tersebut, sistem kemudian menentukan *onset* sebagai awal pergerakan, *apex* sebagai titik puncak pergerakan, dan *offset* sebagai akhir pergerakan. Hasil dari tahap ini berupa rangkaian frame yang telah memiliki informasi *onset*, *apex*, dan *offset* untuk digunakan pada tahap ekstraksi fitur berikutnya.

Sebagai pengujian awal terhadap proses *spotting*, dilakukan perbandingan hasil sistem dengan data CASME yang memiliki informasi *ground truth micro-expression*. Gambar 6.4 menunjukkan perbandingan antara hasil *spotting* sistem dan *ground truth* CASME. Pada grafik tersebut, garis biru menunjukkan sinyal *magnitude*, garis putus-putus menunjukkan *threshold adaptif*, area kuning menunjukkan fase hasil prediksi sistem, titik merah menunjukkan kandidat *apex*, sedangkan area hijau menunjukkan fase *ground truth*. Berdasarkan visualisasi tersebut, salah satu kandidat *apex* berada pada rentang *ground truth*, sehingga proses *spotting* dapat digunakan untuk mendeteksi kandidat *event*.



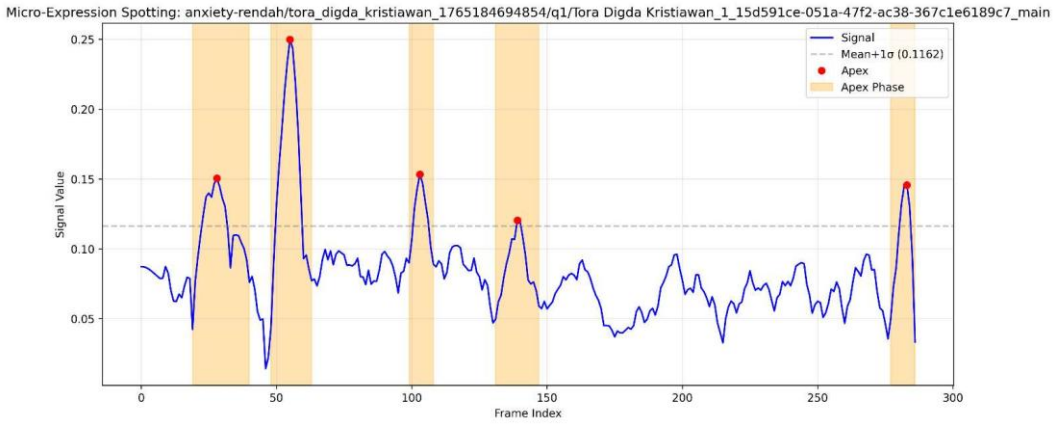
Gambar 6.4 Perbandingan Hasil *Spotting* dengan *Ground Truth* CASME

Sebagai contoh hasil *spotting* pada data penelitian, Gambar 6.5 dan Gambar 6.6 menunjukkan visualisasi sinyal pergerakan pada dua video dengan kategori yang berbeda, yaitu *anxiety* tinggi dan *anxiety* rendah. Grafik tersebut digunakan untuk memperlihatkan bagaimana sinyal *magnitude* menghasilkan kandidat *event onset-apex-offset* berdasarkan *threshold* adaptif yang dihitung pada masing-masing video.



Gambar 6.5 Hasil *Spotting* pada Video *Anxiety* Tinggi

Pada Gambar 6.5 ditampilkan hasil *spotting* pada video dengan kategori anxiety tinggi. Grafik menunjukkan bahwa nilai threshold adaptif berada pada 0,2203. Beberapa titik *apex* terdeteksi ketika sinyal *magnitude* melewati batas tersebut, dengan puncak tertinggi berada di sekitar frame ke-45 dan memiliki nilai mendekati 0,48. Selain itu, beberapa event lain juga muncul pada bagian tengah hingga akhir video, yang menunjukkan adanya beberapa segmen pergerakan wajah dengan intensitas relatif tinggi.



Gambar 6.6 Hasil *Spotting* pada Video *Anxiety* Rendah

Pada Gambar 6.6 ditampilkan hasil *spotting* pada video dengan kategori *anxiety* rendah. Nilai *threshold* adaptif pada grafik ini berada pada 0,1162. Titik *apex* yang terdeteksi memiliki nilai puncak lebih rendah dibandingkan contoh *anxiety* tinggi, dengan puncak tertinggi berada di sekitar frame ke-55 dan memiliki nilai mendekati 0,25. Beberapa event tetap terdeteksi, namun intensitas sinyal pergerakan yang muncul cenderung lebih kecil.

Berdasarkan ketiga visualisasi tersebut, proses spotting mampu menunjukkan bahwa sistem dapat mendeteksi kandidat *event onset-apex-offset* dari sinyal pergerakan wajah. Hasil pada data CASME menunjukkan bahwa kandidat *apex* yang dihasilkan sistem dapat berada pada rentang *ground truth*, sedangkan pada data penelitian terlihat bahwa video dengan kategori *anxiety* tinggi cenderung memiliki *threshold* dan puncak sinyal yang lebih besar dibandingkan video *anxiety* rendah. Hasil *spotting* ini selanjutnya digunakan sebagai dasar untuk menentukan *event* yang akan diproses pada tahap ekstraksi fitur.

Setelah *event onset-apex-offset diperoleh*, hasil *spotting* digunakan sebagai acuan dalam pembentukan data tabular. Data disusun pada level *frame*, sehingga setiap baris merepresentasikan satu *frame* yang berada dalam cakupan *event*. Kolom data memuat informasi *event*, nomor *frame*, serta fitur pergerakan hasil ekstraksi POC-ABS berupa X, Y, *theta*, dan *magnitude* dari ROI mata-alis kiri, mata-alis kanan, dan mulut. Dengan demikian, input model TabR berupa fitur frame terpilih dari *event*, bukan seluruh frame video.

Tabel 6.4 Contoh Bentuk Data Tabular Hasil Ekstraksi Fitur

ROI	Ukuran ROI	Jumlah Blok	Total Blok	Fitur per Blok	Total Fitur
Mulut	70×35 piksel	10×5	50	4	200
Mata-alis kiri	48×32 piksel	6×4	24	4	96
Mata-alis kanan	48×32 piksel	6×4	24	4	96
Total	-	-	98	-	392

Pada penelitian ini, setiap frame dalam cakupan event direpresentasikan ke dalam 392 fitur pergerakan.

Setelah data tabular hasil ekstraksi fitur terbentuk, tahap berikutnya adalah pembentukan model TabR. Data numerik yang digunakan sebagai input model disiapkan berdasarkan daftar fitur yang telah ditentukan, kemudian digunakan untuk klasifikasi dua kelas, yaitu *anxiety* tinggi dan *anxiety* rendah. Pada eksperimen, pembagian data mengikuti skema *question-wise split*, yaitu Q1–Q3 sebagai data latih, Q4 sebagai data validasi, dan Q5 sebagai data uji internal.

Sebelum model akhir ditentukan, dilakukan *hyperparameter tuning* untuk memperoleh konfigurasi model TabR terbaik. Proses *tuning* dilakukan sebanyak 15 percobaan dengan kombinasi parameter yang berbeda. Setiap percobaan dievaluasi menggunakan data validasi Q4, kemudian konfigurasi dengan skor validasi terbaik dipilih sebagai konfigurasi utama. Setelah konfigurasi terbaik diperoleh, model dievaluasi ulang menggunakan 15 seed berbeda untuk melihat stabilitas performa model.

Tabel 6.5 Ruang Pencarian Hyperparameter Model TabR

Parameter	Ruang Pencarian
Context size	4-16
D main	96-256
Encoder blocks	0-2
Predictor blocks	1-2
Context dropout	0,00-0,35
Dropout0	0,05-0,40
Dropout1	0,00-0,30
Learning rate	0,00001-0,0008
Weight decay	0,000001-0,0005

Selain parameter yang dicari melalui proses *tuning*, terdapat beberapa parameter tetap yang digunakan selama proses pelatihan model. Parameter tetap tersebut meliputi *batch size* 128, *patience* 6, *optimizer* AdamW, *normalization* LayerNorm, *activation* ReLU, dan d *multiplier* 2,0.

Tabel 6.6 Parameter Tetap Pelatihan Model TabR

Parameter	Nilai
Batch size	128
Patience	6
Optimizer	AdamW
Normalization	LayerNorm
Activation	ReLU
D multiplier	2,0
Jumlah trial	15

Parameter tetap dipakai sebagai *baseline* stabil. *Batch size* 128 cukup dan masih ringan untuk GPU laptop yang digunakan pada eksperimen. *Patience* 6 untuk *early stopping* agar *training* berhenti saat validasi tidak memaik. AdamW, LayerNorm, ReLU, dan d_multiplier 2.0 dipakai karena konfigurasi standar TabR yang sudah aan, jadi *tuning* difokuskan ke parameter yang paling berpengaruh.

Tabel 6.2 Konfigurasi Terbaik Hasil *Hyperparameter Tuning*

Parameter	Nilai
Trial terbaik	10
Seed tuning	72
Batch size	128
Patience	6
Best epoch tuning	44
Context size	9
D main	133
Encoder blocks	0
Predictor blocks	2
Context dropout	0,0099
Dropout0	0,3967
Dropout1	0,2990
Learning rate	0,0007403
Weight decay	0,00000842

Jumlah parameter	284.621

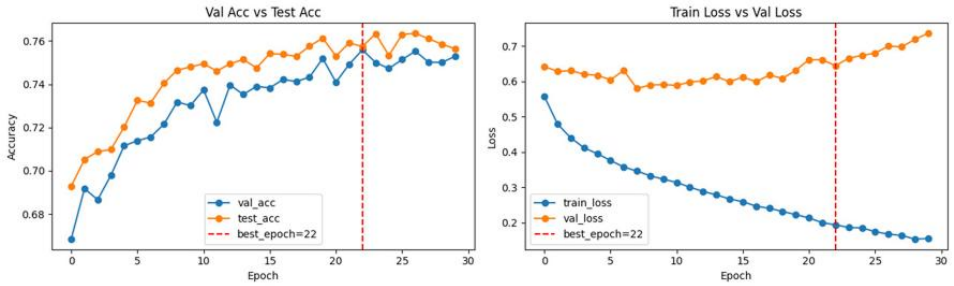
Berdasarkan hasil *tuning*, konfigurasi terbaik diperoleh pada *trial* ke-10. Konfigurasi tersebut menggunakan *context size* 9, d main 133, *encoder blocks* 0, *predictor blocks* 2, *context dropout* 0,0099, dropout0 0,3967, dropout1 0,2990, *learning rate* 0,0007403, dan *weight decay* 0,00000842. Konfigurasi ini menghasilkan jumlah parameter sebanyak 284.621.

Setelah konfigurasi terbaik diperoleh, performa model pada tahap *tuning* dievaluasi pada data latih, validasi Q4, dan uji internal Q5. Hasil evaluasi konfigurasi terbaik pada tahap tuning ditunjukkan pada Tabel 6.7.

Tabel 6.3 Hasil Performa Model Terbaik pada Tahap Tuning

Split	Accuracy	Macro Precision	Macro Recall	Macro F1	ROC-AUC	Cross Entropy
Train	0,9904	0,9913	0,9883	0,9898	0,9996	0,0371
Validasi Q4	0,7592	0,7439	0,7370	0,7399	0,8187	0,8733
Test Q5	0,7579	0,7356	0,7278	0,7312	0,8123	0,8691

Setelah tahap *tuning* selesai, konfigurasi terbaik dijalankan ulang menggunakan beberapa *seed* untuk memperoleh model akhir. Berdasarkan hasil evaluasi ulang, seed 5 menghasilkan nilai validasi terbaik dengan best *epoch* ke-22. Grafik pelatihan model akhir ditunjukkan pada Gambar 6.7.

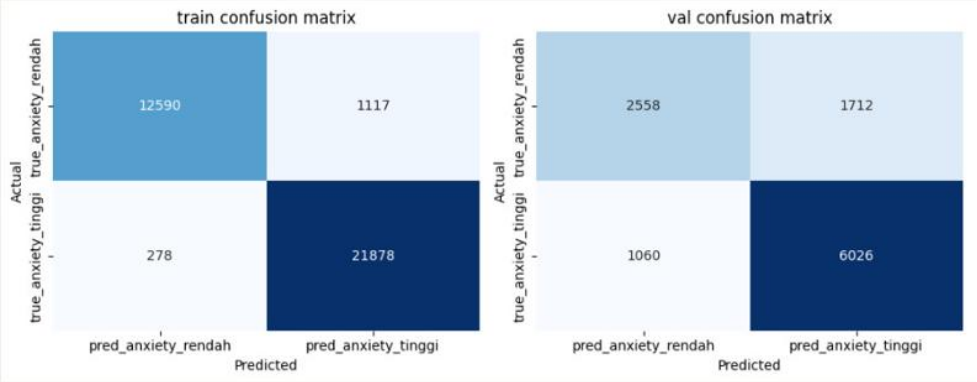


Gambar 6.7 Grafik Pelatihan Model TabR

Berdasarkan Gambar 6.9, *validation accuracy* dan *test accuracy* mengalami peningkatan secara bertahap hingga mendekati *epoch* terbaik. Model terbaik diperoleh pada *epoch* ke-22. Pada *grafik loss*, *train loss* terus mengalami penurunan, sedangkan *validation loss* cenderung meningkat setelah *epoch*

tertentu. Kondisi ini menunjukkan bahwa model mulai terlalu menyesuaikan diri terhadap data *training*, sehingga model pada *epoch* terbaik digunakan untuk tahap pengujian.

Setelah proses pelatihan model akhir selesai, performa model pada data training dan validation ditampilkan dalam bentuk *confusion matrix*. *Confusion matrix* digunakan untuk melihat distribusi prediksi benar dan salah pada masing-masing kelas *anxiety* rendah dan *anxiety* tinggi. Hasil confusion matrix training dan validation ditunjukkan pada Gambar 6.8.

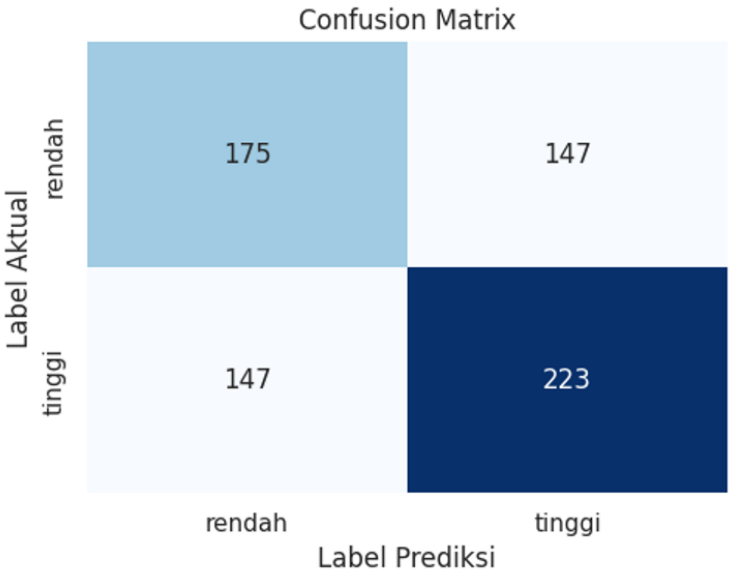


Gambar 6.8 *Confusion Matrix Training* dan *Validation*

Berdasarkan Gambar 6.8, pada data *training* model berhasil mengklasifikasikan 12.590 sampel *anxiety* rendah dan 21.878 sampel *anxiety* tinggi dengan benar. Kesalahan prediksi pada data *training* terdiri dari 1.117 sampel *anxiety* rendah yang diprediksi sebagai *anxiety* tinggi dan 278 sampel *anxiety* tinggi yang diprediksi sebagai *anxiety* rendah. Pada data *validation*, model berhasil mengklasifikasikan 2.558 sampel *anxiety* rendah dan 6.026 sampel *anxiety* tinggi dengan benar. Kesalahan prediksi pada data *validation* terdiri dari 1.712 sampel *anxiety* rendah yang diprediksi sebagai *anxiety* tinggi dan 1.060 sampel *anxiety* tinggi yang diprediksi sebagai *anxiety* rendah.

Pengujian akhir dilakukan menggunakan *datatest* yang tidak digunakan pada proses *training*, *validation*, maupun uji internal. Pengujian ini bertujuan untuk melihat performa model terhadap data baru. Setiap *frame* yang berada dalam cakupan *event onset-apex-offset* diklasifikasikan menggunakan model TabR, kemudian hasil prediksi dibandingkan dengan label aktual untuk

membentuk *confusion matrix datatest*. Hasil *confusion matrix datatest* ditunjukkan pada Gambar 6.9.



Gambar 6.9 *Confussion Matrix Datatest*

Berdasarkan Gambar 6.9, model berhasil mengklasifikasikan 175 sampel *anxiety* rendah dan 223 sampel *anxiety* tinggi dengan benar. Sementara itu, terdapat 147 sampel *anxiety* rendah yang diprediksi sebagai *anxiety* tinggi dan 147 sampel *anxiety* tinggi yang diprediksi sebagai *anxiety* rendah. Hasil tersebut menunjukkan bahwa model masih menghasilkan kesalahan prediksi pada kedua kelas dengan jumlah yang seimbang.

Berdasarkan *confusion matrix datatest*, dilakukan perhitungan metrik evaluasi berupa *accuracy*, *precision*, *recall*, dan F1-score. Hasil metrik evaluasi *datatest* ditunjukkan pada Tabel 6.9.

Tabel 6.4 *Metrik* Evaluasi *Datatest*

Kelas	Precision	Recall	F1-Score	Support
Anxiety rendah	0,54	0,55	0,55	322
Anxiety tinggi	0,60	0,59	0,59	370
Accuracy	-	-	0,58	692

Berdasarkan Tabel 6.9, model memperoleh *accuracy* sebesar 0.58 atau 58% pada *datatest*. Pada kelas *anxiety* rendah, model memperoleh *precision* 0.54, *recall* 0.55, dan F1-score 0.55 dengan support sebanyak 322 sampel. Pada kelas *anxiety* tinggi, model memperoleh *precision* 0.6, *recall* 0.59, dan F1-score sebesar 0.59 dengan support sebanyak 370 sampel. Hasil ini menunjukkan bahwa performa model pada *datatest* lebih rendah dibandingkan hasil validasi dan uji internal pada tahap *tuning*.

6.2 Pembahasan

Bagian ini membahas hasil pengujian yang telah dipaparkan pada sub bab 6.1 dengan mengaitkannya pada tujuan penelitian yang telah ditetapkan pada sub bab 1.4. Pembahasan difokuskan pada kemampuan fitur POC-ABS dalam merepresentasikan pergerakan *micro-expression*, performa model TabR dalam mengklasifikasikan *anxiety* rendah dan *anxiety* tinggi, pola kesalahan prediksi model, serta interpretasi metrik *accuracy*, *precision*, *recall*, dan F1-score.

6.2.1 Representasi Pergerakan Micro-Expression dengan POC-ABS

Berdasarkan hasil pada Gambar 6.2, area wajah yang digunakan dalam penelitian berhasil dipisahkan menjadi ROI mata-alis kiri, mata-alis kanan, dan mulut. Pemisahan ROI ini membuat proses analisis pergerakan tidak dilakukan pada seluruh wajah, tetapi difokuskan pada area yang relevan terhadap perubahan *micro-expression*. Dengan demikian, fitur pergerakan yang dihasilkan POC-ABS berasal dari bagian wajah yang lebih spesifik.

Kemampuan POC-ABS dalam merepresentasikan pergerakan *micro-expression* dapat dilihat dari hasil *spotting* pada Gambar 6.4, Gambar 6.5, dan Gambar 6.6. Pada Gambar 6.4, hasil perbandingan dengan *ground truth* CASME menunjukkan bahwa kandidat *apex* yang dihasilkan sistem berada pada rentang *ground truth*. Hal ini menunjukkan bahwa sinyal *magnitude* hasil POC-ABS dapat digunakan sebagai dasar untuk mendeteksi kandidat *event onset-apex-offset*.

Pada data penelitian, contoh video *anxiety* tinggi dan *anxiety* rendah menunjukkan pola sinyal yang berbeda. Video *anxiety* tinggi memiliki *threshold* adaptif sebesar 0,2203 dengan puncak *magnitude* mendekati 0,48, sedangkan video *anxiety* rendah memiliki *threshold* adaptif sebesar 0,1162 dengan puncak *magnitude* mendekati 0,25. Ringkasan hasil tersebut ditunjukkan pada Tabel 6.10.

Tabel 6.10 Ringkasan Hasil *Spotting*

Data	*Threshold* Adaptif	Puncak *Magnitude*	Keterangan
CASME	-	-	Kandidat *apex* berada pada rentang *ground truth*
Anxiety tinggi	0,2203	±0,48	Memiliki puncak pergerakan lebih besar
Anxiety rendah	0,1162	±0,25	Memiliki puncak pergerakan lebih kecil

Berdasarkan Tabel 6.10, POC-ABS mampu menghasilkan sinyal pergerakan yang dapat membedakan intensitas gerak pada contoh *anxiety* tinggi dan *anxiety* rendah. Nilai *threshold* dan puncak *magnitude* pada *anxiety* tinggi lebih besar dibandingkan *anxiety* rendah, sehingga fitur *magnitude* dapat merepresentasikan tingkat intensitas pergerakan wajah.

Namun, perbedaan nilai *magnitude* tersebut belum dapat digunakan sebagai keputusan klasifikasi secara langsung. Hal ini karena pola pergerakan pada *anxiety* rendah dan *anxiety* tinggi masih dapat saling tumpang tindih pada data yang lebih luas. Oleh karena itu, hasil *spotting* digunakan sebagai acuan untuk memilih *frame* dalam cakupan *event*, kemudian fitur POC-ABS disusun menjadi data tabular sebagaimana ditunjukkan pada Gambar 6.2. Data tabular tersebut menjadi representasi numerik yang digunakan sebagai masukan model TabR.

6.2.2 Performa Model TabR dalam Klasifikasi Anxiety

Performa model TabR dianalisis berdasarkan hasil *tuning*, grafik pelatihan, dan *confusion matrix training-validation* yang telah ditampilkan

pada sub bab 6.1. Berdasarkan Tabel 6.7, konfigurasi terbaik pada tahap *tuning* menghasilkan *accuracy* sebesar 0,9904 pada data *training*, 0,7592 pada data validasi Q4, dan 0,7579 pada data uji internal Q5. Hasil ini menunjukkan bahwa model mampu mempelajari pola data *training* dengan baik, namun performanya menurun pada data di luar *training*.

Penurunan performa tersebut juga terlihat pada Gambar 6.7. Pada grafik *training*, *train loss* terus menurun, sedangkan *validation loss* cenderung meningkat setelah *epoch* tertentu. Sementara itu, *validation accuracy* dan *test accuracy* meningkat hingga mendekati *epoch* terbaik. Model akhir dipilih pada *epoch* ke-22 karena memberikan performa validasi terbaik sebelum kecenderungan *overfitting* meningkat lebih jauh.

Tabel 6.5 Ringkasan Performa Model TabR

Tahap	Accuracy	Keterangan
Training tuning	0,9904	Performa sangat tinggi pada data latih
Validasi Q4	0,7592	Performa menurun pada data validasi
Test Q5	0,7579	Performa relatif stabil pada uji internal
Datatest eksternal	0,58	Performa menurun pada data baru

Berdasarkan Tabel 6.11, performa model pada data *training* jauh lebih tinggi dibandingkan validasi, uji internal, dan *datatest*. Kondisi ini menunjukkan bahwa model TabR mampu mempelajari pola fitur POC-ABS pada data latih, tetapi kemampuan generalisasinya terhadap data baru masih terbatas.

Hasil *confusion matrix training* dan *validation* pada Gambar 6.8 memperkuat kondisi tersebut. Pada data *training*, model menghasilkan jumlah prediksi benar yang dominan pada kedua kelas. Namun pada data *validation*, kesalahan prediksi meningkat, terutama pada kelas *anxiety* rendah yang cukup banyak diprediksi sebagai *anxiety* tinggi. Hal ini menunjukkan bahwa model lebih mudah mengenali pola tertentu pada data training dibandingkan pada data validation.

Berdasarkan Gambar 6.4, proses spotting mampu mendeteksi kandidat *apex* pada rentang *ground truth* CASME, tetapi juga menghasilkan beberapa puncak lain sebagai kandidat *event*. Hal ini menunjukkan bahwa

algoritma *spotting* bersifat sensitif terhadap perubahan kecil pada area ROI. Sensitivitas tersebut berguna agar pergerakan mikro tidak terlewat, tetapi beberapa kandidat *event* belum tentu merepresentasikan *anxiety* secara spesifik. Oleh karena itu, *spotting* digunakan sebagai tahap seleksi awal, sedangkan keputusan akhir tetap ditentukan melalui ekstraksi fitur POC-ABS dan klasifikasi menggunakan TabR.

6.2.3 Analisis Kesalahan Prediksi Model

Berdasarkan *confusion matrix datatest* pada Gambar 6.9, model masih menghasilkan kesalahan prediksi pada kedua kelas. Dari total 692 *event*, terdapat 395 *event* yang diprediksi dengan benar dan 297 *event* yang diprediksi salah. Kesalahan tersebut terdiri dari 144 *false positive*, yaitu *event anxiety* rendah yang diprediksi sebagai *anxiety* tinggi, dan 153 *false negative*, yaitu *event anxiety* tinggi yang diprediksi sebagai *anxiety* rendah. Ringkasan kesalahan prediksi ditunjukkan pada Tabel 6.12.

Tabel 6.6 Ringkasan Kesalahan Prediksi *Datatest*

Komponen	Jumlah
Total event datatest	692
Prediksi benar	395
Prediksi salah	297
False positive	144
False negative	153
Error rate anxiety rendah	44,72%
Error rate anxiety tinggi	41,35%

Berdasarkan Tabel 6.12, kesalahan prediksi model relatif seimbang antara *false positive* dan *false negative*, dengan selisih 9 event. Kondisi ini menunjukkan bahwa kesalahan model tidak hanya dominan pada satu kelas, tetapi terjadi pada kedua arah prediksi. *Error rate* pada kelas *anxiety* rendah sebesar 44,72%, sedangkan *error rate* pada kelas *anxiety* tinggi sebesar 41,35%. Dengan demikian, model sedikit lebih sering salah dalam mengenali event *anxiety* rendah, tetapi perbedaannya tidak terlalu besar.

Tabel 6.7 Statistik *Event* Benar dan Salah Prediksi

Status	Event	Mean Magnitude	Median Magnitude	Mean Durasi Frame	Median Durasi Frame
Benar	395	0,2624	0,1970	12,06	11
Salah	297	0,2329	0,1942	11,65	11

Analisis tambahan dilakukan dengan membandingkan karakteristik *event* yang diprediksi benar dan salah. Berdasarkan Tabel 6.13, event yang salah prediksi memiliki rata-rata *magnitude* sebesar 0,2329, sedikit lebih rendah dibandingkan *event* yang benar sebesar 0,2624. Selain itu, rata-rata durasi *event* yang salah adalah 11,65 *frame*, sedangkan *event* yang benar memiliki rata-rata durasi 12,06 *frame*. Perbedaan ini menunjukkan bahwa *event* dengan sinyal pergerakan yang lebih lemah atau durasi yang lebih pendek cenderung lebih sulit diklasifikasikan, meskipun perbedaannya tidak terlalu besar.

Selain faktor durasi dan *magnitude*, distribusi *magnitude* antar kelas juga menunjukkan adanya tumpang tindih. *Median magnitude anxiety* rendah sebesar 0,2363, sedangkan *median magnitude* anxiety tinggi sebesar 0,1681. Hal ini menunjukkan bahwa sebagian event *anxiety* rendah dapat memiliki intensitas pergerakan yang berada pada rentang *anxiety* tinggi, dan sebaliknya. Kondisi tersebut mendukung bahwa fitur pergerakan wajah saja belum sepenuhnya memisahkan kedua kelas secara tegas.

Berdasarkan temuan tersebut, kesalahan prediksi lebih mungkin disebabkan oleh kemiripan pola fitur antar kelas, *overlap* nilai *magnitude*, serta keterbatasan informasi pada sebagian *event* yang berdurasi pendek. Hal ini tidak menunjukkan bahwa fitur POC-ABS atau model TabR tidak dapat digunakan, tetapi menunjukkan bahwa karakteristik visual *anxiety* pada *micro-expression* bersifat halus dan tidak selalu muncul sebagai pola gerakan yang tegas. Oleh karena itu, hasil ini menjadi dasar bahwa kombinasi POC-ABS dan TabR sudah mampu membentuk model klasifikasi awal, tetapi masih memerlukan peningkatan pada kualitas *event*, jumlah variasi data, dan representasi fitur untuk memperkuat generalisasi model.

6.2.4 Evaluasi Metrik Precision, Recall, dan F1-Score

Evaluasi metrik dilakukan untuk melihat performa model secara lebih rinci, tidak hanya berdasarkan *accuracy*. Metrik *precision*, *recall*, dan F1-score digunakan untuk menilai kemampuan model dalam mengenali masing-masing kelas, yaitu *anxiety* rendah dan *anxiety* tinggi. Ringkasan performa model pada data internal dan datatest eksternal ditunjukkan pada Tabel 6.14.

Tabel 6.8 Ringkasan Perbandingan Performa Model

Split	Accuracy	Macro Precision	Macro Recall	Macro F1
Train	0,99	0,99	0,99	0,99
Validasi Q4	0,76	0,74	0,74	0,74
Test internal Q5	0,76	0,74	0,73	0,73
Datatest	0,58	0,57	0,57	0,57

Berdasarkan Tabel 6.14, model memperoleh performa tinggi pada data *training* dengan *accuracy* sebesar 0,99. Pada data validasi Q4 dan *test* internal Q5, *accuracy* menurun menjadi 0,76. Penurunan tersebut menunjukkan bahwa performa model pada data di luar *training* lebih rendah dibandingkan data latih, tetapi masih relatif stabil pada pengujian internal.

Pada *datatest*, *accuracy* model menurun menjadi 0,58 dengan *macro* F1-score sebesar 0,57. Penurunan ini menunjukkan bahwa model masih menghadapi tantangan dikarenakan kemiripan pola fitur antar kelas. Hasil metrik per kelas pada *datatest* ditunjukkan pada Tabel 6.15.

Tabel 6.9 Metrik Evaluasi *Datatest*

Kelas	Precision	Recall	F1-Score	Support
Anxiety rendah	0,54	0,55	0,55	322
Anxiety tinggi	0,60	0,59	0,59	370
Accuracy	-	-	0,58	692

Berdasarkan Tabel 6.15, kelas *anxiety* rendah memperoleh *precision* sebesar 0,54, *recall* sebesar 0,55, dan F1-score sebesar 0,55. Sementara itu, kelas *anxiety* tinggi memperoleh *precision* sebesar 0,60, *recall* sebesar 0,59, dan F1-score sebesar 0,59. Nilai *precision*, *recall*, dan F1-score yang saling berdekatan terjadi karena jumlah kesalahan prediksi pada kedua arah relatif seimbang, sehingga keseimbangan antara ketepatan prediksi dan kemampuan model dalam menemukan data aktual juga menghasilkan nilai F1-score yang tidak jauh berbeda dari *precision* dan *recall*. Nilai tersebut menunjukkan bahwa model sedikit lebih baik dalam mengenali *anxiety* tinggi dibandingkan *anxiety* rendah.

Namun, selisih performa antar kelas tidak terlalu besar. Hal ini menunjukkan bahwa model masih mengalami kesulitan dalam membedakan pola *anxiety* rendah dan *anxiety* tinggi secara konsisten. Dengan demikian, hasil *precision*, *recall*, dan F1-score menunjukkan bahwa kombinasi fitur POC-ABS dan model TabR dapat digunakan untuk klasifikasi *anxiety* berbasis *micro-expression*, tetapi kemampuan generalisasi pada datatest eksternal masih perlu ditingkatkan.

BAB VII. KESIMPULAN DAN SARAN

7.1 Kesimpulan

Berdasarkan hasil penelitian yang telah dilakukan, kesimpulan yang diperoleh adalah sebagai berikut

1. Fitur pergerakan halus yang diekstraksi menggunakan POC-ABS mampu merepresentasikan perubahan *micro-expression* pada area mata-alis kanan, mata-alis kiri, dan mulut. Fitur X, Y, *theta*, dan *magnitude* dapat membentuk representasi pergerakan pada *frame* dalam cakupan *event onset-apex-offset*, meskipun pola pergerakan *anxiety* rendah dan *anxiety* tinggi masih dapat saling tumpang tindih.
2. Model TabR dapat digunakan untuk mengklasifikasikan *anxiety* rendah dan *anxiety* tinggi berdasarkan data tabular hasil ekstraksi POC-ABS. Pada pengujian *datatest* eksternal, model memperoleh *accuracy* sebesar 0,58 dari 692 sampel, dengan *precision*, *recall*, dan *F1-score* masing-masing sebesar 0,54, 0,55, dan 0,55 pada kelas *anxiety* rendah, serta 0,60, 0,59, dan 0,59 pada kelas *anxiety* tinggi. Hasil tersebut menunjukkan bahwa model memiliki performa yang lebih baik dalam mengenali *anxiety* tinggi, namun kemampuan generalisasi terhadap data baru masih perlu ditingkatkan karena adanya variasi subjek, kandidat *event* hasil *spotting*, dan kemiripan pola pergerakan antar kelas.

7.2 Saran

Beberapa saran yang dapat dilakukan untuk pengembangan penelitian selanjutnya adalah sebagai berikut:

1. Mengembangkan metode *spotting micro-expression* yang lebih akurat dalam mengidentifikasi *event apex* sehingga *event* yang diperoleh benar-benar merepresentasikan pola ekspresi *anxiety* dan bukan gerakan wajah alami atau ekspresi *non-anxiety*.
2. Menambahkan jumlah dan variasi *dataset* dengan kondisi ekspresi, dan subjek yang lebih beragam agar model dapat mempelajari pola *micro-expression anxiety* secara lebih konsisten dan meningkatkan kemampuan generalisasi sistem.

Melakukan pengujian menggunakan model klasifikasi lain atau pendekatan *deep learning* berbasis temporal untuk meningkatkan kemampuan sistem dalam mempelajari hubungan perubahan ekspresi antar *frame* pada video *micro-expression*.

DAFTAR PUSTAKA

Abu, N. A. B., & Mohd, O. B. (2026). Cold-Start Generalization in Educational Interaction Data: Comparing Student-Wise and Question-Wise Splits with Probabilistic Calibration. 1(1).

Allaert, B., Bilasco, I. M., & Djeraba, C. (2022). Micro and Macro Facial Expression Recognition Using Advanced Local Motion Patterns. IEEE Transactions on Affective Computing, 13(1), 147–158. https://doi.org/10.1109/TAFFC.2019.2949559

American Psychiatric Association. (2022). Diagnostic and Statistical Manual of Mental Disorders: Fifth Edition Text Revision DSM-5-TR™. American Psychiatric Association. https://doi.org/https://doi.org/10.1176/appi.books.9780890425787

Archana, R., & Jeevaraj, P. S. E. (2024). Deep learning models for digital image processing: A review. Artificial Intelligence Review, 57(1), 11. https://doi.org/10.1007/s10462-023-10631-z

Arik, S. Ö., & Pfister, T. (2021). TabNet: Attentive Interpretable Tabular Learning. Proceedings of the AAAI Conference on Artificial Intelligence, 35(8), 6679–6687. https://doi.org/10.1609/aaai.v35i8.16826

Ben, X., Ren, Y., Zhang, J., Wang, S.-J., Kpalma, K., Meng, W., & Liu, Y.-J. (2021). Video-based Facial Micro-Expression Analysis: A Survey of Datasets, Features and Algorithms. IEEE Transactions on Pattern Analysis and Machine Intelligence, 1–1. https://doi.org/10.1109/TPAMI.2021.3067464

Bin Talib, K. H., Xu, K., Cao, Y., Ping Xu, Y., Xu, Z., Zaman, M., & Akhunzada, A. (2025). Micro-Expression Recognition Using Convolutional Variational Attention Transformer (ConVAT) With Multihead Attention Mechanism. IEEE Access, 13, 20054–20070. https://doi.org/10.1109/ACCESS.2025.3530114

Bruni, V., & Vitulano, D. (2021). A Fast Preprocessing Method for Micro-Expression Spotting via Perceptual Detection of Frozen Frames. Journal of Imaging, 7(4), 68. https://doi.org/10.3390/jimaging7040068

Chen, J., Zhao, X., Xiong, Z., & Liu, G. (2025). EEG-Based Micro-Expression Recognition: Flexible Brain Network Reconfiguration Supporting Micro-Expressions Under Positive Emotion. Psychology Research and Behavior Management, Volume 18, 781–796. https://doi.org/10.2147/PRBM.S506311

Chen, S., & Zhong, C. (2025). Mental health assessment model for college students based on facial expression recognition. Scientific Reports, 15(1), 45413. https://doi.org/10.1038/s41598-025-29461-w

Choirina, P., Rosiani, U. D., & Fitriani, I. M. (2022). Pengenalan Ekspresi Mikro Wajah Berdasarkan Point Feature Tracking Menggunakan Fase Apex Pada Database Ekspresi Mikro. Edu Komputika Journal, 9(1), 28–36. https://doi.org/10.15294/edukomputika.v9i1.56600

Coppini, S., Lucifora, C., Vicario, C. M., & Gangemi, A. (2023). Experiments on real-life emotions challenge Ekman's model. Scientific Reports, 13(1), 9511. https://doi.org/10.1038/s41598-023-36201-5

Dawel, A., Krumhuber, E. G., & Palermo, R. (2025). Faking It Isn't Making It: Research Needs Spontaneous and Naturalistic Facial Expressions. Affective Science. https://doi.org/10.1007/s42761-025-00320-1

Dewi, A. D. A., Muniandy, N. D., Subardjo, R. Y. S., Nurmaguphita, D., & Fauzia, F. R. (2025). The correlation between mental health and eating behavior in Indonesian college students: A cross-sectional study. JHeS (Journal of Health Studies), 9(1), 71–79. https://doi.org/10.31101/jhes.4023

Dong, Z., Wang, G., Lu, S., Li, J., Yan, W., & Wang, S.-J. (2022). Spontaneous Facial Expressions and Micro-expressions Coding: From Brain to Face. Frontiers in Psychology, 12, 784834. https://doi.org/10.3389/fpsyg.2021.784834

Ghafarfaraji, S. (2025). AI-based recognition of facial and micro-expressions for the diagnosis of mental and neurological disorders: A systematic review. BMC Psychiatry. https://doi.org/10.1186/s12888-025-07739-7

Gorishniy, Y., Rubachev, I., Kartashev, N., Shlenskii, D., Kotelnikov, A., & Babenko, A. (2024). TABR: TABULAR DEEP LEARNING MEETS

NEAREST NEIGHBORS. https://doi.org/https://doi.org/10.48550/arXiv.2307.14338

Han, B., Yoo, C.-H., Kim, H.-W., Yoo, J.-H., & Jang, J. (2023). Deep emotion change detection via facial expression analysis. Neurocomputing, 549, 126439. https://doi.org/10.1016/j.neucom.2023.126439

Huang, J.-T., Sharma, A., Sun, S., Xia, L., Zhang, D., Pronin, P., Padmanabhan, J., Ottaviano, G., & Yang, L. (2020). Embedding-based Retrieval in Facebook Search. Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2553–2561. https://doi.org/10.1145/3394486.3403305

Kagawade, V. C., & Angadi, S. A. (2021). Savitzky–Golay filter energy features-based approach to face recognition using symbolic modeling. Pattern Analysis and Applications, 24(4), 1451–1473. https://doi.org/10.1007/s10044-021-00991-z

Kim, M., Cho, Y., & Kim, S.-Y. (2022). Effects of diagnostic regions on facial emotion recognition: The moving window technique. Frontiers in Psychology, 13, 966623. https://doi.org/10.3389/fpsyg.2022.966623

Lai Yee, N., Asyraf Zulkifley, M., Harmoko Saputro, A., & Raihanah Abdani, S. (2022). Apex Frame Spotting Using Attention Networks for Micro-Expression Recognition System. Computers, Materials & Continua, 73(3), 5331–5348. https://doi.org/10.32604/cmc.2022.028801

Lei, J., Leigh, E., Charman, T., Russell, A., & Hollocks, M. J. (2024). Exploring the association between social camouflaging and self- versus caregiver-report discrepancies in anxiety and depressive symptoms in autistic and non-autistic socially anxious adolescents. Autism, 28(10), 2657–2674. https://doi.org/10.1177/13623613241238251

Li, J., Wang, S.-J., Wang, Y., Zhou, H., & Fu, X. (2025). Parallel Spatiotemporal Network to recognize micro-expression. Neurocomputing, 636, 129891. https://doi.org/10.1016/j.neucom.2025.129891

Li, Y., Wei, J., Liu, Y., Kauttonen, J., & Zhao, G. (2022). Deep Learning for Micro-Expression Recognition: A Survey. IEEE Transactions on Affective

Computing, 13(4), 2028–2046. https://doi.org/10.1109/TAFFC.2022.3205170

Liu, Y., Li, Y., Yi, X., Hu, Z., Zhang, H., & Liu, Y. (2022). Micro-expression recognition model based on TV-L1 optical flow method and improved ShuffleNet. Scientific Reports, 12(1), 17522. https://doi.org/10.1038/s41598-022-21738-8

Lu, S., Li, J., Wang, Y., Dong, Z., Wang, S.-J., & Fu, X. (2022). A More Objective Quantification of Micro-Expression Intensity through Facial Electromyography. Proceedings of the 2nd Workshop on Facial Micro-Expression: Advanced Techniques for Multi-Modal Facial Expression Analysis, 11–17. https://doi.org/10.1145/3552465.3555038

Möcklinghoff, S., Rapoport, O., & Neidhardt, E. (2024). Examining the role of post-event processing in test anxiety—Pilot testing in three student samples. International Journal of Educational Research Open, 7, 100363. https://doi.org/10.1016/j.ijedro.2024.100363

Nur Cahyo, D., Zulfia Zahro', H., & Vendyansyah, N. (2023). PENGENALAN EKSPRESI MIKRO WAJAH DENGAN EKSTRAKSI FITUR PADA KOMPONEN WAJAH MENGGUNAKAN METODE LOCAL BINARY PATTERN HISTOGRAM. JATI (Jurnal Mahasiswa Teknik Informatika), 7(1), 822–829. https://doi.org/10.36040/jati.v7i1.6167

Nurshabrina, M. F., Nadira Agustine, S., & Dariyo, A. (2024). Kecemasan Bersekolah pada Anak Pra-Sekolah. Action Research Literate, 8(7), 1784–1789. https://doi.org/10.46799/arl.v8i7.445

Oktamarin, L., Kurniati, F., Sholekhah, M., Nurjanah, S., Oktaria, S. W., Sukmawati, S., & Apriyani, T. (2022). GANGGUAN KECEMASAN (AXIETY DISORDER) PADA ANAK USIA DINI. Jurnal Multidisipliner Bharasumba, 1(02), 119–134. https://doi.org/10.62668/bharasumba.v1i02.192

Pasaribu, J., Yudistira, N., & Mahmudy, W. F. (2024). Tabular Data Classification and Regression: XGBoost or Deep Learning with Retrieval-Augmented Generation. IEEE Access, 1–1. https://doi.org/10.1109/ACCESS.2024.3518205

Rosiani, U. D., Choirina, P., Politeknik Negeri Malang, Muna, N., Jember, P. N., Mulyanto, E., Institut Teknologi Sepuluh Nopember, Sumpeno, S., Institut Teknologi Sepuluh Nopember, Purnomo, M., & Institut Teknologi Sepuluh Nopember. (2020). A Novel Approach on Motion Estimation for Micro-Expression Recognition Using Phase Only Correlation with All Block Search (POC-ABS). International Journal of Intelligent Engineering and Systems, 13(6), 546–559. https://doi.org/10.22266/ijies2020.1231.48

Sathyanarayanan, S. (2024). Confusion Matrix-Based Performance Evaluation Metrics. African Journal of Biomedical Research, 4023–4031. https://doi.org/10.53555/AJBR.v27i4S.4345

Schwarz, M., Chapman, K., & Häussler, B. (2022). Multilingual Medical Entity Recognition and Cross-lingual Zero-Shot Linking with Facebook AI Similarity Search. berLEF@SEPLN, (Computer Science, Medicine). https://ceur-ws.org/Vol-3202/livingner-paper12.pdf

Sejati, R. P. H., & Mardhiyyah, R. (2021). Deteksi Wajah Berbasis Facial Landmark Menggunakan OpenCV Dan Dlib. Jurnal Teknologi Informasi, 5(2), 144–148. https://doi.org/10.36294/jurti.v5i2.2220

Sharma, D., Singh, J., Sehra, S. S., & Sehra, S. K. (2024). Demystifying Mental Health by Decoding Facial Action Unit Sequences. Big Data and Cognitive Computing, 8(7), 78. https://doi.org/10.3390/bdcc8070078

Sinulingga, H. R., & Kong, S. G. (2023). Key-Frame Extraction for Reducing Human Effort in Object Detection Training for Video Surveillance. Electronics, 12(13), 2956. https://doi.org/10.3390/electronics12132956

Wang, J., Xu, S., & Zhang, T. (2024). A UNIQUE M-PATTERN FOR MICRO-EXPRESSION SPOT- TING IN LONG VIDEOS.

Wang, Y., See, J., Phan, R. C.-W., & Oh, Y.-H. (2015). Efficient Spatio-Temporal Local Binary Patterns for Spontaneous Facial Micro-Expression Recognition. PLOS ONE, 10(5), e0124674. https://doi.org/10.1371/journal.pone.0124674

Watson, D., Clark, L. A., Simms, L. J., & Kotov, R. (2022). Classification and assessment of fear and anxiety in personality and psychopathology.

Neuroscience & Biobehavioral Reviews, 142, 104878. https://doi.org/10.1016/j.neubiorev.2022.104878

Yang, S., & Berdine, G. (2024). Confusion matrix. The Southwest Respiratory and Critical Care Chronicles, 12(53), 75–79. https://doi.org/10.12746/swrccc.v12i53.1391

Yu, J., Zhang, Y., Zhao, G., He, P., Zhang, Z., Cai, Z., Liu, Q., Sun, J., & Liang, J. (2024). Micro-Expression Spotting Based on Optical Flow Feature with Boundary Calibration. Proceedings of the 32nd ACM International Conference on Multimedia, 11490–11496. https://doi.org/10.1145/3664647.3689142

Zhang, H., & Zhang, H. (2022). A Review of Micro-expression Recognition based on Deep Learning. 2022 International Joint Conference on Neural Networks (IJCNN), 01–08. https://doi.org/10.1109/IJCNN55064.2022.9892307

Zhao, G., Li, X., Li, Y., & Pietikäinen, M. (2023). Facial Micro-Expressions: An Overview. Proceedings of the IEEE, 111(10), 1215–1235. https://doi.org/10.1109/JPROC.2023.3275192